

Fondamenti di Informatica

Gualtiero Volpe
gualtiero.volpe@unige.it

Radoslaw Niewiadomski
radoslaw.niewiadomski@unige.it

Credits: prof. Volpe è l'autore della maggior parte dei slides

1

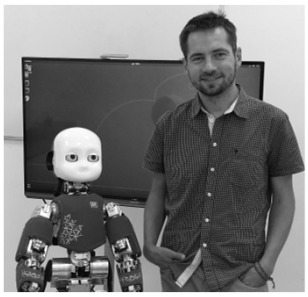


Docente:

- Radoslaw Niewiadomski

Informatica affettiva, robotica sociale, sistemi di riconoscimento delle emozioni, interazione multimodale, realtà virtuale, benessere psicologico

https://radoslawniewiadomski.github.io/



- Ricevimento su appuntamento
  - Online oppure VB stanza 18 oppure dopo la lezione
- Contatto: solo via e-mail: @unige,it

2

Tre assistenti:



gabriele.romano@edu.unige.it



marco.fedozzi@iit.it



alice.nardelli@edu.unige.it

3

Orario

- Lunedi, tre ore con la piccola pausa in mezzo
  - principalmente teoria
  - qualche quiz ed esercizio
- Mercoledì, due ore
  - esercizi con il supporto degli assistenti
- Codice degli esempi:
  https://github.com/radoslawniewiadomski/Fondamenti2025-2026

Attenzione: ci sono anche esempi con errori intenzionali!

4



Regole

- Materiale oltre ai libri:
  - internet, ci sono dei corsi fantastici
  - chatgpt, etc.
- Sfruttate le lezioni in presenza per discutere con il docente e i colleghi
- Non facciamo registrazioni deelle lezioni su Teams…

5



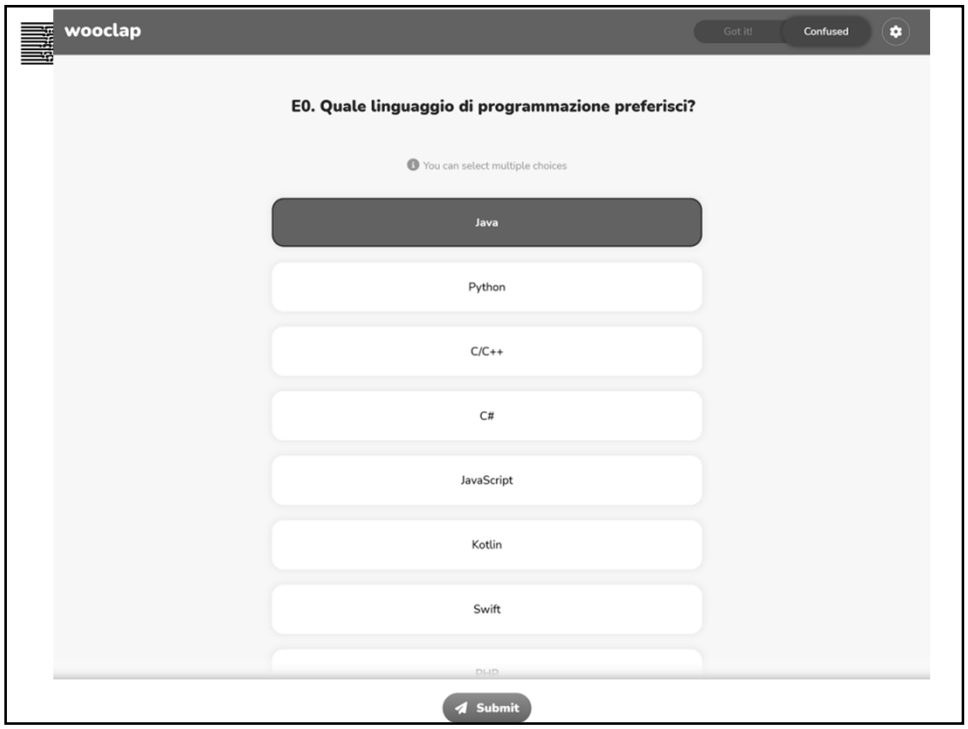
Wooclap

- Crea il tuo account:
  - usate l'email UNIGE
- Usa sempre lo stesso account:
  - 1 punto in più all'esame alla persona (o alle persone) che risponde/rispondono al maggior numero di domande
  - 2 punti in più all'esame per il maggior numero di risposte corrette (al vincitore), 1 punto al secondo posto
- I risultati non vengono pubblicati, ma i vincitori vengono contattati via e-mail all'indirizzo inserito nel sistema Wooclap prima dell'appello di giugno

6

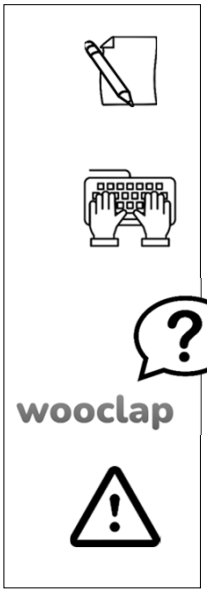

1
Vai a wooclap.com
2
Immettere il codice dell'evento nel banner superiore
Codice evento
GUITIQ
wooclap

7


wooclap
Got it!
Confused
E0. Quale linguaggio di programmazione preferisci?
You can select multiple choices
Java
Python
C/C++
C#
JavaScript
Kotlin
Swift
Submit

8

Icone sulle slide



- Il docente aggiunge la spiegazione con un disegno/schema
- Codice su *github (gia presente!)*

https://github.com/radoslawniewiadomski/Fondamenti2025-2026

wooclap

- Domanda/quiz
- Errore!

9

3. Programmazione avanzata in C++

Argomenti principali:

- puntatori e aritmetica dei puntatori
- allocazione dinamica della memoria
- template delle funzioni
- classi
- template delle classi
- strutture dati (introduzione)

Specifici a C++ (tranne le classi e strutture dati!):

- non presenti nella maggior parte dei linguaggi

10

Riassunto: una variabile

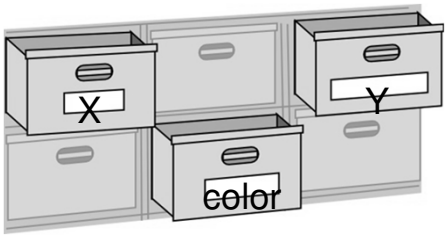
- è un contenitore di dati situato in una porzione di memoria destinata a contenere valori, suscettibili di modifica nel corso dell'esecuzione di un programma
- consente di registrare un'informazione e riutilizzarla in altri parti del programma senza dover riscriverla

11

Una variabile

- serve a SALVARE un dato nella memoria del computer
- Come ricordarsi dove ho salvato un dato?
- Ogni variabile ha un nome simbolico e unico
- Ogni volta che il nome di una variabile compare in un programma, verrà sostituito con il valore (= il dato memorizzato)





Cassetto di uno scaffale

12

Una variabile

- Abbiamo bisogno di dichiarare la variabile e specificare il suo tipo di dato

int punto_x;

tipo — nome

- Successivamente, possiamo memorizzare un valore nella variabile:

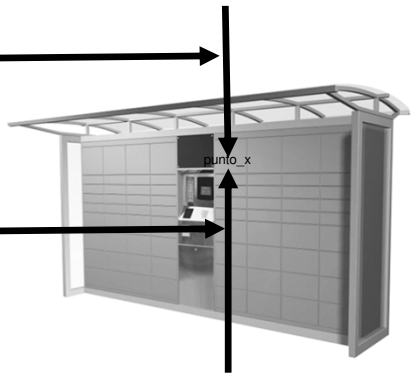
punto_x = 5;

nome — valore

- È possibile effettuare la dichiarazione e l'inizializzazione in una singola riga

int punto_x=5;

Da ora in poi, esiste una variabile chiamata 'punto_x' e può memorizzare valori interi.



Ora memorizzo qui un numero intero 5

13

Tipo riferimento

- Il meccanismo del riferimento (*reference*) consente di dare *nomi multipli* a una variabile.
  - viene usato l'operatore: *&*
- È come dire «genero un altro nome per la variabile già esistente», oppure un *alias*
- Entrambi i nomi possono essere utilizzati per cambiare il valore della stessa cella di memoria.
  - *come un «soprannome» che indentifica la persona*

```
int& ref_x…
int &ref_x…
```

15

Riferimento

Da ora in poi, questo posto di memoria «ha» due nomi 'punto_x' e 'ref_punto_x'

- Dichiariamo una variabile:

int punto_x;

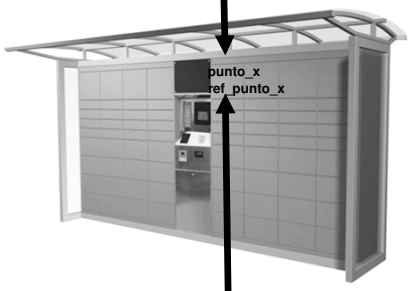
tipo — nome

- Specifichiamo il suo valore:

int punto_x=5;

- Successivamente, creiamo un riferimento:

int &ref_punto_x = punto_x;



16

Riferimento

- Il meccanismo del riferimento (*reference*) consente di dare *nomi multipli* a una variabile.
- Esempio:

```
#include <iostream>
using namespace std;
int main() {
     int x = 1;
     int& y = x; // y è un riferimento
     x++;
     cout << x << endl;   // stampa 2
     y++;
     cout << y << endl;   // stampa 3
     return 0; }
```

17

Riferimento: esempio completo

```
#include <iostream>
using namespace std;

int main()
{
     int x = 1;
     int& y = x;// y è un riferimento a x
     cout << x << endl;    // stampa 1
     cout << y << endl;    // stampa 1
     x++;
     cout << x << endl;    // stampa 2
     cout << y << endl;    // stampa 2
     y++;
     cout << x << endl;    // stampa 3
     cout << y << endl;    // stampa 3
     return 0;
}
```



E001

18

Riferimento: esempio completo

```
#include <iostream>
using namespace std;

int main()
{
    int x = 1;
    int& y = x;        // y è un riferimento a x
    cout << x << endl; // stampa 1
    cout << y << endl; // stampa 1
    x++;
    cout << x << endl; // stampa 2
    cout << y << endl; // stampa 2
    y++;
    cout << x << endl; // stampa 3
    cout << y << endl; // stampa 3
    return 0;
}
```



E001

19

Tipo riferimento

- Nelle dichiarazioni di variabili di tipo riferimento, l'inizializzazione è obbligatoria
- Non è permesso di dichiarare una variabile di rifermento senza associarla ad un'altra variabile



E002

```
#include <iostream>
using namespace std;
int main() {
    int x = 1;
    int z = 1;
    int& y; //ERRORE
    x++;
    cout << x << endl;
    return 0; }
```

error: 'y' declared as reference but not initialized

20

Tipo riferimento

- Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?



W003

```
#include <iostream>
using namespace std;
int main()
{
 int a=3;
 int c=7;
 int& b=a;
 b=c;
 cout << a <<", " << b;
}
```

21



Tipo riferimento

- Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
#include <iostream>
using namespace std;
int main()
{
     int a=3;
     int c=7;
     int& b=a;
     b=c;
     c--;
     b=b+2;
     a--;
     cout << a <<", " << b;
}
```



W204

22



Tipo riferimento

- Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
#include <iostream>
using namespace std;
int main()
{
    int a=3;
    int c=7;
    int& b=a;
    b=c;
    c--;
    b=b+2;
    a--;
    cout << a <<", " << b;
}
```



E204W

23

Passaggio per riferimento (richiamo, 1 semestre)

```
#include <iostream>
using namespace std;
void scambia(int &n, int &m)
{
   int t;
   t = n;
   n = m;
   m = t;
}

int  main() {
   int a = 1;
   int b = 2;
   scambia (a,b);
   cout<<a<<","<<b;
}
```

Perché uno dei due modi di passare i parametri attuali ai parametri formali si chiama «**passaggio per riferimento**»?

Perché non si crea una nuova variabile n, ma si fa riferimento (*reference*) a una variabile già esistente.

24

Introduzione ai puntatori

- In C/C++, esiste un altro modo per accedere ai valori nella memoria del computer
- Invece di utilizzare i nomi simbolici (come *punto_x*, ad esempio), possiamo specificare quale spazio nella memoria ci interessa
  - è come dire: «*Prendi l'oggetto che si trova nella prima posizione a destra della prima riga del mio scaffale», invece di usare il nome del cassetto*
- Per fare ciò, servono i **puntatori**, che memorizzano gli indirizzi di memoria in cui si trovano i dati di cui abbiamo bisogno

Ho bisogno del dato che si trova nel primo **cassetto** *a destra, nella prima riga…*



25

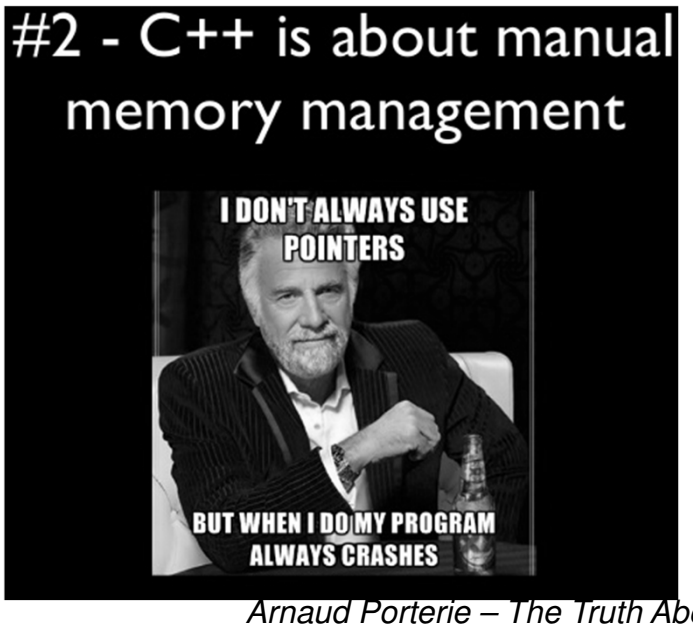
Puntatori

- Molto utilizzati in C++ e non presenti in altri linguaggi di programmazione
- i puntatori consentono di lavorare direttamente sulla memoria di un PC
- Tuttavia, non permettono di effettuare operazioni aggiuntive
- A volte possono essere rischiosi



- Perché potrebbero essere rischiosi?
- Sbagliando l'indirizzo, si potrebbe modificare involontariamente qualcos'altro nello spazio di memoria
  - lo spazio utilizzato da un altro programma potrebbe essere compromesso
  - il rischio di un arresto immediato del PC

26



Arnaud Porterie – The Truth About C++

27



Indirizzi di memoria

- Gli indirizzi di memoria sono numeri progressivi, di solito espressi in sistema esadecimale.
- Le locazioni di memoria sono numeri interi positivi, determinati dal numero di bit previsti dall'architettura del computer.
- I nostri PC sono normalmente a 64 bit; di conseguenza, anche i puntatori dovrebbero essere numeri a 64 bit.

28



Puntatori

- Il puntatore rappresenta l'indirizzo di memoria di una variabile o una funzione
- Il puntatore ci permette di accedere (leggere o scrivere) al contenuto di quella zona di memoria
- Attraverso un puntatore è possibile:
  - leggere o modificare un valore (variabile)
  - eseguire una funzione (poiché anche le funzioni sono memorizzate in un spazio di memoria)

29



Puntatori

- Una variabile di tipo puntatore rappresenta l'indirizzo di memoria di una variabile (di un certo tipo!) o di una funzione.
- I valori che tali variabili assumono sono pertanto indirizzi di locazioni di memoria.

Sintassi:

tipo id_or_init*

Esempio:

```
int* px;
```

```
int* px;

int *px;
```

30



Puntatori

- Come assegnare un valore a px?
- Nella maggior parte dei casi, noi programmatori non conosciamo l'indirizzo della variabile
  - Scrivendo: *int c=1;* non so dove si trova il *c* nella memoria del computer



- Il vantaggio di utilizzare i nomi simbolici (=le variabili) è di non dover tenere conto dei dettagli di gestione della memoria

31

Operatore & (address-of)

- L'operatore & (*address-of*) restituisce l'indirizzo della locazione di memoria in cui risiede l'espressione cui è applicato.
- Esempio:

```
int x = 10;
int* px = &x;
```

Il valore di **x** è 10

Il valore di **px** è l'indirizzo della locazione di memoria che contiene **10**

32

Commento 1

```
int x = 10;
int* px = &x;
```

- Il valore di px è l'indirizzo della locazione di memoria di *x*
- *px* è una variabile
  - si può stampare il suo contenuto (=indirizzo memorizzato in *px*) sullo schermo usando *cout !*

33

Commento 2: Attenzione al simbolo &

- Operatore logico
  if(a==0 && b==1)…
- Operatore di *riferimento*
  int y; int& x = y;
- Operatore di *address-of*
  int x;
  int* p = &x;
- Operatore sugli bit..

int x = 6; // 110 in binario
int y = 5; // 101 in binario
int z = x & y; // 100 → 4

34

Esempio

wooclap

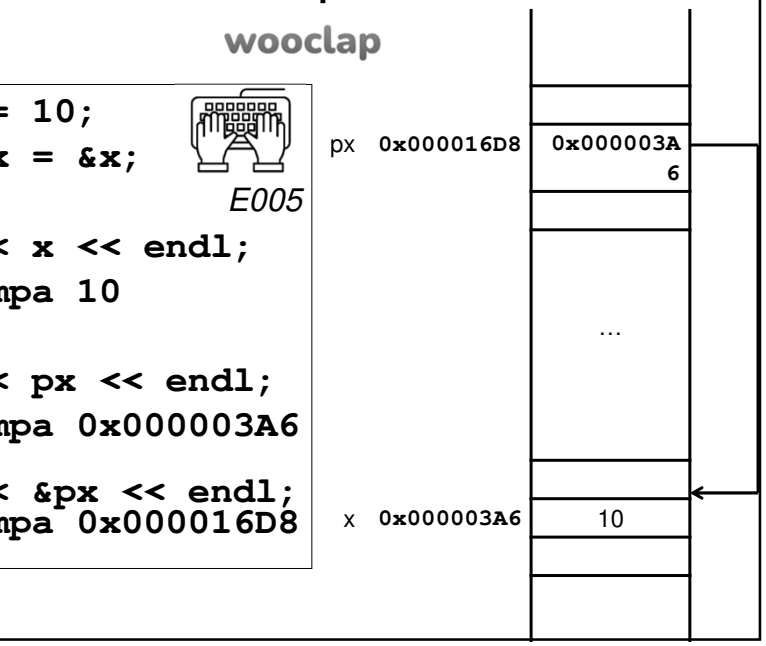
```
int x = 10;
int* px = &x;
```

E005

```
cout << x << endl;
// stampa 10

cout << px << endl;
// stampa 0x000003A6

cout << &px << endl;
// stampa 0x000016D8
```



35

Operatore di *dereference*

- Per accedere all'oggetto puntato da una variabile puntatore si applica l'operatore * (operatore di dereference, operatore di dereferenziazione)

- Esempio:

```
int x = 10;
int* px;
px = &x;       //px punta alla variabile x

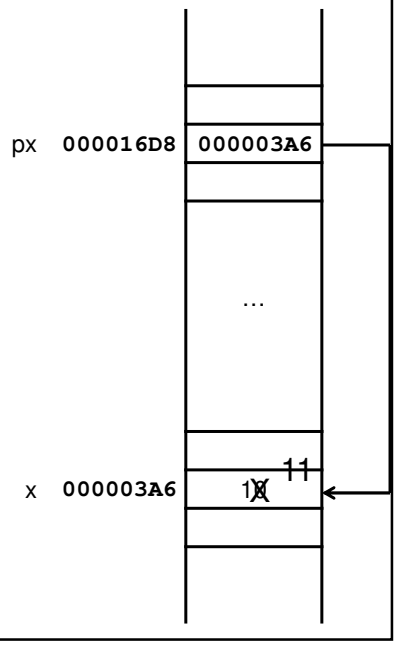
*px = x + 1; // assegna a *px ovvero
             // alla variabile puntata
             // da px il valore x + 1
```

36

Esempio

```
int x = 10;
int* px = &x;

*px = x + 1;
```



37

Accesso ai dati

- Accesso al valore attraverso ad un puntatore si chiama accesso indiretto

```
int x = 10;
x = 13; //accesso diretto
int* px = &x;
*px = 15; //accesso indiretto
```

38

Questi 4 esempi sono equivalenti...

```
int x = 10;
int* px = &x;
*px = x +5;
cout<<x;
```

```
int x = 10;
int* px = &x;
x = *px +5;
cout<<x;
```

```
int x = 10;
int* px = &x;
*px = x +5;
cout<<*px;
```

```
int x = 10;
int* px = &x;
x = *px +5;
cout<<*px;
```



E004

39

(Anche) questi 4 esempi sono equivalenti...

```
int x = 10;
int* px = &x;
*px = *px +5;
cout<<x;
```

```
int x = 10;
int* px = &x;
x = x +5;
cout<<x;
```

```
int x = 10;
int* px = &x;
*px = *px +5;
cout<<*px;
```

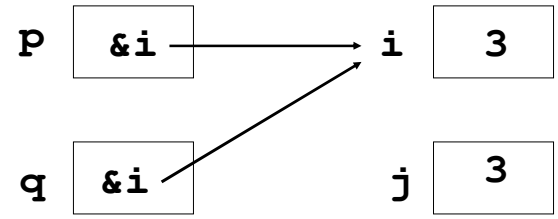
```
int x = 10;
int* px = &x;
x = x +5;
cout<<*px;
```



E004

40

Esempio di utilizzo dei puntatori




```
int i, j;
int* p; int* q;
p = &i; // p = indirizzo di i
```

assegna a `p` l'indirizzo di `i`

```
*p = 3; // equivale a i = 3
```

assegna il valore 3 alla variabile puntata da `p`

```
j = *p; // equivale a j = i
```

assegna a `j` il valore della variabile puntata da `p`

```
q = p;  //
```

assegna a q lo stesso indirizzo contenuto in p

41

Attenzione..

- al puntatore non inizializzato (che può puntare verso qualche spazio in memoria, dipende da compilatore…)
- Prova questo codice:

```
int main()
{
    int* px;
   *px = 1;
   cout<< *px;
}
```





E006

- Il programma si compila MA si può fermare durante l'esecuzione (dipende da compilatore)



- Un errore comune quando si utilizzano i puntatori è non assegnare loro un valore iniziale.

42

Memoria e puntatori

- A una variabile puntatore è associata una locazione di memoria destinata a contenere un indirizzo; non viene invece riservata memoria per l'oggetto a cui il puntatore farà riferimento.
- Lo spazio allocato a una variabile puntatore è sempre uguale, indipendentemente dal tipo dell'oggetto puntato.
- Pero…

43

Attenzione..



```
#include <iostream>
using namespace std;
struct t{
   int x; float y; float z;
};

int main(){
  int x = 10;
  t t1;
  int* px = &x;
  t* pt1 = &t1;

 cout<<px<<pt1<<endl;
 cout<<*px<<endl;

  px = pt1;
}
```



Errore!!

E007

Anche se i puntatori contengono indirizzi di memoria (ad esempio a 64 bit), non è consentita l'assegnazione diretta tra puntatori a tipi diversi!

44

Memoria e puntatori

- per inizializzare una variabile puntatore ad un indirizzo specifico (raro!) è necessario effettuare un casting (conversione esplicita):



```
int main()
{
  int *x = (int*) 6422076;
  cout << x;
}
```

E008

45



Memoria e puntatori

- per inizializzare una variabile puntatore ad un indirizzo specifico (raro!) è necessario effettuare un casting (conversione esplicita):



```
int main()
{
  int *x = (int*) 0x0012ff78;
  cout << x;
}
```

46



Puntatori *NULL*

- Un puntatore non inizializzato, come qualsiasi variabile, ha un valore aleatorio, ovvero punta ad una locazione di memoria presa a caso.
- Questo è pericoloso e pertanto è importante assegnare sempre un valore iniziale.
- Si può assegnare il valore iniziale 0 o NULL.
- Un puntatore il cui valore è 0 o NULL è detto puntatore nullo e non punta ad alcun oggetto.
- Esempio:

```
double* p = 0;
char* q = NULL;
```

47

Puntatori *NULL*

- Tentare di de-referenziare un puntatore nullo, provoca un errore a run-time! Ad esempio:

```
double* p = 0;
*p = 2.5; // ERRORE A RUN-TIME
```

- Quindi, prima di de-referenziare un puntatore occorre sempre verificare che non sia nullo:

```
if (q != 0)
    *q += 1;
```

48

Esempio di utilizzo dei puntatori

```
#include <iostream>
using namespace std;

int main()
{
    double z=0.5;
    double* pz=NULL;
    pz=&z;
    double* pz2=NULL;
    cout <<  pz << ", "  << pz2 << endl;
    pz2=pz;
    cout <<  pz << ", "  << pz2 << endl;

    cout << *pz << ", " << *pz2 << endl;

return 0;
}
```



E009



Cosa viene stampato quando eseguo la riga
1) verde,
2) rossa,
3) blue?

49

Esempio di utilizzo dei puntatori

```
#include <iostream>
using namespace std;

int main()
{
    double z=0.5;
    double* pz=NULL;
    pz=&z;
    double* pz2=NULL;
    cout <<  pz << ", "  << pz2 << endl;
    pz2=pz;
    cout <<  pz << ", "  << pz2 << endl;

    cout << *pz << ", " << *pz2 << endl;
    return 0;
}
```

E009



Cosa viene stampato quando eseguo la riga
1) verde,
2) rossa,
3) blue?

50

Esempio di utilizzo dei puntatori

```
#include <iostream>
using namespace std;
int main()
{
    double z=0.5;
    double y=0.5;
    double* p1=&z;
    double* p2=&y;

    cout <<  p1 << ", "  << p2 << endl;
    p2=p1;
    z+=1.0;
    cout << *p1 << ", " << *p2 << endl;
    return 0;
}
```

W010



Cosa viene stampato quando eseguo la riga blue?

wooclap

51

Esempio di utilizzo dei puntatori

```
#include <iostream>
using namespace std;
int main()
{
    double x=0.9, y=0.5, z=0.1;
    double* p1=&x;
    double* p2=&y;
    double* p3=&z;
    p2=p1;
    *p2=*p2+1;
    p2=p3;
    *p2=*p2+1;

    cout << x << ", " << y << ", " << z;
    return 0;
}
```



Cosa viene stampato quando eseguo la riga blue?

wooclap

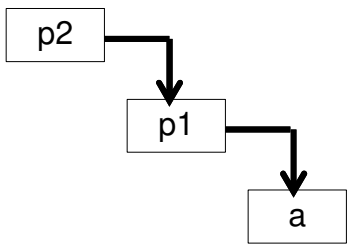
W011

52

Puntatore a puntatore

- Un puntatore può puntare a una variabile di qualunque tipo, quindi anche a un'altra variabile di tipo puntatore. In questo caso si ha un puntatore a puntatore.
- Esempio:

```
int a = 3;
int* p1 = &a;
int** p2 = &p1;
*p1 = 4;
**p2 = 5;
```



53

Esempio dei puntatori a puntatori



```
#include <iostream>
using namespace std;

int main(){
    int a = 3;
    int* p1 = &a;
    int** p2 = &p1;
    cout << p1 <<", "<< p2 << endl;
    cout << *p1 <<", "<< *p2 << endl;
    cout <<  **p2;
    **p2 = 5;
    cout << **p2;

    return 0;
}
```

E012

54

Esempio dei puntatori a puntatori







```
#include <iostream>
using namespace std;

int main(){
    int a = 3;
    int* p1 = &a;
    int** p2 = &p1;
    cout << p1 <<", "<< p2<< endl;
    cout << *p1 <<", "<< *p2<< endl;
    p2=p1;
    return 0;
}
```

E013

- E' possibile copiare in indirizzo da *int** a *int***?
- No!

Error: cannot convert 'int*' to 'int**' in assignment

55

Esempio dei puntatori a puntatori

```
#include <iostream>
using namespace std;
int main()
{
    double x=0.9, y=0.5, z=0.1;

    double* p1=&x;
    double* p2=&y;
    double* p3=&z;

    double** pp;
    pp = &p1;

    p2 = p3;
    p1 = p2;

    cout << **pp;
    return 0;
}
```

W212

Cosa viene stampato sullo schermo?

wooclap



56

Esempio dei puntatori a puntatori

```
#include <iostream>
using namespace std;

void scambia(int &n, int &m)
{
    int t;
    t = n;n = m;m = t;
}

int  main() {
    int a=1;
    int b=2;

    int* p1=&a;
    int* p2=&b;
    int** pp;
    pp = &p1;

    scambia (a,b);
    cout<<*p1<<","<<**pp;
}
```

W213

Cosa viene stampato sullo schermo?

wooclap



57



Lezione 2

58



Richiamo - Array

- Un array è una sequenza finita di elementi omogenei (dello stesso tipo).
- Sintassi:
  tipo id`[`*dim*`];`
  tipo id`[`*dim*`]={`*lista_valori*`};`
  tipo id`[]={`*lista_valori*`};`
- Esempi:

```
double a[25]; //array di 25 double
const int c = 2;
char b[2*c]={'a','e','i','o'};
char d[]={'a','e','i','o','u'};
```

61



Richiamo - Array

- Sugli array non sono definite né operazioni aritmetiche, né operazioni di confronto, né l'assegnamento.
- L'unica operazione definita è la selezione con indice (subscripting), ottenuta nella forma: *identifier* **[** *expression* **]**
  - L'identificatore è il nome dell'array;
  - Il valore dell'espressione, di tipo discreto, è l'indice dell'elemento cui si vuole accedere.
- Gli elementi di un array di dimensione **N** sono numerati da **0** a **N-1.**
- Esempio: **`a[3]=7`** assegna il quarto elemento.

62



Richiamo - Array

```
int main() {
    const int size = 6;
    int a[size] = {3, 2, 4, 1, 9, 2};
    for (int k = 1; k < size; k++)
    {
        cout << a[k] <<", ";
    }
}
```

63

Richiamo - Array

```
int main() {
    const int size = 6;
    int a[size] = {3, 2, 4, 1, 9, 2};
    for (int k = 1; k < size; k++)
    {
        cout << a[k] <<", ";
    }
}
```

Avete provato di scrivere

cout << a;



64

Puntatori e array

```
#include <iostream>
using namespace std;

int main()
{
    int a [] = {1,2,3};
    cout<<a;

    return 0;
}
```

E014

Questo codice non stampa il contenuto dell'array ma l'indirizzo della memoria

- Il nome di un array *corrisponde* a un puntatore al suo primo elemento.

65

Puntatori e array

```
#include <iostream>
using namespace std;

int main()
{
    int a [] = {1,2,3};
    cout<<*a;

    return 0;
}
```



E014

- De-referenziandolo si ottiene quindi il valore del primo elemento dell'array.

66

Puntatori e array

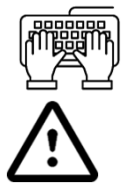
- Il nome di un array corrisponde a un puntatore al suo primo elemento.
- De-referenziandolo si ottiene quindi il valore del primo elemento dell'array.

- Nota: Il nome di un array si comporta come un puntatore costante al primo elemento dell'array; pertanto, il suo indirizzo non può essere modificato a run-time.

67

A proposito della nota..

```
#include <iostream>
using namespace std;
int main()
{
    int a [] = {1,2,3};
    int b [] = {4,5,6};
    a=b;
    return 0;
}
```



E015

Il nome di un array si comporta come un puntatore costante

Errore di compilazione!

68

Altro frequente errore..

```
#include <iostream>
using namespace std;
int main()
{
    char* a;
    cin>>a;

}
```





E016

- Importante differenza fra *puntatore a char* ed un *array di char*: creando un puntatore non si riserva lo spazio nella memoria per memorizzare una sequenza di caratteri.
- Il codice E016 può creare degli errori! (dipende da compilatore)

69



Aritmetica dei puntatori

- Operazioni applicabili ai puntatori:
  - Incremento unitario `p++;`
  - Decremento unitario `p--;`

    - mi sposto con puntatore di 1 posizione, p.e.:
      - se *p* è un puntatore ad un intero
      - mi sposto di 4 byte (4 byte servono per memorizzare 1 *int*)
    - utile per scorrere un array

70



Aritmetica dei puntatori

- Operazioni applicabili ai puntatori:
  - Incremento unitario `p++;`
  - Decremento unitario `p--;`
  - Incremento di un intero *n* `p += n;`
  - Decremento di un intero *n* `p -= n;`

    - mi sposto di *n* posizioni, p.e.:
      - se *p* è un puntatore ad un intero, e
      - n eguale a 5
      - mi sposto di 4x5 byte

71



Aritmetica dei puntatori

- Operazioni applicabili ai puntatori:
  - Incremento unitario `p++;`
  - Decremento unitario `p--;`
  - Incremento di un intero *n* `p += n;`
  - Decremento di un intero *n* `p -= n;`
  - Sottrazione `t = p - q;`
- Sottrazione di due puntatori restituisce il numero di elementi tra di essi (in questo caso fra p e q)

72



Aritmetica dei puntatori

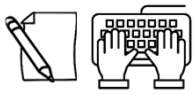
- Se un puntatore a un tipo T viene incrementato o decrementato di 1, il suo valore viene incrementato o decrementato di un numero pari alla dimensione in byte del tipo T.
- Incrementare (o decrementare) di un intero *n* un puntatore a un tipo T significa incrementarlo di un numero di byte pari a *n* × *dimensione*(T).
- Esempio:

```
int b[3] = {8, 9, 10};
int* p = b; p++;
```

p ora punta a b[1] ed è stato incrementato di 4 byte

73

Esempio





```
#include <iostream>
using namespace std;
int main()
{
   int b[3] = {8, 9, 10};
   int* p = b;
   cout <<"*p="<<*p<<", p= "<< p;
   p++;
   cout <<"*p="<<*p<<", p= "<< p;
   return 0;
}
```

E020

Cosa viene stampato quando eseguo la riga blue?

74

Esempio

E020

```
#include <iostream>
using namespace std;
int main()
{
   int b[3] = {8, 9, 10};
   int* p = b;
   cout <<"*p="<<*p<<", p= "<< p;
   p++;
   cout <<"*p="<<*p<<", p= "<< p;
   return 0;
}
```

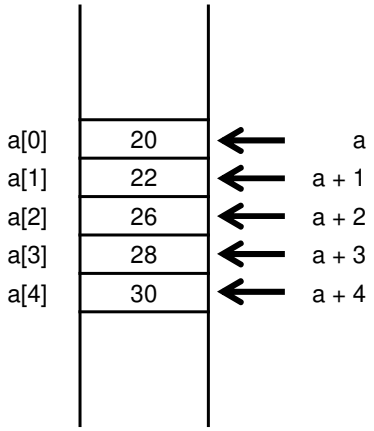


75

Puntatori e array

- Gli altri elementi dell'array si possono ottenere incrementando il puntatore associato al nome dell'array di una quantità pari all'indice di ciascun elemento.

```
int a[5] = {20,
22,26,28,30};
cout << a[2];
// stampa 26
cout << *(a+2);
// stampa 26
```



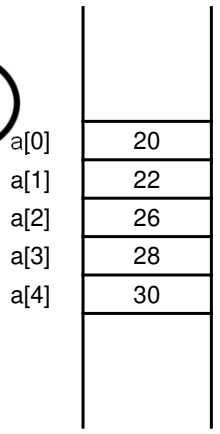
76

Puntatori e array

- Gli altri elementi dell'array si possono ottenere incrementando il puntatore associato al nome dell'array di una quantità pari all'indice di ciascun elemento.

```
cout << a[2];
// stampa 26
cout << *a+2;
// stampa ?
```

W017



wooclap

77

A proposito della nota precedente





```
#include <iostream>
using namespace std;
int main()
{
    int a [] = {1,2,3};
    int b [] = {4,5,6};

    a++;
    return 0;
}
```

E015

Il nome di un array si comporta come un puntatore costante al primo elemento dell'array.
Non è permesso di modificare 'a'

Errore di compilazione!

78

Esempio precedente



```
#include <iostream>
using namespace std;
int main()
{
   int b[3] = {8, 9, 10};
   int* p = b;
   cout <<"*p="<<*p<<", p= "<< p;
   p++;
   cout <<"*p="<<*p<<", p= "<< p;
   return 0;
}
```

E020

79

Errore!

```
#include <iostream>
using namespace std;
int main()
{
   int b[3] = {8, 9, 10};
   int* p = b;
   cout <<"*b="<<*b<<", b= "<< b;
   b++;
   cout <<"*b="<<*b<<", b= "<< b;
   return 0;
}
```

E020

- Non è possibile fare la stessa operazione usando direttamente il nome dell'array (b). Perché?
- Il nome di un array si comporta come un puntatore costante

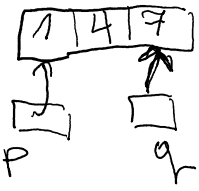
error: invalid array assignment

80

Esempio di sottrazione

```
#include <iostream>
using namespace std;
int main()
{
  int b[3] = {1, 4, 7};
  int* p = b;
  int *q = p+2;
  cout <<*q-*p;
  return 0;
}
```





W019

Cosa viene stampato quando eseguo la riga blue?

wooclap

81



Aritmetica dei puntatori

- L'aritmetica dei puntatori si può usare per scandire un array.
- Cosa fa questo programma?

```
int main() {
  char s[32]; cin >> s;
  char* p = s;
  while (*p != '\0') {
     if (*p >= 'A' && *p <= 'Z')
          *p = *p – 'A' + 'a';
     p++; }
  cout << s;
  return 0; }
```





E021

82



Aritmetica dei puntatori

- Come fare lo stesso compito con un *for*?

```
int main() {
  char s[32]; cin >> s;
  for (char *p=s; *p != '\0'; p++){
     if (*p >= 'A' && *p <= 'Z')
          *p = *p – 'A' + 'a';
     }
  cout << s;
  return 0;
}
```

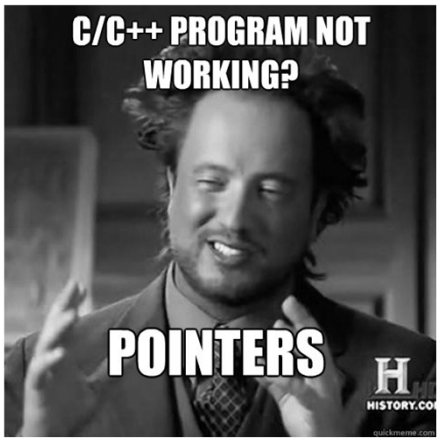
E021

83

Consiglio

- È più sicuro utilizzare indici [] al posto dei puntatori
- Tuttavia, spesso il codice esistente fa uso di puntatori per operare sugli array.
- Prova rifare l'esercizio E021 senza usare dei puntatori





84

Esempio – sottrazione

```
void swap(int &a, int &b) {
…

int main() {
    srand(time(NULL));

    int array[10];
    for (int i = 0; i < 10; ++i)  array[i] = i;

     for (int i = 0; i < 10; i++) {
        int randomIndex = rand() % 10;
        swap(array[i], array[randomIndex]);
    }


    int num1, num2;
    cout << "Inserisci due numeri compresi tra 0 e 9: ";
    cin >> num1 >> num2;

    int* ptr1 = NULL;
    int* ptr2 = NULL;

    for (int i = 0; i < 10; ++i) {
        if (array[i] == num1) ptr1 = &array[i];
        if (array[i] == num2) ptr2 = &array[i];
    }

    int distance = abs(ptr2 - ptr1);
..
```

Programma:

- crea un array che contiene 10 cifre in ordine casuale.
- trova due valori specificati dall'utente nell'array e calcola la distanza tra di essi usando puntatori

E060

85

Rischi dei puntatori

```
int main()
{
    int a = 4;
    int b = 8;
    int* pa=&a;

    cout<<*pa<<endl;
    pa++;
    cout<<*pa;
    return 0;
}
```





E054

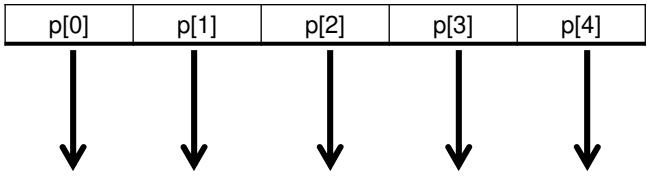
- Creo due variabili
- Non è garantito che le due variabili occupino spazi di memoria consecutivi!
- In questo caso il programma NON stamperà «8»
- Controlla l'indirizzi di «a» e «b»

86

Array di puntatori

- Se bisogna definire molti puntatori a uno stesso tipo, si può utilizzare un array di puntatori.
- Esempio:

```
int* p[5]; // array di 5
           // puntatori a int
```



87

Esempio

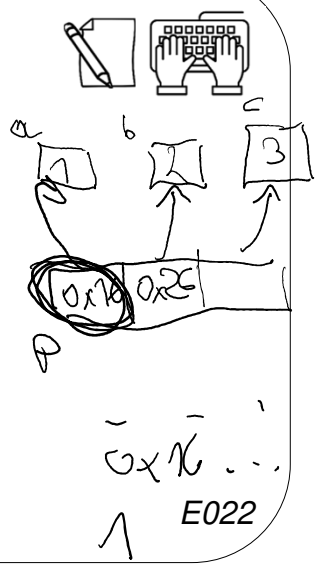
```
#include <iostream>
using namespace std;

int main() {
   int a = 1;
   int b = 2;
   int c = 3;

   int* p [3];

   p[0] =&a;
   p[1] =&b;
   p[2] =&c;

   cout<<p<<endl;
   cout<<*p<<endl;
   cout<<**p<<endl;
}
```



E022

88

Array di puntatori

E022

```
int main() {
    int a = 1, b = 2, c = 3;
    int* p [3];
    p[0] =&a;
    p[1] =&b;
    p[2] =&c;
….
```

Se p è un array di puntatori, **p è il valore a cui punta il primo elemento dell'array di puntatori.

89

Esempio

E022

```
#include <iostream>
using namespace std;

int main() {
   int a = 1;
   int b = 2;
   int c = 3;

   int* p [3];

    p[0] =&a;
    p[1] =&b;
    p[2] =&c;

    cout<<p<<endl;
    cout<<*p<<endl;
    cout<<**p<<endl;
}
```

90

Array di puntatori – operazioni sui valori

```
int main() {
    int a = 11; int b = 12; int c = 13;
    int* p [3];
    p[0] =&a; p[1] =&b; p[2] =&c;

    //cambio 3 valori
    for (int i=0; i < 3; i++) (*p[i])++;

    // stampo  3 VALORI
    for (int i=0; i < 3; i++) cout<<*p[i]<<", ";

    //stampo 3 indirizzi
    for (int i=0; i < 3; i++) cout<<p[i]<<", ";
    …
```





E018

91

Array di puntatori – operazioni sui valori

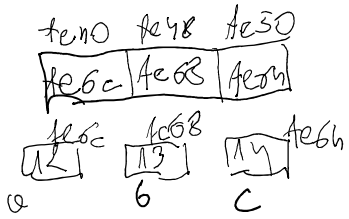
```
int main() {
   int a = 11; int b = 12; int c = 13;
   int* p [3];
   p[0] =&a; p[1] =&b; p[2] =&c;

   //cambio 3 valori
   for (int i=0; i < 3; i++) (*p[i])++;

   // stampo  3 VALORI
   for (int i=0; i < 3; i++) cout<<*p[i]<<", ";

   //stampo 3 indirizzi
   for (int i=0; i < 3; i++) cout<<p[i]<<", ";
    …
```

E018



Array di puntatori – domanda

```
int main() {
   int* p [3];

   for (int i=0;i < 3; i++){
      int temp;
      cin >> temp;
      p[i] = &temp;
   }

   for (int i=0;i < 3; i++){
       cout << *p[i];
   }
}
```

E221

Input: 1, 2, 3
Output: 3, 3, 3,

Perché?

Attenzione:

- *temp* è una variabile locale
- a ogni iterazione del ciclo viene distrutta e ricreata
- questo provoca un comportamento indefinito
- dopo il ciclo, tutti i puntatori p[i] contengono indirizzi non più validi



Cosa viene stampato sullo schermo quando eseguo questo programma?

```
int main() {
    int a = 1, b = 2, c = 3;
    int* p [3];
    p[0] =&a; p[1] =&b; p[2] =&c;

    int** q=p;

    q++;
    **q+=100;

   for (int i=0; i<3;i++)
     cout << *(p[i])<< ", ";
}
```



wooclap

W023

94



Cosa viene stampato sullo schermo quando eseguo questo programma?

```
int main() {
   int a=1, b=2, c=3;

    int* p [3];
    p[0]=&a; p[1]=&b; p[2]=&c;
    int** q=p;

    **q+=100;
    **(q+1)+=100;

   for (int i=0; i<3;i++)
     cout << *(p[i])<< ", ";
}
```



wooclap

W024

95

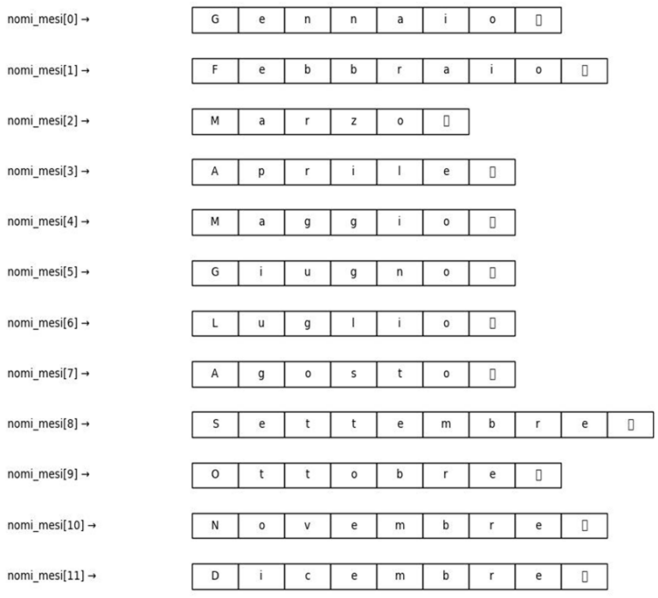


Array di puntatori

- Esempio: si possono usare gli array di puntatori per definire una sequenza di stringhe di lunghezza diversa.

```
char* nomi_mesi[12] =
 {"Gennaio", "Febbraio",
  "Marzo", "Aprile", "Maggio",
  "Giugno", "Luglio", "Agosto",
  "Settembre", "Ottobre",
  "Novembre", "Dicembre"};
```

96



97

Esempio: Stampare i nomi dei mesi



```
…
int main(){
// Iterazione attraverso l'array per stampare i mesi

    for (int i = 0; i < 12; ++i) {
        cout << i + 1 << ". " << nomi_mesi[i] << endl;
    }

    return 0;
}
```

E025

98

Puntatori a Costanti e Costanti Puntatore

Sintassi:

L'oggetto puntato è immodificabile

Puntatore a costante:

`const` *tipo*`*` *id_or_init*`;`

Costante puntatore: *L'indirizzo è immodificabile*

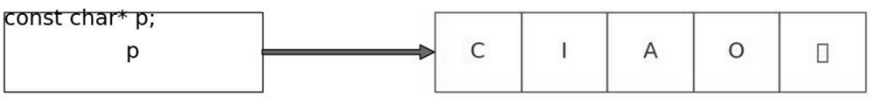
tipo`*` **`const`** *id = exp*`;`

Costante puntatore a costante:

`const` *tipo*`*` **`const`** *id = exp*`;`

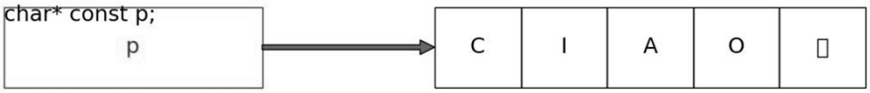
99

1) Puntatore a costante



✔ È possibile cambiare p (l'indirizzo)
✘ NON è possibile modificare il contenuto puntato (*p)

2) Puntatore costante



✘ NON è possibile cambiare p (indirizzo fisso)
✔ È possibile modificare il contenuto puntato (*p)

100

Puntatore a costante

- non posso modificare l'oggetto a cui il puntatore punta attraverso il puntatore (posso farlo direttamente usando il nome della variabile)

Costante puntatore

- non posso cambiare il valore del puntatore (cioè l'indirizzo a cui esso punta)

caso del nome di un array

101



Esempio di puntatore a costante

```
const int c1 = 3;
int c2 = 5;
const int* pc1 = &c1;  // ok
const int* pc2 = &c2;  // ok
pc2 = pc1; // ok
pc1 = &c2; // ok
c2 = 2;    // ok
*pc1 = 2;  // errore: non è
           // possibile modificare il
           //valore di c2 tramite pc1
```

Usando pc1 e pc2, il mio accesso alla memoria è solo in lettura. Non posso modificare i valori delle variabili a cui puntano.

102



Esempio di costante puntatore

```
int a, b;
int* const pa = &a;
*pa = 3;  // ok
pa = &b;  // errore: pa e' costante
```

103

Esempio di costante puntatore a costante

```
// Esempio di costante puntatore
// a costante
const int b = 2;
const int c = 3;
const int* const a = &c;
a = &b;  // errore
*a = 2;  // errore
c = 5;   // errore
```

104

Esempio di costante puntatore a costante

```
// Esempio di costante puntatore
// a costante

 const int b = 2;
 const int c = 3;
 const int* const a = NULL;
 a=&b;// errore
```

105

Quali 2 righe del programma sono sbagliate?

```
int main() {
    int a = 1, b = 2, c = 3, d = 4;
    int* const p3 = &c;
    int* const p1 = &a;
    const int* p2 =&b;
    const int* p4 =&d;

    a++;b++;c++;d++;
     p2=p4;
     p1=p3;
    (*p2)++;
    (*p1)++;
    cout<<a<<b<<c<<d;
}
```

wooclap



W026

106

Puntatore come argomento di una funzione

```
bool a_maggiore_b(int *a, int *b)
{
    if (*a > *b) return true;
    return false;
    }

int main(){
    int a = 30;
    int b = 20;
    int * pa=&a;
    int * pb=&b;
    cout<<a_maggiore_b(pa,pb);
    ..
}
```





Questa funzione riceve due puntatori e controlla se il valore puntato dal primo puntatore è maggiore del valore puntato dal secondo puntatore

E055

108

Una funzione restituisce un puntatore

```
int* a_maggiore_b(int *a, int *b) {
    if (*a > *b) return a;
    return b;
    }

int main() {
    int a = 30;
    int b = 20;

    int * pa=&a;
    int * pb=&b;
    int *maggiore = a_maggiore_b(pa,pb);
    …
}
```



Questa funzione restituisce il puntatore al valore maggiore fra le due

E056

109

Puntatore maggiore?

```
int* maggiore_indirizzo(int *a, int *b){
    if (a > b) return a;
    return b;
    }

int main() {
    int a = 20;
    int b = 30;

    int * pa=&a;
    int * pb=&b;
    int *maggiore = maggiore_indirizzo(pa,pb);
    …
}
```

Questa funzione trova l'indirizzo maggiore!

E057

110



Lezione 3

111



Passaggio di puntatori come parametri di funzioni

- Il passaggio del puntatore a una variabile come parametro di una funzione (passaggio per indirizzo), equivale di fatto a passare il parametro per riferimento.
- Il passaggio per indirizzo è il modo tipico di operare il passaggio per riferimento in C.
- In C++, di norma, si usa direttamente il passaggio per riferimento che è più comodo.

112

Passaggio per indirizzo



```
void incrp(int* p, int n)
{
  *p = *p + n;
}

void incrr(int& p, int n)
{
  p = p + n;
}

int main() {
     int a = 10;
     int b = 2;int c=3;
     incrp(&a, b);
     incrr(a, c);
     return 0;
}
```

Passaggio per indirizzo

Passaggio per rifermento

a ora vale 12

a ora vale 15

E058

113

Passaggio per indirizzo

```
void incrp(int* p, int n)
{
  *p = *p + n;
}

void incrr(int& p, int n)
{
  p = p + n;
}

int main() {
     int a = 10;
     int b = 2;int c=3;
     incrp(&a, b);
     incrr(a, c);
     return 0;
}
```

E058

114

Passaggio di un array

- Scrivere:
  int function1 (int a [], int n)…
  Vs.
  int function1 (int a, int n)..*
- In entrambi casi:
  - passiamo l'indirizzo dove si trova (il primo elemento di) un array:

    int a [] = {1,2,3,4,5};
    int x = function1 (a, 5);

  - funzione *function1(..,..)* opera nello stesso spazio di memoria della funzione *main()*
- Usando dei puntatori, possiamo accedere e modificare il contenuto dell'array

115

Esempio (1 semestre)

funzione *add1* aggiunge 1 ad ogni elemento di un array

```
#include <iostream>
using namespace std;

void add1(int a [], int size) {
    for (int i=0; i<size; i++)
        a[i] +=1;
}

int main() {
    int s[] = {1,2,3,4,5};
    add1(s,5);

    for (int i=0; i<5;i++)
      cout << s[i]<< ", ";
    return 0;
}
```



E027

116

Esempio

funzione *add1* aggiunge 1 ad ogni elemento di un array

```
#include <iostream>
using namespace std;

void add1(int a [], int size) {
    for (int* pi=a; pi<a+size; pi++)
        (*pi)+=1;
}

int main() {
    int s[] = {1,2,3,4,5};
    add1(s,5);

    for (int i=0; i<5;i++)
      cout << s[i]<< ", ";
    return 0;
}
```



E027

Risultato è lo stesso

117

Esempio

funzione *add1* aggiunge 1 ad ogni elemento di un array

```
#include <iostream>
using namespace std;


void add1(int* a, int size) {
    for (int* pi=a; pi<a+size; pi++)
        (*pi)+=1;
}

int main() {
    int s[] = {1,2,3,4,5};
    add1(s,5);

    for (int i=0; i<5;i++)
      cout << s[i]<< ", ";
    return 0;
}
```



E027

Risultato è lo stesso

118

Esempio



```
#include <iostream>
using namespace std;

void add1(int* a, int size) {
    int* stop = a + size;
    for (;a<stop;)
        (*a++)+=1;
}

int main() {

    int s[] = {1,2,3,4,5};
    add1(s,5);

    for (int i=0; i<5;i++)
      cout << s[i]<< ", ";
    return 0;
}
```

E059

A volte, un vantaggio dell'utilizzare un puntatore come parametro formale è che sia possibile usarlo direttamente per scorrere l'array.

Risultato è lo stesso

119

Uso dei puntatori con array multidimensionali

```
void add1(int a[][3]) {
        for (int j=0; j<3; j++)
            a[i][j] += 1;
}

int main() {
    int s[3][3] = {1,2,3,4,5,6,7,8,9};
    add1(s);
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            cout << s[i][j]<< ", ";

    return 0;
}
```

E067

```
for (int i=0; i<3; i++)
```

Esempio del primo semestre: **funzione *add1(..)* aggiunge 1 ad ogni elemento di un array 2D**

120

Uso dei puntatori con array multidimensionali

```
void add1(int **a) {
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            a[i][j] += 1;
}

int main() {
    int s[3][3] =
{1,2,3,4,5,6,7,8,9};
    add1(s);
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            cout << s[i][j]<< ", ";

    return 0;
```

E067

Errore: non è permesso utilizzare in puntatore a puntatore con array multidimensionali

121

una matrice 2x3 cioè un array di 2 array di 3 interi ciascuno.

Uso dei puntatori con array multidimensionali

```
void add1(int (*a)[3]) {
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            a[i][j] += 1;
}

int main() {
    int s[3][3] = {1,2,3,4,5,6,7,8,9};
    add1(s);
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            cout << s[i][j]<< ", ";

    return 0;
```

E067

Ricordiamoci che una matrice 3x3 = un array di 3 array di 3 interi ciascuno (Un array bidimensionale è un array di array**)**

Serve un **puntatore a un array 1D**

Inoltre bisogna specificare la dimensione dell'array

122



Uso dei puntatori con array multidimensionali

```
void add1(int (*a)[3]) {
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            a[i][j] += 1;
}

int main() {
    int s[3][3] = {1,2,3,4,5,6,7,8,9};
    add1(s);
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            cout << s[i][j]<< ", ";

    return 0;
}
```

E067

Serve un **puntatore a un array**

Bisogna specificare la dimensione dell'array

Se dimentico le parentesi?

Senza inserire le parentesi, il parametro formale diventa un array di puntatori

123



Uso dei puntatori con array multidimensionali

- Un array a due dimensioni è un array monodimensionale i cui elementi sono a loro volta degli array.
- Gli elementi degli array vengono memorizzati riga per riga.
- Abbiamo notato in passato che per passare un array a una funzione, è necessario specificare il numero di colonne, mentre non è necessario specificare il numero di righe.
- La ragione di questo è dovuta ai puntatori, in quanto il compilatore deve sapere il numero di colonne per saltare di riga in riga in memoria.

124



Cosa viene stampato sullo schermo quando eseguo questo programma?

```
void add1(int (*a)[3]) {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            **(a+i) = 1;
}

int main() {
    int s[3][3] = { {1,2,3}, {4,5,6}, {7,8,9}
};
    add1(s);

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cout << s[i][j] << ", ";

    return 0;
}
```

W214

wooclap



125



Cosa viene stampato sullo schermo quando eseguo questo programma?

```
void add1(int (*a)[3]) {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            *(*a+j) = 1;
}

int main() {
    int s[3][3] = { {1,2,3}, {4,5,6}, {7,8,9}
};
    add1(s);

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cout << s[i][j] << ", ";

    return 0;
}
```

W215

wooclap



126

Perché questo codice è sbagliato?

```
void add1(int (*a)[3]) {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            **a+j = 1;
}

int main() {
    int s[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };
    add1(s);

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cout << s[i][j] << ", ";

    return 0;
}
```

È sbagliato perché **a + j è una somma di due valori interi (s[0][0] e j) e non una locazione di memoria a cui si può assegnare 1

127

Puntatori a funzione

- Un puntatore a funzione punta alla prima istruzione di una funzione
  - dopo il che viene eseguito anche il resto della funzione
- La funzione può essere chiamata in 2 modi:
  - in modo diretto (vedi il primo semestre)
  - in modo indiretto, tramite il puntatore

128



Puntatori a funzione

- Sintassi:
  *tipo_di_ritorno (*id_puntatore) (<parametri>);*
- Esempio 1:
  int (*pf1) (char, int);
  - pf1 è puntatore ad una funzione che riceve un *char* ed un *int* e restituisce un *int*
  - può puntare, p.e., a questa funzione:

```
int funzione1 (char a, int b){
   return 0;}
```

129



Puntatori a funzione

- Sintassi:
  *tipo_di_ritorno (*id_puntatore) (<parametri>);*
- Esempio 2:
  float (*pf2)(int i, int j);
  - pf2 è puntatore ad una funzione che riceve due *int* e restituisce un *float*
  - può puntare, p.e., a questa funzione:

```
float funzione2 (int a, int b){
     return 0.1;}
```

130

Attenzione alle parentesi..

1. `int (*pf1) (char, int);`

- é un puntatore pf1 a una funzione che:
  - riceve un *char* ed un *int* e
  - restituisce un *int*

2. `int *f1 (char, int);`

- é una funzione *f1* che:
  - riceve un *char* ed un *int* e
  - restituisce un puntatore ad un intero

131

Esempio relativo al punto 2



E028

```
#include <iostream>
using namespace std;

int *secondo_elemento (int a[]){
  int *b = a;
  b++;
  return b;
}

int main(){
    int a []={1,2,4};
    cout << *secondo_elemento(a);
}
```

questa funzione restituisce il puntatore al secondo elemento dell'array

132

Puntatori a funzione

- Un puntatore a funzione viene inizializzato assegnando il nome di una funzione.
- Sintassi:

  id_puntatore = id_funzione;
- Per chiamare la funzione si de-referenzia il puntatore e si passano i parametri attuali nel modo usuale.
- Sintassi:
  *(*id_puntatore)(exp$_1$, …, exp$_n$);*

133

Puntatori a funzione: esempio 1

E029



```
double media (int a, int b) {
    return (a + b) / 2.0;
}

int main() {

    double (*pf)(int, int);

    pf = media; // inizializzazione

    int x, y;
    cout << "Inserire due numeri: ";
    cin >> x >> y;

    double m = (*pf)(x, y);
    cout << "Media = " << ;
    return 0;
}
```

puntatore a una funzione che riceve due parametri *int* e restituisce un *double*

chiamata la funzione puntata da pf passando x e y,
il valore *double* restituito viene assegnato alla variabile *m*

134

Puntatori a funzione: esempio 2

```
double somma(double x, double y){
  return x+y;
}
double differenza(double x, double y){
  return x-y;
}

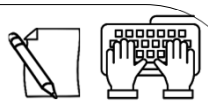
int main(){
    double (*pf) (double, double);
    pf = somma;

    double a,b;
    cin>>a>>b;

    if (a>b)
        pf = somma;
    else
        pf=differenza;

    cout << (*pf)(a,b);
}
```

E030



Un puntatore a una funzione è una variabile

Possiamo modificarne il suo valore

Il programma:
definisce due funzioni: somma e differenza, che eseguono rispettivamente l'operazione di somma e sottrazione di due numeri x e y

poi, utilizza un puntatore a funzione per scegliere dinamicamente quale funzione chiamare in base al confronto tra due numeri immessi dall'utente

135

Puntatori a funzione: esempio 2

```
double somma(double x, double y){
  return x+y;
}
double differenza(double x, double y){
  return x-y;
}
int main(){
 double (*pf) (double, double);

 pf = somma;

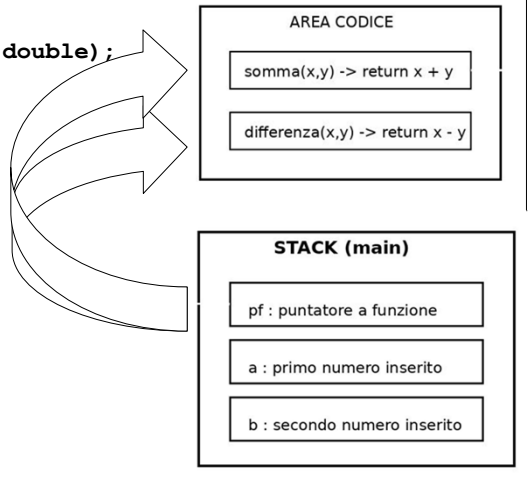
 double a,b;
 cin>>a>>b;

 if (a>b)
        pf = somma;
 else
        pf=differenza;

 cout << (*pf)(a,b);
}
```

E030





136

Quale è risultato di questo programma, se inserisco 4 e 4?

```
int *f1 (int *a){
  return a;
}

int *f2(int *b){
    b+=4;
  return b;
}

int *f3(int *c){
    c+=2;
    return c;
 }

int main() {
    int* (*pf) (int*) = NULL;
    int x [] = {1,2,3,4,5};

    int a, b;
    cin>>a>>b;

    if (a>b) pf=f1;
    if (a<b) pf=f2;
    if (a==b) pf=f3;

    cout << (*pf)(x);
    return 0;
}
```

W061



wooclap

137

Quale è risultato di questo programma, se inserisco 2 e 4?

```
int *f1 (int *a){
  return a;
}

int *f2(int *b){
    b+=4;
  return b;
}

int *f3(int *c){
    c+=2;
    return c;
 }

int main(){
    int* (*pf) (int*) = NULL;
    int x [] = {1,3,5,7,9};

    int a,b;
    cin>>a>>b;

    if (a>b) pf=f1;
    if (a<b) pf=f2;
    if (a==b) pf=f3;

    cout << *(*pf)(x);
    return 0;
}
```

W062



wooclap

138



Puntatori a funzione

- Un puntatore a funzione può essere passato come parametro ad un'altra funzione.
- La funzione che riceve in ingresso un puntatore a funzione, all'interno del suo corpo potrà chiamare la funzione puntata dal parametro attuale.
- Remainder: non è possibile di passare direttamente un nome della funzione come il parametro (possibile nei altri linguaggi)

139



Esempio

```
void test(int (*f)(int)) {
    …
    int k = (*f)(3);
}
```

- La parte in blue:
  - è un parametro della funzione *test*
  - è un puntatore ad una funzione che prende un intero ed restituisce un altro intero
- Il programmatore quando crea la funzione *test*
  - può anche non sapere quale funzione verrà eseguita all'interno di *test*
  - sa che questa funzione ha bisogno di un intero e che restituisce un altro intero

140

Puntatori a funzione: esempio



```
double inverso (int a) {
      return 1.0 / a;
}

double quadrato (int a) {
      return double(a * a);
}

double somma(int n, double (*f)(int)) {
      double s = 0.0;
      for (int i = 1; i <= n; i++)
            s += (*f)(i);
      return s;
}

int main() {
      cout << somma(5, inverso) << endl;
      cout << somma(5, quadrato) << endl;
      return 0;
}
```

E031

141

A proposito dell'esempio precedente…

- Abbiamo creato una funzione di somma che opera su qualsiasi altra funzione che accetta un argomento di tipo intero e restituisce un valore intero
  - è una alternativa a scrivere *n* funzioni per calcolare *n* somme diverse (e.g., *somma_inversi*, *somma_quadrato*)
- L'utilizzo dei puntatori alle funzioni è utile quando è presente una meta-azione, come ad esempio la somma, e vi sono sotto-azioni distinte
  - è facile aggiungere una nuova sotto-azione (qua nuova operazione che può essere aggregata con la *somma*)
  - meno errori (evitando di ripetere lo stesso codice più volte)

142

Lo stesso codice senza puntatori..

```
…
double somma_inverso (int n) {
        double s = 0.0;
        for (int i = 1; i <= n; i++)
               s += 1.0 /(i);
        return s;
 }
double somma_quadrato (int n) {
         double s = 0.0;
         for (int i = 1; i <= n; i++)
                s+= i*i;
         return s;
}

int main() {
         cout<<somma_inverso(5)<< endl;
         cout<<somma_quadrato(5)<< endl;
         return 0;
}
```

E031b

143

```
double valore_assoluto (int a) {
if (a < 0)
       return -a;
else
       return a;
}

double cubo(int a) {
       return a * a * a;
}

double doppio(int a) {
       return 2 * a;
}

double solo_pari(int a) {
if (a % 2 == 0)
       return a;
else
       return 0;
}
```

Tutte le funzioni che prendono un intero e restituiscono un double possono essere usare con il mio puntatore pf

E031

144

Esempio 2

un codice che utilizza puntatori a funzione per calcolare l'area di un quadrato, rettangolo e trapezio.

```
double areaQuadrato(double lato, double = 0, double = 0) {
   return lato * lato; }

double areaRettangolo(double base, double altezza, double = 0) {
   return base * altezza; }

double areaTrapezio(double base1, double base2, double altezza) {
   return ((base1 + base2) * altezza) / 2; }

double calcolaArea(double (*funzioneArea)(double, double, double), double a, double
b = 0, double c = 0) {
   return funzioneArea(a, b, c);
}

int main() {
   double lato = 4.0;
   double base = 5.0, altezza = 3.0;
   double base1 = 6.0, base2 = 4.0, h = 2.0;

   cout << "Area del quadrato: " << calcolaArea(areaQuadrato, lato ) << endl;
   cout << "Area del rettangolo: " << calcolaArea(areaRettangolo, base, altezza) << endl;
   cout << "Area del trapezio: " << calcolaArea(areaTrapezio, base1, base2, h) << endl;
}
```



E225

145

Array di puntatori a funzione

- Nel caso occorra scegliere una funzione all'interno di un insieme di funzioni, si possono usare gli array di puntatori a funzione.
- Sintassi:
  tipo_ritorno (**id_puntatore*[*dim*])(*<parametri>*)*;*
- Nota: tutte le funzioni puntate devono avere lo stesso tipo del valore di ritorno e gli stessi tipi dei parametri.

147



Array di puntatori a funzione

- Sintassi:
  tipo_ritorno (**id_puntatore*[*dim*])(*<parametri>*)*;*
- Esempio:
  `double (*pf[3])(int, int)`
- *pf* è un array di 3 puntatori a funzioni che ricevono 2 interi e restituiscono un *double*
- Tutte le 3 funzioni hanno la stessa lista dei parametri e ritorni

148



Esempio

```
double inverso (int a) {
  return 1.0 / a;
}

double quadrato (int a) {
  return double(a * a);
}

double assoluto (int a) {
   return abs(a);
}

double somma(int *a, double (*f)(int)) {
    double s=0.0;
    for (int i = 0; i < 5; i++)
           s += (*f)(*(a+i));
     return s;
}

int main() {
   int a [] = {1,-2,3,-4,5};
   double (*pf[3])(int) =  {inverso, quadrato, assoluto};

   for(int i=0;i<3;i++) cout << somma(a, pf[i]) << endl;
   return 0;
}
```

E064

La funzione ***somma*** prende un puntatore a un array di interi e un puntatore a una funzione che accetta un intero e restituisce un double, e restituisce un double

L'array ***pf*** di tre puntatori a funzione che accettano un argomento di tipo int e restituiscono un valore di tipo double.

149

Quale è risultato di questo programma, se inserisco 3 e 4?

```
double somma(double x, double y)     { return x + y;}
double differenza (double x, double y){return x - y;}
double prodotto (double x, double y)   {return x * y;}
double rapporto (double x, double y)   {
    if (y != 0.0) return x / y;
    else return 0.0;
}

int main() {
    const int dim = 4;
    double (*pf[dim])(double, double) =  {somma, differenza,
prodotto, rapporto};
    double op1, op2;
    char op;

    cin >> op1 >>  op2;
    cout << (*(pf+2))(op1, op2) << endl;
    return 0;
}
```



W216

150

Il codice equivalente

```
double somma(double x, double y) { return x + y;}
double differenza (double x, double y){return x - y;}
double prodotto (double x, double y){return x * y;}
double rapporto (double x, double y){
    if (y != 0.0) return x / y;
    else return 0.0;
}

int main() {
    const int dim = 4;
    double (*pf[dim])(double, double) =  {somma, differenza,
prodotto, rapporto};
    double op1, op2;
    char op;
    double (**ppf) (double, double) = pf;

    cin >> op1 >>  op2;
    cout << (*(ppf+2))(op1, op2) << endl;
    return 0;
}
```

W216

un puntatore a puntatore a funzione che riceve due double e restituisce un double

151

Esempio 2: calcolatrice

```
#include <iostream>
using namespace std;

double somma(double x, double y) {
      return x + y;
}
double differenza (double x, double y) {
      return x – y;
}
double prodotto (double x, double y) {
      return x * y;
}
double rapporto (double x, double y) {
      if (y != 0.0) return x / y;
      else return 0.0;
}
```

E032

Il resto del programma sul slide successivo

152

Esempio 2: calcolatrice

```
int main() {
  const int dim = 4;
  char operatori[dim] = {'+', '–', '*', '/'};
  double (*pf[dim])(double, double) =
    {somma, differenza, prodotto, rapporto};

  double op1, op2, res; char op;
  cin >> op1 >> op >> op2;

  for (int i = 0; i < dim; i++)
       if (op == operatori[i])
              res = (*pf[i])(op1, op2);

  cout << op1 << op << op2 << " = " << res;
  cout << endl;
  return 0;
}
```

un array di puntatori a funzione: pf contiene *dim* funzioni (inizializzato con *somma, differenza, prodotto, rapporto*) che prendono due *double* e restituiscono un *double*

E032

153



Puntatori a struttura

- I puntatori a variabili di tipo struttura si definiscono e si inizializzano nel modo usuale.
- Esempio:

```
struct persona {
    char nome[32];
    char cognome[32];
};
persona impiegato = {"Mario", "Rossi"};
persona* p = &impiegato;
```

154



Puntatori a struttura

- Dato un puntatore a struttura, per accedere a un campo della struttura puntata:
  - Si de-referenzia il puntatore e si utilizza l'usuale operatore `.`
    Esempio:
    ```
    cout << (*p).nome;
    cout << (*p).cognome;
    ```
    Nota: l'operatore `.` ha sempre la precedenza sull'operatore `*`: Le parentesi sono obbligatorie!
    ```
    cout << *p.nome;  // ERRORE!
    ```
  - Si utilizza l'operatore `->`
    Esempio:
    ```
    cout << p->nome;
    cout << p->cognome;
    ```

155

Esempio 1: Puntatori a struttura



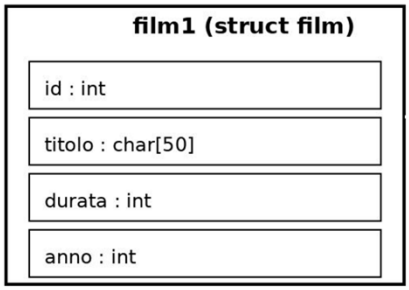
```
struct film {
     int id;
     char titolo[50];
     int durata;
     int anno;
};

int main() {
     film *p;
     film film1;

     cout << "Inserisci il titolo: ";
     cin >> film1.titolo;
     cout << "Inserisci durata: ";
     cin >> film1.durata;
     cout << "Inserisci anno: ";
     cin >> film1.anno;
     p=&film1;
     cout << "Titolo: " << p->titolo << endl;
     return 0;
}
```

E033

Esempio 1: Puntatori a struttura





p = &film1 -> p contiene l'indirizzo della struttura film1

p->titolo -> accesso al campo titolo tramite puntatore

Esempio 2: puntatori alla struttura come il parametro di 1 funzione

```
void stampaFilm(film* f) {
   cout << "ID: " << f->id << endl;
   cout << "Titolo: " << f->titolo << endl;
   cout << "Durata: " << f->durata << " minuti" << endl;
   cout<< "Anno: " << f->anno << endl;
}
```

Il resto del programma sul slide successivo

158

Esempio 2: puntatore alla struttura come il parametro



```
int main() {
    film elencoFilm[5];
    char titolo[50]; int durata; int anno;

    for (int i = 0; i < 5; ++i) {
        cout << "Inserisci il titolo " << i + 1 << "? ";
         cin >> titolo;
        cout << "Inserisci durata: "; cin >> durata;
        cout << "Inserisci anno: "; cin >> anno;
        elencoFilm[i].id = i + 1;
        elencoFilm[i].durata = durata;
        elencoFilm[i].anno = anno;
        strcpy(elencoFilm[i].titolo, titolo);
    }
    for (int i = 0; i < 5; ++i) {
        cout << "Film " << i + 1 << endl;
        stampaFilm(&elencoFilm[i]);
        cout << endl;
}
```

E034

159



Un array dei puntori alle strutture

- È possibile creare un array di puntatori a strutture,
- ad esempio, un array contenente i dati sugli studenti, oppure i dati sui film
- Sintassi:

```
nome_struttura* nomedelarray[numero];
```

160



Attenzione…

```
...
int main() {
    film* elencoFilm[5];
    for (int i = 0; i < 5; ++i) {
        film film1;

        cout << "Inserisci il titolo del film: " << i + 1 << "? ";
        cin >> film1.titolo;
        cout << "Inserisci durata: ";
        cin >> film1.durata;
        cout << "Inserisci anno: ";
        cin >> film1.anno;
        film1.id=i;
        elencoFilm[i]=&film1;
    }

    cout << "\n Titoli dei film:\n";
    for (int i = 0; i < 5; ++i) {
        cout << "Film " << i + 1 << ": " << elencoFilm[i]->titolo<< «, durata
"<<elencoFilm[i]->durata << endl;
    }
    return 0;
}
```



- Voglio una database dei film
- Il codice non da un errore, ma il risultato è diverso dalle aspettative
- Perché?

E035

161

Soluzione del problema è lungo



```
int main() {
film* elencoFilm[2];
film film1;
cout << "Inserisci il titolo del primo film: ";
cin >> film1.titolo;
cout << "Inserisci durata: "; cin >> film1.durata;
cout << "Inserisci anno: "; cin >> film1.anno;
film1.id=0;
elencoFilm[0]=&film1;

film film2;
cout << "Inserisci il titolo del secondo film: ";
cin >> film2.titolo;
cout << "Inserisci durata: "; cin >> film2.durata;
cout << "Inserisci anno: "; cin >> film2.anno;
film2.id=2;
elencoFilm[1]=&film2;

for (int i = 0; i < 2; ++i) {
        cout << "Film " << i + 1 << ": " << elencoFilm[i]->titolo << «,
durata: "<<elencoFilm[i]->durata << " minuti " << endl;
    }
    return 0;
}
```

E036

162

Lezione 4

163



- Ovviamente soluzione del slide precedente non è ottimale se il nostro dataset è composto di 1000 film.

- Allocazione dinamica di memoria (il topic della Lezione 4) permette di risolvere questo problema nel modo più efficace

164



Allocazione dinamica

- In C++ lo spazio di memoria occupato da una variabile (ad esempio un array) deve essere noto a priori, ovvero in fase di compilazione.
- Tale meccanismo prende il nome di allocazione statica della memoria.
- Non sempre, tuttavia è possibile stabilire a priori le dimensioni delle strutture dati.
- In C++ è possibile allocare memoria anche dinamicamente, ovvero durante l'esecuzione di un programma (allocazione dinamica).
- L'accesso alla memoria allocata in modo dinamico avviene tramite puntatori.

165



Quando può essere utile

- Quasi tutti i banchi di dati non hanno a priori noto il numero degli elementi:
  - registro delle macchine dei genovesi,
  - oggetti in vendita in un negozio,
  - studenti dell'università,

  ….
- Anche il nostro registro di film

166

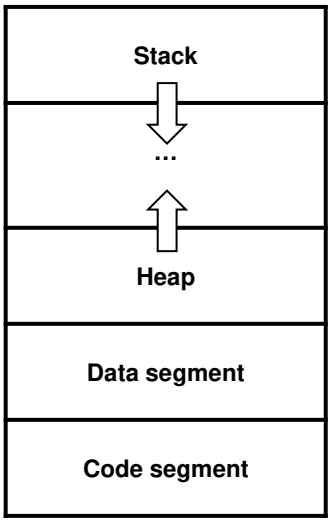


Allocazione dinamica

- Il sistema operativo assegna ai programmi specifici segmenti della memoria primaria:
  - Il code segment ospita le istruzioni del programma;
  - Il data segment ospita costanti e variabili che esistono durante l'intera esecuzione del programma (ad esempio variabili globali);
  - Lo stack contiene le variabili locali, i parametri delle funzioni e le copie dei registri per restituire il controllo alle funzioni chiamanti al termine delle funzioni chiamate;
  - La heap viene utilizzata per l'allocazione dinamica delle variabili.

167

Allocazione dinamica



Indirizzi di memoria “alti”

L'organizzazione della memoria dipende dal computer e dal sistema operativo. Solitamente, gli indirizzi dello stack decrescono, mentre gli indirizzi dell'heap crescono

Indirizzi di memoria “bassi”

168

Esperimento 1

```
int a=0; int b=0; float f=0;

int main() {
    int c=0;
    int d=0;
    float g=0;

    cout<<"variabili globali"<<endl;
    cout<<&a<<endl;
    cout<<&b<<endl;
    cout<<&f<<endl;

    cout<<"variabili locali"<<endl;
    cout<<&c<<endl;
    cout<<&d<<endl;
    cout<<&g<<endl;

    return 0;
}
```

E222

```
variabili globali
0x407030
0x407034
0x407038
variabili locali
0x61fe34
0x61fe30
0x61fe2c

Process returned 0 (0x0)   execution time : 1.131 s
Press any key to continue.
```

Data segment

Stack

Provate adesso sui vostri laptop!

169

Operatore `new`

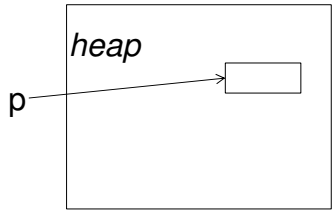
- L'operatore **`new`** alloca dinamicamente nella heap un'area di memoria atta a contenere un oggetto del tipo specificato e ritorna un puntatore a tale area di memoria.

- Sintassi:
  `new` *tipo*`;`
  `new` *tipo*`[`*dimensione*`];` (per gli array)

170

Esempio

```
int* p;
p = new int;
*p=10;
```



- dichiaro un puntatore
- allocco nella heap lo spazio per un intero, cioè 4 byte
  - creo una zona di memoria che può contenere un intero
- operatore *new* restituisce un puntatore che viene assegnato a *p*

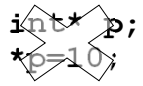
- attraverso il puntatore *p* posso accedere questo spazio di memoria e posso memorizzare un numero intero

- In pratica, per i tipi semplici all. dinamica viene utilizzata raramente. Perché?

171

Esempio

```
int* p;
p = new int;
*p=10;
```



- dichiaro un puntatore
- allocco nella heap lo spazio per un intero, cioè 4 byte
  - creo una zona di memoria che può contenere un intero
- operatore *new* restituisce un puntatore che viene assegnato a *p*

- attraverso il puntatore *p* posso accedere questo spazio di memoria e posso memorizzare un numero intero

- In pratica, <u>per i tipi semplici all. dinamica viene utilizzata raramente.</u> Perché?

172

Esperimento 2

```
int a=0;int b=0; float f=0;

int main() {
   int c=0; int d=0; float g=0;

   cout<<"variabili globali"<<endl;
   cout<<&a<<endl<<&b<<endl<<&f<<endl
   cout<<"variabili locali"   <<endl;
   cout<<&c<<endl<<&d<<endl<<&g<<endl

   int *p = new int;
   int *q = new int;
   float *r = new float;

   cout<<"allocazione dinamica " <<endl
   cout<<p<<endl;
   cout<<q<<endl;
   cout<<r<<endl;

   cout<<"altre variabili locali"<<endl;
   cout<<&p<<endl;
   cout<<&q<<endl;
   cout<<&r<<endl;

   return 0;
}
```

```
variabili globali
0x407030
0x407034
0x407038
variabili locali
0x61fe4c
0x61fe48
0x61fe44
allocazione dinamica
0xf31760
0xf31780
0xf31ad0
altre variabili locali
0x61fe38
0x61fe30
0x61fe28

Process returned 0 (0x0)   execution time : 0.435 s
Press any key to continue.
```

Data segment

Stack

Heap

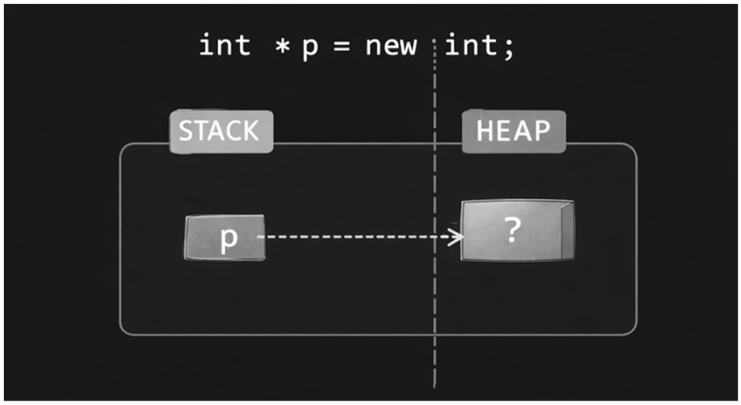
Stack

wooclap

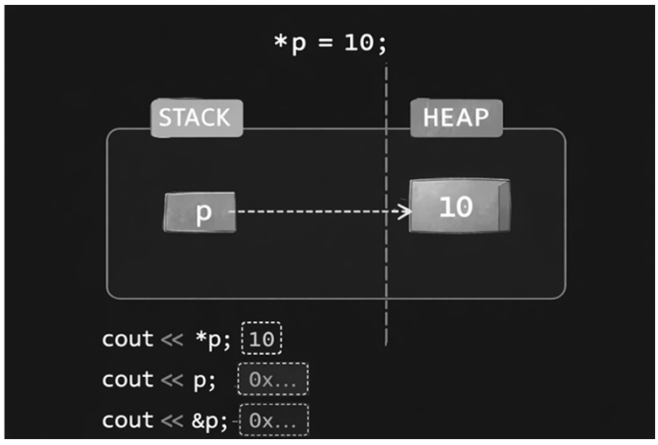
E222

Provate adesso sui vostri laptop!

173


int *p = new int;
STACK
HEAP
p
?

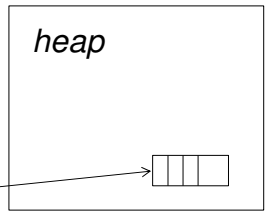
174


*p = 10;
STACK
HEAP
p
10
cout << *p; 10
cout << p; 0x...
cout << &p; 0x...

175

Esempio più interessante

```
char* stringa;
stringa = new char[30];
```



- dichiaro un puntatore ad un *char*
- allocco nella heap lo spazio per 30 caratteri
- operatore *new* restituisce un puntatore che viene assegnato a *stringa*

- attraverso il puntatore *stringa* posso accedere questo spazio di memoria e posso memorizzare una sequenza di 30 caratteri

176

Operatore `new`

- La dimensione dell'area di memoria da allocare si può definire a run-time. Ad esempio:

```
int n;
cout << "Inserire le dimensioni di una stringa";
cin >> n;
char* stringa = new char[n];
if (stringa != 0) cin >> stringa;
```

- Nota: se la memoria disponibile è esaurita, **`new`** restituisce un puntatore nullo. Prima di de-referenziare il puntatore restituito da **`new`**, occorre quindi assicurarsi che sia valido!

177



Deallocazione

- Un oggetto allocato dinamicamente resta allocato finché:
  - Non viene esplicitamente deallocato;
  - Il programma non termina.
- In C++ non esiste un meccanismo automatico per recuperare la memoria allocata e non più utilizzata (garbage collection).
- Pertanto, la memoria inutilizzata, ma non esplicitamente deallocata di norma non è più disponibile!

178



Operatore `delete`

- L'operatore **`delete`** dealloca l'area di memoria, precedentemente allocata in modo dinamico, puntata da un puntatore:
- Sintassi:
  `delete` *puntatore*`;`
  `delete[]` *puntatore*`;` (per gli array)
- Esempio:

```
int *p; char *stringa;
p = new int;
stringa = new char[30];
delete p;
delete[] stringa;
```

179



Vantaggi allocazione dinamica

- Il meccanismo di allocazione e deallocazione dinamica della memoria consente:
  - La gestione efficiente delle risorse di memoria in modo da allocare solo lo spazio realmente necessario;
  - La creazione di strutture dati fatte da un numero di elementi che possono variare durante l'esecuzione del programma.
- Occorre tuttavia usare con attenzione gli operatori **`new`** e **`delete`**!

180



Esempio 1: allocazione dinamica di array

```
#include <iostream>
using namespace std;
int main() {
  int n;
  cout << "Inserire il numero di elementi: ";
  cin >> n;
  if (n <= 0) return -1;
  int* a = new int[n];
  if (a == 0) return -1;
  for (int i = 0; i < n; i++)
      a[i] = i + 1;
  for (int j = 0; j < n - 1; j++)
      cout << a[j] << ", ";
  cout << a[n - 1] << endl;
  delete [] a;
  return 0;
}
```



E037

181

Esempio 2: allocazione dinamica di stringhe



```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
  char s1[256]; char s2[256]; char* ps3;
  cout << "Inserire due stringhe: ";
  cin >> s1 >> s2;
  int dim1 = strlen(s1);
  int dim2 = strlen(s2);
  ps3 = new char[dim1+dim2+1]; //allocazione
  if (ps3 == 0) return -1;
  strcpy(ps3, s1);
  strcat(ps3, s2);
  cout << "Stringa concatenata: " << ps3 << endl;
  delete[] ps3;                      //deallocazione
  return 0;
}
```

E038

182

Commento su delete

- Libera la memoria allocata dinamicamente in modo che possa essere riallocata mediante successive chiamate dell'operatore *new*
- Questo rende riutilizzabile la memoria puntata, ma non cancella il puntatore che può essere riutilizzato per puntare ad un'altra variabile successivamente allocata con *new*

183

Quale è risultato di questo programma, se inserisco 4?

```
int main() {
    int* v;
    int n;

    cout<<"Inserisci un numero positivo e pari";
    cin>>n;
    v = new int[n+1];
    v[0] = 0;

    for(int i = 1; i < n+1; i++) v[i] = v[i-1] + i;
    cout << v[2] << " ";

    delete[] v;
```

W297



184

Allocazione di diverse aree con lo stesso puntatore



```
int main() {
int* a;

    for (int k = 0; k < 10000; k++) {
        a = new int[k];
        if (a == NULL) {
            cout << "Memory allocation failed." << endl;
            return -1;
        }
        for (int i = 0; i < k; i++) a[i] = i + 1;
        for (int j = 0; j < k; j++) cout << a[j] << ", ";
    }
    delete[] a;
    return 0;
}
```

E039



- Posso usare lo stessa variabile di puntatore per allocare vari spazi nella memoria
- In questo codice c'è un errore. Dove?
- Controlla l'uso della memoria nel Task Manager

185

Allocazione di diverse aree con lo stesso puntatore



```
int main() {
int* a;

   for (int k = 0; k < 10000; k++) {
      a = new int[k];
      if (a == NULL) {
         cout << "Memory allocation failed." << endl;
         return -1;
      }
      for (int i = 0; i < k; i++) a[i] = i + 1;
      for (int j = 0; j < k; j++) cout << a[j] << ", ";
      }
   delete[] a;
   return 0;
}
```

E039

- Posso usare lo stessa variabile di puntatore per allocare vari spazi nella memoria
- In questo codice c'è un errore. Dove?
- Controlla l'uso della memoria nel Task Manager

186

Allocazione di diverse aree con lo stesso puntatore



```
int main() {
int* a;

   for (int k = 0; k < 10000; k++) {
      a = new int[k];
      if (a == NULL) {
         cout << "Memory allocation failed." << endl;
         return -1;
      }
      for (int i = 0; i < k; i++) a[i] = i + 1;
      for (int j = 0; j < k; j++) cout << a[j] << ", ";
      delete[] a;
      }
return 0;
}
```

- Versione corretta:
  - in ogni ciclo di for, operatore *delete* dovrebbe liberare la memoria allocata con *new*
  - Ricontrolla l'uso della memoria nel Task Manager

187

Le strutture statiche Vs dinamiche

```
const int n = 4;
int a[n];
```

```
int *p;
int n = 0;
cin>>n;
p = new int [n];
```

Strutture dinamiche, p.e., liste (vedremmo a maggio)

- Non posso cambiare le dimensioni di *a* dopo la compilazione del programma
- Una volta deciso il valore di *n* durante l'esecuzione del programma, non è più possibile modificare le dimensioni dell'array a cui punta *p*
- Posso effettuare una riallocazione dinamica:
  - creo un nuovo array con *n+1* elementi,
  - copio tutti gli elementi,
  - cancello il vecchio array.
- è sempre possibile aggiungere nuovi elementi,
- l'unico limite è la memoria disponibile.

maggiore flessibilità

188

Esempio di riallocazione

```
int main() {
    int* array = new int[3];// Allocazione per un array di 3 int
    for (int i = 0; i < 3; ++i) array[i] = i * 10;
    …
    int* newArray = new int[5]; // Riallocazione per un array di 5 interi

    // Copia dei valori dall'array originale al nuovo array
    for (int i = 0; i < 3; ++i)  newArray[i] = array[i];

    delete[] array; // Deallocazione della memoria dell'array originale

    // Aggiunta di valori aggiuntivi al nuovo array
    for (int i = 3; i < 5; ++i) newArray[i] = i * 10;

    for (int i = 0; i < 5; ++i) cout << newArray[i] << " ";

    delete[] newArray;
    return 0;
}
```

E069



189

Esempio di riallocazione

```
int main(){
    int* array1 = new int[3];
    for (int i = 0; i < 3; ++i) array1[i] = i * 10;
    cout << "Valori del array prima di riallocazione:" << endl;
    for (int i = 0; i < 3; ++i) cout << array1[i] << " ";

    int* newArray = new int[5];
    for (int i = 0; i < 3; ++i) newArray[i] = array1[i];
    delete[] array1;

    array1=newArray;
    for (int i = 3; i < 5; ++i) array1[i] = i * 10;
    cout << "Valori del nuovo array dopo la riallocazione:" << endl;
    for (int i = 0; i < 5; ++i) cout << array1[i] << " ";
    delete[] array1;
    return 0;
}
```



E069

191

Esempio 2: aggiungere 2 nome

```
#include <cstring> // Per la funzione strcpy
int main() {
    char* nome = new char[6];
    strcpy(nome, "Maria");

    cout << "Nome: " << nome << endl;

    char*  doppionome = new char[13]; // Allocazione di nuova memoria

    strcpy(doppionome, nome);
    delete[] nome;
    strcat(doppionome, " ");
    strcat(doppionome, "Carla");
    cout << "Nuovo nome: " << doppionome << endl;

    delete[] doppionome;
    return 0;
}
```

E070



192

Quale è risultato di questo programma?

```
int main() {
    int dim = 3;
    int* v = new int[dim];
    for (int i = 0; i < dim; i++) v[i] = i + 1;

    for (int turno = 0; turno < 5; turno++) {
        int* temp = new int[dim+1];
        for (int i = 0; i < dim; i++) temp[i] = v[i];
        temp[dim] = dim+1;
        delete[] v;
        v = temp;
        dim++;
    }
    int somma = 0;
    for (int i = 0; i < dim; i++) somma += v[i];
    cout << "Somma totale: " << somma << endl;
    delete[] v;
```



wooclap

W288

193

Allocazione dinamica delle strutture

- Struttura per descrivere un film
- allochiamo la memoria usando operatore *new*
- possiamo accedere o cambiare i valori usando l'operatore ->
- liberiamo la memoria

```
struct Film {
    int id;
    char titolo[50];
    int durata;
    int anno;
};
…
 Film* p = new Film;

 p->durata = 2;
 cout<<p->durata;

 delete p;
```

194

soluzione del problema del codice *E036*

```
int main() {
    const int numFilm = 3;
    Film* elencoFilm[numFilm];

    for (int i = 0; i < numFilm; ++i) {
        elencoFilm[i] = new Film;
        cout << "Inserisci il titolo del film " << i + 1 << "? ";
        cin >> elencoFilm[i]->titolo;
        cout << "Inserisci durata: ";
        cin >> elencoFilm[i]->durata;
        cout << "Inserisci anno: ";
        cin >> elencoFilm[i]->anno;
        elencoFilm[i]->id = i + 1;
    }
    cout << "Titoli dei film: ";
    for (int i = 0; i < numFilm; ++i) {
        cout << "Film " << i + 1 << ": " << elencoFilm[i]->titolo << endl;
    }
    for (int i = 0; i < numFilm; ++i) {
        delete elencoFilm[i];
    }
    return 0;
}
```



E040

195

```
struct Film {
    int id;
    char* titolo;
    int durata;
    int anno;
};

int main() {
    Film* p = new Film;

    int n;
    cout << "Quanto e' lungo il titolo del film?";
    cin >> n;
    p->titolo = new char[n+1];
    cout << "Inserisci il titolo: " ;
    cin >>  p->titolo;
    cout << "Inserisci durata: ";
    cin >> p->durata;
    cout << "Inserisci anno: ";
    cin >> p->anno;
    ….
```

E041

- Cerchiamo di migliorare il nostro database:
  - Ho bisogno di una struttura per memorizzare un titolo di lunghezza variabile
  - alloco din. la memoria
    - sia per la struttura *p*
    - che per la stringa *titolo*
  - come si libera la memoria in questo caso?



196

- Quando non ho più bisogno dell'elemento al quale punta p, devo liberare
  - sia la memoria allocata per memorizzare il titolo,
  - sia la memoria allocata per memorizzare l'elemento al quale punta p.

```
    ….
    delete [] p->titolo;
    delete p;

    return 0;
}
```



E041

197

```
#include <iostream>
using namespace std;

struct film
{
    int id;
    char* titolo;
    int durata;
    int anno;
};

int main()
{
    int n_film;
    cout << "Quanti film vuoi inserire? ";
    cin >> n_film;
```

Film* pfilm = new Film;

creata dinamicamente una singola struttura di tipo Film.

Film* films = new Film[n_film];

alloca dinamicamente un array di *n_film* strutture di tipo Film

Cosa devo scrivere se voglio un array di *n_film* puntatori a film?



Film pfilms = new Film*[n_film];**

198

Quale è risultato di questo programma?

```
#include <iostream>
using namespace std;

int main() {
    int** v = new int*[3];
    for(int i = 0; i < 3; i++) *v[i] = i + 5;
    for(int i = 0; i < 3; i++) cout << *v[i] << " ";
    for(int i = 0; i < 3; i++) delete v[i];
    delete[] v;
    return 0;
}
```

W286



199

Database di film: versione completa

Il programma permette all'utente di **inserire di film**, memorizza i loro dati usando **allocazione dinamica**, li **stampa** e poi **libera la memoria**.

L'utente sceglie quanti film vuole memorizzare nel database e, per ogni film, è libero di scegliere la lunghezza di ogni titolo.

Il programma:
- Chiede **quanti film** si vogliono inserire
- Crea **un array dinamico di puntatori a film**
- Per ogni film: 1) alloca una struttura film, 2) alloca memoria per il **titolo, 3)** legge **titolo, durata e anno**
- **Poi, stampa la lista dei film inseriti**
- Libera tutta la **memoria dinamica (delete)**:
  - prima i titoli
  - poi le strutture film
  - infine l'array dei puntatori.

200

```
int main() {

    int n_film;
    cout << "Quanti film vuoi inserire? ";
    cin >> n_film;

    film** films = new film*[n_film];

  for (int i = 0; i < n_film; ++i) {
        films[i] = new film;
        int n;
        cout << "Quanto è lungo il titolo del film " << i + 1 << "? ";
        cin >> n;
        films[i]->titolo = new char[n+1];  // Allochiamo memoria per il titolo
        cout << "Inserisci il titolo del film " << i + 1 << ": ";
        cin >> films[i]->titolo;
        cout << "Inserisci durata (in minuti) del film " << i + 1 << ": ";
        cin >> films[i]->durata;
        cout << "Inserisci l'anno di produzione del film " << i + 1 << ": ";
        cin >> films[i]->anno;
    }
for (int i = 0; i < n_film; ++i) {
        cout << "\nFilm " << i + 1 << ":" << endl;
        cout << "Titolo: " << films[i]->titolo << endl;
        cout << "Durata: " << films[i]->durata << " minuti" << endl;
        cout << "Anno: " << films[i]->anno << endl;
    }

for (int i = 0; i < n_film; ++i) {
        delete[] films[i]->titolo;
        delete films[i];
    }

delete[] films;
return 0;
}
```

Creo un array di puntatori per il numero di film specificato dall'utente

Alloco memoria per ogni film

Deallocazione della memoria



E223

Database di film: versione completa

Il nostro programma può essere ovviamente esteso.
Per esempio, possiamo permettere all'utente di correggere un eventuale errore nel titolo.

E224 è un esempio dell'estensione:

- **l'utente sceglie il film che vuole correggere e inserisce il nuovo titolo;**
- **il programma deve liberare la memoria prevista per il titolo vecchio e allocare nuovo spazio in memoria per il titolo nuovo.**

E224



Funzioni ricorsive

- Una funzione ricorsiva è una funzione che, in modo diretto o indiretto, chiama se stessa.
- Nella ricorsione diretta, una funzione contiene nel proprio corpo una chiamata a se stessa.
- Nella ricorsione indiretta, una funzione chiama, ad esempio, una seconda funzione che a sua volta richiama la prima funzione.
- Un processo ricorsivo deve avere una terminazione, altrimenti si ripete all'infinito.

203



Funzioni ricorsive: esempio

- Le successioni definite per ricorrenza si possono tipicamente implementare con funzioni ricorsive.
- Un esempio classico è il fattoriale:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \cdot (n-1)! & \text{se } n > 0 \end{cases}$$



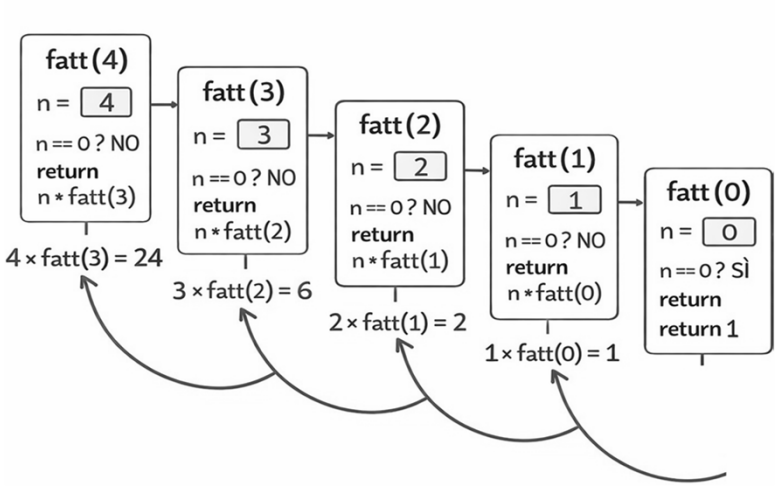
```
int fatt(int n)
{
 if (n == 0)
      return 1;
 return n * fatt(n - 1);
}
```

E042

204

```
int fatt(int n)
{
 if (n == 0)
      return 1;
 return n * fatt(n - 1);
}
```

4!



206

Funzioni ricorsive: esempio

Un altro esempio: minimo di un array

```
int min(int a, int b) {
  if (a < b) return a;
  else return b;
}

int min_rec(int v[], int i, int j) {
  if (i == j) return v[i];
  else return min(v[i], min_rec(v, i+1, j));
}

int min_in_array(int v[], int n) {
      return min_rec(v, 0, n-1);
}

int main() {
    int a[10] = {1, 3, 8, 2, 5, 0, 4, 6, 9, 7};
    int m = min_in_array(a, 10);
    cout << "min = " << m << endl;
    return 0;
}
```

E043

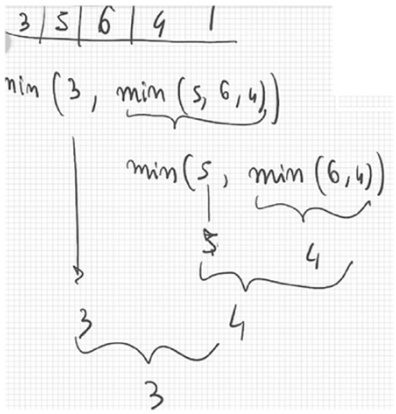
207

```
int min(int a, int b) {
  if (a < b) return a; else return b;
}
int min_rec(int v[], int i, int j) {
  if (i == j) return v[i];
  else return min(v[i], min_rec(v, i+1, j));
}

int min_in_array(int v[], int n) {

          return min_rec(v, 0, n-1);
}
```

3	5	6	4



208

La torre di Hanoi

```
void hanoi(int n, char A, char
B, char C)
{
  if (n > 0)
  {
    hanoi(n-1, A, C, B);
    hanoi(n-1, B, A, C);
  }
}

int main(int argc, char *argv[])
{
  hanoi(3, 'A', 'B', 'C');
  return 0;
}
```

W066

In questo caso quante volte viene chiamata la funzione *hanoi*?

wooclap



209

La torre di Hanoi

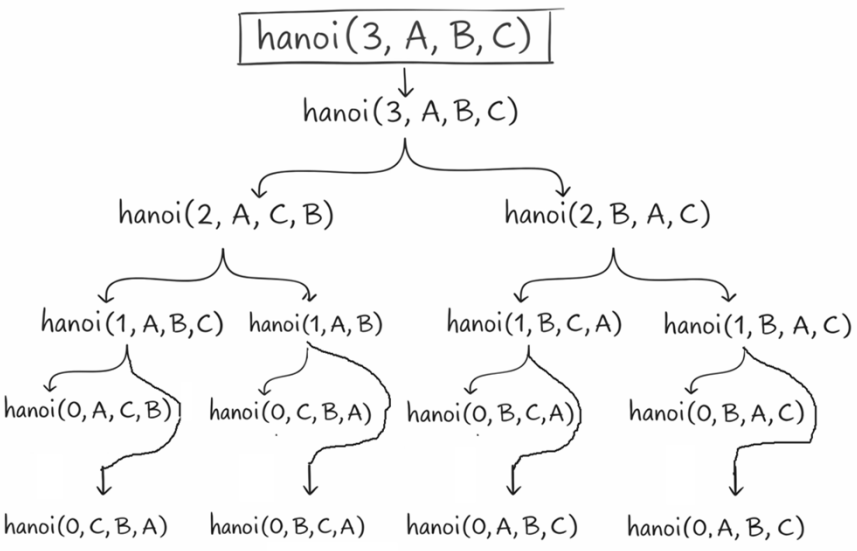
```
void hanoi(int n, char A, char B, char C)
{
  if (n > 0)
  {
    hanoi(n-1, A, C, B);
    hanoi(n-1, B, A, C);
  }
}

int main(int argc, char *argv[])
{
  hanoi(3, 'A', 'B', 'C');
  return 0;
}
```

```
hanoi(3, A, B, C)
│
├── hanoi(2, A, C, B)
│   │
│   ├── hanoi(1, A, B, C)
│   │   │
│   │   ├── hanoi(0, A, C, B)   → caso base (termina)
│   │   └── hanoi(0, B, A, C)   → caso base
│   │
│   └── hanoi(1, C, A, B)
│       │
│       ├── hanoi(0, C, B, A)   → caso base
│       └── hanoi(0, B, C, A)   → caso base
│
└── hanoi(2, B, A, C)
    │
    ├── hanoi(1, B, C, A)
    │   │
    │   ├── hanoi(0, B, A, C)   → caso base
    │   └── hanoi(0, A, B, C)   → caso base
    │
    └── hanoi(1, A, B, C)
        │
        ├── hanoi(0, A, C, B)   → caso base
        └── hanoi(0, C, A, B)   → caso base
```

W066

210



211

La Torre di Hanoi è un rompicapo composto da tre paletti e un certo numero di dischi di grandezza decrescente, che possono essere infilati in uno qualsiasi dei paletti. Il gioco inizia con tutti i dischi incolonnati su un paletto in ordine decrescente, in modo da formare un cono. Lo scopo del gioco è portare tutti i dischi su un paletto diverso, potendo spostare solo un disco alla volta e potendo mettere un disco solo su un altro disco più grande, mai su uno più piccolo (da Wikipedia)



212

Template di funzione

- I template di funzione (function template) forniscono un meccanismo per creare funzioni generiche, cioè funzioni in cui uno stesso parametro può assumere differenti tipi.
- Esempio: calcolo del valore assoluto

```
int abs(int x)
{
     if (x < 0) return –x,
     else return x;
}
```

- Ma *abs* si può applicare anche a *long, float* a e *double*. Posso evitare di scrivere quattro funzioni *abs*, una per ciascun tipo di dato?

213



Template di funzione

```
int abs(int x) {
  if (x < 0) return -x;
  else return x;
}
long abs(long x) {
  if (x < 0) return -x;
  else return x;
}
float abs(float x) {
  if (x < 0) return -x;
  else return x;
}
double abs(double x) {
  if (x < 0) return -x;
  else return x;
}
```

Posso evitare di scrivere quattro funzioni *abs*, che hanno tutte lo stesso identico corpo e differiscono solo per il tipo a cui si applicano?

214



Template di funzione

- I template di funzione permettono di definire un modello di funzione applicabile a diversi tipi di dato. Nel caso della funzione *abs*:

```
template <typename T>
T abs(T x)
{
  if (x < 0) return -x;
  else return x;
}
```

  - *template* e *typename* sono le parole chiavi
  - T viene chiamato *tipo generico*, a differenza di *int, char*, ecc., che vengono nominati *tipi specifici*

215



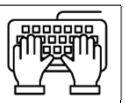
- Quando il compilatore analizzerà la chiamata a *abs*, genererà un codice in cui *T* verrà sostituito con il tipo effettivamente utilizzato
- Durante la compilazione, il compilatore «istanzia» il template e crea la funzione corretta
- Nulla ci vieta di non utilizzare template e scrivere a mano varie versioni della stessa funzione

216



Template di funzione

- Esempio:



```
int main() {
  int a; double z;
  cout << "Inserire un numero intero: ";
  cin >> a;
  cout << "Inserire un numero reale: ";
  cin >> z;
  int r = abs(a);
  cout << "|" << a << "| = " << r << endl;
  double s = abs(z);
  cout << "|" << z << "| = " << s << endl;
  return 0;
}
```

E044

- Nella prima chiamata di *abs* il tipo generico T viene sostituito con il tipo int, nella seconda chiamata di *abs* T viene sostituito con double.

217



Lezione 5

218



Template di funzione

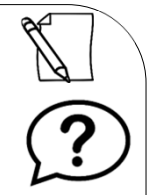
- Per riassumere la sintassi:

  template <*typename* T_1, …, *typename* T_M>
  tipo id **(** *$tipo_1$ id_1*, …, *$tipo_N$ id_N* **)** **{** *corpo* **}**

- Note:
  - La funzione generica deve essere scritta immediatamente dopo l'istruzione: *template <….>*
  - Al posto di **`typename`** si può talvolta trovare la parola chiave **`class`**.

219

Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?



```
#include <iostream>
using namespace std;

template <typename T>
T f1 (T x) {
  return 4.0/x;
}

int main() {
  cout << f1(3) << endl;
  return 0;
}
```

wooclap

W045

220

Template di funzione

– Le funzioni template possono avere più di un tipo generico. Ad esempio:

```
template <typename S, typename T>
S test(T a, S b, int c) {
      …
      corpo …
     }
```

- Due tipi generici possono corrispondere a due tipi specifici distinti oppure allo stesso tipo specifico.

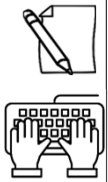
221

Esempio 1

```
#include <iostream>
using namespace std;
template <typename W, typename S>
S f2 (W x, S y)
{
    if( x > y) return 1.2;
    return 0.1;
}

int main(){
    double d1=4.2;
    double d2=4.1;
    cout << f2(d1,d2) << endl;
    return 0;
}
```

E052



In questo caso sia W che S sono *double*:

- La risposta è: 1.2

222

Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
#include <iostream>
using namespace std;

template <typename W, typename S>
S f2 (W x, S y)
{
    if( x > y) return 1.2;
    return 0.1;
}
int main(){
    double d1=4.2;
    int d2=4.8;
    cout << f2(d1,d2) << endl;
    return 0;
}
```

W053



wooclap



223

Errore 1. Perché?

no matching function for call to 'f2(double&)

```
#include <iostream>
using namespace std;

template <typename W, typename S>
S f2 (W x) {
  int z = x/2;
  return z;
}

int main() {
  double d1=3.9;
  cout << f2(d1) << endl;
  return 0;
}
```

E046

Problema di deduzione degli argomenti del template:

- nella funzione principale, quando viene chiamata *f2*, il parametro del template *S* non è esplicitamente specificato,
- deduzione degli argomenti del template fallisce.

224

Errore 2. Perché?

no matching function for call to 'f2(double&)

```
#include <iostream>
using namespace std;

template <typename W, typename S>
S f2 (W x) {
  int z = x/2;
  return z;
}

int main() {
 double d1=3.9;
 int d2 = f2(d1) << endl;
 return 0;
}
```

E046

Problema di deduzione degli argomenti del template:

- nella funzione principale, quando viene chiamata *f2*, il parametro del template *S* non è esplicitamente specificato,
- deduzione degli argomenti del template fallisce.

225

Per risolvere questo problema, si può specificare esplicitamente gli argomenti del template:

Ad esempio: **`int r = abs<int>(a);`**

Una soluzione:

```
#include <iostream>
using namespace std;

template <typename W, typename S>
S f2 (W x) {
  int z = x/2;
  return z;
}
int main() {
 double d1=3.9;
 int d2 = f2<int,double>(d1);
 cout<<d2;
 return 0;
}
```

E047

226

Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?



```
#include <iostream>
using namespace std;

template < typename S, typename T>
S f1 (T x) {
  return 4.0/x;
}

int main() {
  cout << f1<int,double>(3) << endl;
  return 0;
}
```

W048

wooclap



227

Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
#include <iostream>
using namespace std;

template < typename S, typename T>
S f1 (T x) {
  return  x/ 4.0;
}

int main() {
  cout << f1<double,int>(3.0) << endl;
  return 0;
}
```

W049



228

Template di funzione

- Esempio (ordinamento Bubble Sort generico):

```
template <typename T>
void BubbleSort (T v[], int n) {
      int i, j;
      T tmp;
      for (i = n – 1; i > 0; i--)
            for (j = 0; j < i; j++)
                  if (v[j] > v[j+1]) {
                        tmp = v[j];
                        v[j] = v[j+1];
                        v[j+1] = tmp;
                  }
}

…
```

E050

229



Template di funzione

- Attenzione: l'istanziazione di una funzione template richiede che le operazioni eseguite su variabili del tipo generico T siano definite per il tipo specifico che sostituisce T. Ad esempio:

```
struct Impiegato {
  char nome[32];
  char cognome[32]; };

int main(){
  int a[5] = {3, 2, 6, 1, 4};
  Impiegato imp[2] =
    {{"Mario","Rossi"},{"Paola","Bianchi"}};
  BubbleSort(a, 5);          // OK!
  BubbleSort(imp, 2);        // ERRORE!
return 0; }
```

```
no match for 'operator>'
(operand types are
'Impiegato' and
'Impiegato')
```



E051

230



4. Programmazione modulare e tipi di dato astratti

231

Programmazione procedurale (primo semestre)

- Un programma, avendo un database di film, esegue le seguenti azioni attraverso le apposite funzioni, p.e.:
  - trova tutti i film il cui titolo inizia con una lettera specificata come parametro di ingresso e stampa il titolo a schermo,
  - trova il film più lungo,
  - ……
- Per scriverlo, abbiamo bisogno di:
  - una struttura dati,
  - funzioni che operano sui film (ovvero sulle variabili definite nella struttura).

232

Programmazione procedurale (primo semestre)

- La programmazione procedurale consiste nel suddividere un problema in funzioni, le quali vengono poi utilizzate all'interno della funzione principale (*main*):
  - si identificano i compiti,
  - si scrive una funzione per risolvere ciascun compito,
  - queste funzioni vengono richiamate dalla funzione *main*.
- La gestione dei dati (variabili, array, ecc.) viene principalmente realizzata in *main(),* e i dati vengono trasmessi alle funzioni come parametri.

233

- Ciò che abbiamo visto finora è teoricamente sufficiente per scrivere qualsiasi programma
  - programmi diventano sempre più complessi e lunghi
- Per semplificare la programmazione, si propone di adottare un paradigma chiamato programmazione modulare:
  - suddivisione del programma in singoli moduli,
  - indipendenza dei moduli tra loro,
  - interazione minima di ciascun modulo con il mondo esterno.
- In pratica: *programmazione ad oggetti* (Object-Oriented Programming, OOP)

234

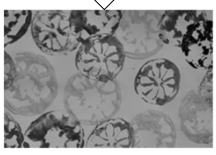
Concetto di una classe ed di un oggetto





- La **classe** è un modello e gli **oggetti** sono entità costruite seguendo quel modello: da un unico modello (classe) possono essere creati molti oggetti.
- La classe *Casa* (o *Auto*) è un modello, non rappresenta alcuna casa concreta (auto).
- Utilizzandola, possiamo definire diversi oggetti (o istanze della classe), ad esempio:
  - VillaBonino13_Genova,
  - Fiat_500_biancaAA001AA, ecc.





© studioaccendo, 123RF Free Images

235



Struttura (1 semetre)

- Una struttura è una collezione eterogenea di dati, ognuno con il proprio tipo.
- Quando dichiaro una nuova struttura *X*, è come se stessi definendo un nuovo tipo.
- Posso dichiarare molte variabili di tipo struttura *X*.
- Le classi possono essere considerate come un'evoluzione del concetto di struttura.

236



Classi

- In C++ una classe definisce un tipo di dato personalizzato, specificato dal programmatore.
- Come tale, la classe identifica l'insieme di valori che il tipo può assumere e le operazioni che possono essere applicate a tali valori.

- Una classe contiene pertanto:
  - dati (detti anche attributi o variabili membro)
  - funzioni (dette anche metodi o funzioni membro) che manipolano i dati, il tutto contenuto in un'unica entità.

237

Classi

Nome: House
Attributi: address, color, area…
Metodi: opendoor, closedoor



CLASSE

Caratterstiche

Comportamenti/Azioni

Nome: Car
Attributi: model, color, weight,
Metodi: break, accelerate,…

Nome: Student
Attributi: name, age, matricola,….
Metodi: enroll_to_exam, resign, party,



238

Classi e oggetti

Class: Car
Fields: model, color, weight,…
Methods: break, accelerate,…

Class: House
Fields: address, color, area…
Methods: opendoor, closedoor

Object 1: citroen
model = cv2
color = red
weight …



Object 2: Vw
model = maggiolino
color = red
weight …



Object 3: Renault
model = twingo
color = blue
weight …





Object 1: Scary_House
address = …
color = red
area = small



Object 2: Old_House
address = …
color = green
area = big

239



Attenzione

- Nel contesto del programma, p.e. il videogioco, o il database (banco di dati) le classi possono rappresentare entità come personaggi, veicoli o edifici

MA..

- Le classi sono utilizzate anche per definire concetti più astratti o non materiali, come:
  - stringhe, array
  - suoni e colori
  - emozioni

240



Altri esempi

- Una classe: Lavoratore:
  - Quali sono i suoi possibili attributi?
  - Quali sono delle possibili metodi?



241

Altri esempi

wooclap

- Una classe: Matrice:
  - Quali sono i suoi possibili attributi?
  - Quali sono delle possibili metodi?



242

Altri esempi

- Una classe: Email:
  - Quali sono i suoi possibili attributi?
  - Quali sono delle possibili metodi?



243



Programmazione orientata agli oggetti

(object-oriented programming)

- organizza il codice in entità più ampie e significative che modellano le parti del problema
  - definisce le relazioni tra variabili interconnesse (attributi) e funzioni (metodi)
  - raggruppa dati (attributi/variabili) e il loro comportamento (metodi/funzioni) in un'unica posizione nel codice
- analogia con oggetti della vita reale
- lo stesso codice può essere riutilizzato in molti programmi
- tecnica particolarmente utile se molte persone programmano insieme
- debugging: più facile capire come funziona un programma

244



Definizione di una classe

- La definizione di una classe è fatta di due parti:
  - Dichiarazione:
    descrive gli attributi e i metodi della classe.
    L'insieme degli attributi e dei metodi è anche detto interfaccia della classe.
  - Definizione dei metodi:
    descrive l'implementazione dei metodi della classe.
    Tale implementazione si può trovare:
    - all'interno della dichiarazione della classe,
    - dopo la dichiarazione stessa nello stesso file,
    - in un file separato rispetto a quello che contiene la dichiarazione.

245



Dichiarazione di una classe

- Un elenco dei attributi e delle metodi.
- Sintassi per la dichiarazione di una classe:

```
class class_name {
        tipo1 nome_attributo1;
        …
        tipon nome_attributon;
        tipo1 nome_metodo1(tipo1 id1, …, tipok1 idk1);
        …
        tipom nome_metodom(tipo1 id1, …, tipokm idkm);
};
```

246



Dichiarazione di una classe

- Esempio:

```
class Punto {
      double _x;
      double _y;
      void riscala(double f);
};
```

- Commento 1: in questo caso la definizione del metodo *riscala(double)* è posizionata al di fuori della dichiarazione
- Commento 2: spesso i nomi dei attributi iniziano con _ (convenzione, non è obbligatorio)

247



Definizione di una classe

- Sintassi per la definizione dei metodi (implementazione delle funzioni membro):

```
tipo_i nome_classe::nome_metodo_i(tipo_1 id_1, …, tipo_ki id_ki)
{
        // implementazione della funzione
}
```

- Esempio:

```
void Punto::riscala(double f)
{
      _x *= f;
      _y *= f;
}
```

248



Oggetti

- Una variabile del tipo definito dalla classe è detta oggetto.
- Si dice anche che un oggetto è un'istanza di una classe, ovvero è uno specifico valore nell'insieme di valori che la classe rappresenta (ad es. uno specifico punto).
- Esempio:

```
int main() {
      Punto p;    // p è un oggetto
      ...         // di classe Punto
      return 0;
}
```

249

Accesso ad attributi e metodi

- Dato un oggetto, l'accesso ai suoi membri (attributi e metodi) avviene attraverso l'operatore . (come per le strutture):
  - Accesso gli attributi: *nome_oggetto.nome_attributo;*
  - Chiamata a metodi: *nome_oggetto.nome_metodo (parametri attuali);*

250

Accesso ad attributi e metodi

- Esempio:

```
Punto p;

p._x = 100;
p._y = 200;

p.riscala(0.5);
```

Dichiaro un oggetto *p* di tipo Punto:
- questo oggetto è un'istanza della classe *Punto*,
- rappresenta uno dei vari punti nello spazio 2D.

Attribuisco i valori agli attributi.

Chiamo il metodo *riscala* sull'oggetto p.

251

Intero codice

```
#include <iostream>
using namespace std;

class Punto {
   public:
   double _x;
   double _y;
   void riscala(double f);
};

void Punto::riscala(double f) {
                _x *= f;
                _y *= f;
}

int main() {
    Punto p;
    p._x = 100;
    p._y = 200;
    p.riscala(0.5);
    cout<<p._x;

   return 0;
}
```

E086



- Viene creato un oggetto *p* della classe *Punto*
- Le coordinate del punto vengono inizializzate a *_x = 100* e *_y = 200*
- Il metodo *riscala* viene chiamato con il fattore 0.5, riducendo entrambe le coordinate del punto *p* della metà
- Infine, il valore della coordinata *_x* viene stampato, risultando in 50

252

Esempio 2: Database dei film

```
struct Film {
   char titolo [20];
   int durata;
   int anno;
};

void stampa(film p)
{
   cout << "Titolo: " << p.titolo << endl;
   cout << "Durata: " << p.durata << " minuti" << endl;
   cout<< "Anno: " << p.anno << endl;
}

int main(){
   Film p;
   cout << "Inserisci il titolo (max 20 char): " ;
   cin >>  p.titolo;
   cout << "Inserisci durata: ";
   cin >> p.durata;
   cout << "Inserisci anno: ";
   cin >> p.anno;
   stampa(p);
   return 0;
}
```

E071



Questo è un vecchio esempio:

- un struttura *Film* viene usata per memorizzare i dati di un film

253

Esempio 2: Database dei film

```
class Film {
   public:

   char _titolo [20];
   int _durata;
   int _anno;
   void stampa();
};

void Film::stampa() {
   cout << "Titolo: " << _titolo << endl;
   cout << "Durata: " << _durata << " minuti" << endl;
   cout<< "Anno: " << _anno << endl;
}

int main() {
   Film p;
   cout << "Inserisci il titolo (max 20 char): " ;
   cin >> p._titolo;
   cout << "Inserisci durata: ";
   cin >> p._durata;
   cout << "Inserisci anno: ";
   cin >> p._anno;
   p.stampa();
   return 0;
}
```



E271

- In rosso sono evidenziati i principali cambiamenti fra E071 (versione con il struct) e E271
- Il metodo *stampa* non ha più bisogno degli argomenti

254

Controllo di accesso ai membri

- Per impostazione predefinita gli attributi e i metodi di una classe sono accessibili solo dai metodi della classe stessa!
  Si dice che attributi e metodi sono privati.
- Esempio:

```
int main() {
      Punto c;
      c._x = 100;            // ERRORE!!!
      c._y = 200;            // ERRORE!!!
      c.riscala(2.0);        // ERRORE!!!
      return 0;
}
```

255

Controllo di accesso ai membri

- Per rendere accessibili attributi e metodi all'esterno della classe, ad esempio da parte delle funzione `main`, occorre dichiarare in modo esplicito quali membri si vogliono rendere pubblici.
- Ciò si ottiene facendo precedere i membri pubblici dalla parola chiave `public`.
- Attenzione: è buona pratica rendere pubblico solo quanto strettamente necessario e, in particolare, è preferibile mantenere privati gli attributi della classe.

256

Controllo di accesso ai membri

- Sintassi:

```
class nome_classe {
public:
     // Dichiarazione sezione pubblica
private:
     // Dichiarazione sezione privata
};
```

```
class nome_classe {
     // Dichiarazione sezione privata
public:
     // Dichiarazione sezione pubblica
};
```

257

Controllo di accesso ai membri

- Esempio:

E072

```
class Punto {
      void test(int n);
public:
      double _x;
      double _y;
      void riscala(double f);
};

int main() {
      Punto p;
      p._x = 100;       // OK
      p._y = 200;       // OK
      p.riscala(2.0);   // OK
      p.test(3);        // ERRORE!
      // test è membro privato!
      return 0; }
```



258

Controllo di accesso ai membri

```
class film {

   char _titolo [20]; //ERRORE
   int _durata;
   int _anno;

   void stampa();
};

void film::stampa() {
   cout << "Titolo: " << _titolo << endl;
   cout << "Durata: " << _durata << " minuti" << endl;
   cout<< "Anno: " << _anno << endl;
}

int main(){
   film p;
   cout << "Inserisci il titolo (max 20 char): " ;
   cin >>  p._titolo;
   cout << "Inserisci durata: ";
   cin >> p._durata;
   cout << "Inserisci anno: ";
   cin >> p._anno;
   p.stampa();
   return 0;
}
```

Senza scrivere la parola chiave *public*,

- non si possono accedere i campi *_x* e *_y*,
- né chiamare il metodo *stampa*

259

Quando i metodi e attributi privati sono utili?

- Per guidare una macchina:
  - non ho bisogno di conoscere tutti i dettagli del motore
  - meglio se non inizierei a riparare il motore senza avere la preparazione
- Un programmatore che utilizza una classe non ha bisogno di avere accesso a tutti i dettagli del codice.
- Tutti gli accessi esterni sono "a rischio" di introduzione di errori.





260

private Vs. public

- Come norma generale, si tende a rendere private variabili e funzioni il più possibile.
- Spesso, tutti gli attributi sono privati e i loro valori sono accessibili e modificabili solo attraverso le funzioni (vedremmo più avanti).
- Usare «private»:
  - consente di identificare gli errori nel codice più facilmente
  - impedisce di apportare modifiche dall'esterno (ad esempio, da *main()* oppure da altri oggetti)

261

```
#include <iostream>
using namespace std;

class Punto {
   public:
   double _x;
   double _y;
   void cambiaX (double f);
   double leggiX ();
};

void Punto::cambiaX(double f) {
    _x = f;
}

double Punto::leggiX() {
    return _x;
}

int main() {
    Punto p;
    p._x = 100;
    cout<<p._x;

    return 0;
}
```

Perché non conviene creare attributi "public"?

- questo codice cambia il valore di un attributo pubblico direttamente nella *main*

262

```
#include <iostream>
using namespace std;

class Punto {
   public:
   double _p[2];
   void cambiaX (double f);
   double leggiX ();
};

void Punto::cambiaX(double f) {
    _p[0] = f;
}

double Punto::leggiX() {
   return _p[0];
}

int main() {
    Punto p;
    p._x = 100; //errore
    cout<<p._x; //errore

    return 0;
}
```

Perché non conviene creare attributi "public"?

- se un giorno qualcuno modifica **il codice della classe *Punto*, la *main* non funzionerà più**.
- Se utilizzassi nella *main* i metodi *cambiaX* e *leggiX,* il codice di *main* sarebbe corretto anche dopo la modifica del codice della classe *Punto*.

263

La *main()* corretta:

```
#include <iostream>
using namespace std;

class Punto {
        double _p[2];
    public:
        void cambiaX (double f);
        double leggiX ();
};

void Punto::cambiaX(double f) {
        _p[0] = f;
}

double Punto::leggiX() {
        return _p[0];
}
```

E073

```
int main() {
    Punto p;

    p.cambiaX(100);
    cout<<p.leggiX();

    return 0;
}
```

In questo caso posso modificare la parte privata senza conseguenze

264

Puntatori ad oggetti

- E' possibile creare un puntatore ad oggetto usando la sintassi con *

```
Punto* pp;   // Dichiarazione di un puntatore
pp = NULL;   // Inizializzazione del puntatore a NULL

Punto p;    // Dichiarazione di un oggetto di tipo Punto
p._x = 100; // Assegnamento del valore 100 all'attributo _x
            //  dell'oggetto p
p._y = 200; // Assegnamento del valore 200 all'attributo _y
             //   dell'oggetto p

pp = &p;    // Assegnazione dell'indirizzo di memoria
            // dell'oggetto p al puntatore pp
```

265

Accesso ad attributi e metodi

- Nel caso si abbia un puntatore ad un oggetto, l'accesso ai suoi attributi e metodi avviene attraverso l'operatore ->
  - Accesso gli attributi: *nome_oggetto->nome_attributo;*
  - Chiamata a metodi: *nome_oggetto->nome_metodo (parametri attuali);*

```
Punto* pp;
// inizializzazione del puntatore...
pp->_x = 100;
pp->_y = 200;
pp->riscala(0.5);
```

266

Esempio con uso del puntatori

```
#include <iostream>
using namespace std;

class Punto {
   public:
            double _x;
            double _y;
            void riscala(double f);
};

void Punto::riscala(double f)
{
            _x *= f;
            _y *= f;
}

int main()
{
    Punto p;
    Punto * pp = &p;
    pp->_x = 100;
    pp->_y = 200;
    pp->riscala(0.5);
    cout<<pp->_x;
    return 0;
}
```

E074



l'accesso agli attributi e metodi dell'oggetto p della classe Punto avviene attraverso l'operatore ->

267

Attenzione!

```
#include <iostream>
using namespace std;

class Punto {
   public:
            double _x;
            double _y;
            void riscala(double f);
};

void Punto::riscala(double f)
{
            _x *= f;
            _y *= f;
}

int main()
{
    Punto *pp;  //non è inizializzato
    pp->_x = 100;
    pp->_y = 200;
    pp->riscala(0.5);
    cout<<pp->_x;
    return 0;
}
```

E075

- questo codice non dà un errore di compilazione, ma è incorretto
- la soluzione è di usare l'allocazione dinamica della memoria!
- vedremmo più avanti come fare l'allocazione dinamica



268

Tipologie di metodi

- Costruttori: sono invocati automaticamente per creare e inizializzare un oggetto.
- Distruttori: sono invocati automaticamente per distruggere un oggetto.
- Selettori: accedono agli attributi di un oggetto e ne restituiscono il valore (il loro nome di solito inizia con get).

269



Tipologie di metodi

- Modificatori: accedono agli attributi di un oggetto e ne modificano il valore (il loro nome di solito inizia con set).
- Operatori: 1) elaborano gli attributi di un oggetto oppure 2) implementano per la classe gli operatori standard C++ (p.e., +,-,…)
- Iteratori: elaborano collezioni di oggetti, ad esempio un array di oggetti.

270



Costruttori

- I costruttori sono metodi invocati automaticamente per la creazione e la inizializzazione di un oggetto.
- Hanno lo stesso nome della classe.
- Possono avere zero o più parametri.
- Non restituiscono valori di ritorno (nemmeno void!).

271



Costruttori

- vengono utilizzati:
  - per dare i valori iniziali agli attributi,
  - per allocare dinamicamente la memoria (p.e. array),
  - per fare operazioni I/O come l'accesso ad un file, connessione via rete, creare un log..

272



Costruttori

- Di solito una classe ha tre tipi di costruttori:
  - Costruttore di default: inizializza un oggetto con valori predefiniti (di default) per gli attributi. Ve ne può essere uno soltanto.
  - Costruttori con parametri: inizializzano un oggetto con i valori per gli attributi passati come parametri al costruttore. Una classe può avere più costruttori con parametri.
  - Costruttore di copia: inizializza un oggetto copiando i valori degli attributi da un altro oggetto passato come parametro. Ve ne può essere uno soltanto.

273

Costruttori

- Esempio (dichiarazione dei costruttori):

```
class Punto {
public:
     Punto();
     Punto(double x, double y);
     Punto(const Punto& p);
     void riscala(double f);
private:
     double _x;
     double _y;
};
```

Costruttore di default

Costruttore con parametri

Costruttore di copia

274

Commento su costruttore di copia

- **`Punto(const Punto& p);`**

- Fondamentale è il passaggio **per riferimento (&)**:
  - se l'oggetto fosse passato per valore, il metodo chiamerebbe ricorsivamente all'infinito
  - ogni passaggio per valore crea infatti una copia dell'oggetto!
- Inoltre, l'oggetto trasferito non deve essere modificato all'interno del costruttore.
  - per questo è necessario aggiungere **const**.

275



Costruttori

- Esempio (implementazione dei costruttori):

```
Punto::Punto() {
      _x = 0.0;
      _y = 0.0;
}

Punto::Punto(double x, double y) {
      _x = x;
      _y = y;
}

Punto::Punto(const Punto& p) {
      _x = p._x;
      _y = p._y;
}
```

E076

276



Costruttori

- Esempio (uso dei costruttori):

```
int main() {
   Punto p1;
   // costruttore di default
   //x e y valgono 0

   Punto p2(10, 20);
   // costruttore con parametri
   // x e 10, y e 20

   Punto p3(p2);
   // costruttore di copia
   // x e 10, y e 20

   return 0; }
```

E076

All'interno dello stesso programma, posso creare diversi oggetti utilizzando diversi costruttori

277

Costruttori

- Esempio (uso dei costruttori e puntatori) :

```
int main() {

  // costruttore di default
   Punto* pa = new Punto();

   // costruttore con parametri
   Punto* pb = new Punto(10, 20);

   // costruttore di copia
   Punto* pc = new Punto(*pa);

   …
```

E077

- Alloco dinamicamente la memoria per un oggetto puntato da pa/pb/pc
- Per creare un oggetto dinamicamente, utilizzo un dei costruttori

278

Costruttori

- Esempio (uso dei costruttori e puntatori) :

```
  …
  delete pa;
  delete pb;
  delete pc;
  return 0;
}
```

- Ovviamente, è importante ricordarsi di liberare la memoria quando i tre oggetti non sono più necessari.

279

Puntatore ad un oggetto

Sintassi:

```
Classe* nome_del_puntatore =
                new un_dei_Costrutori(..);
```

280

Soluzione dell'errore di E075



```
#include <iostream>
using namespace std;

class Punto {
   public:
            double _x;
            double _y;
            void riscala(double f);
};

void Punto::riscala(double f)
{
            _x *= f;
            _y *= f;
}

int main()
{
    Punto *pp = new Punto ();
    pp->_x = 100;
    pp->_y = 200;
    pp->riscala(0.5);
    cout<<pp->_x;
    return 0;
}
```

E078

Bisogna allocare la memoria con l'operatore «*new*» per un oggetto di tipo *Punto* prima di assegnare valori ai suoi attributi utilizzando il puntatore *pp*.

281

Puntatore ad un oggetto

Sintassi:

```
Classe* nome_del_puntatore =
                new un_dei_Costrutori(..);
```

- Attenzione: Nel linguaggio C++, il **costruttore di default** viene automaticamente generato dal compilatore **se** la classe non ha alcun costruttore definito esplicitamente.
- Quindi, se la classe non ha un costruttore dichiarato, il compilatore fornirà un costruttore di default senza parametri, che può essere utilizzato con l'operatore *new*

282

In conseguenza..

```
class Punto
{
public:
   double _x;
   double _y;
};
```

questo codice si compila...

```
class Punto
{
public:
   Punto(double x, double y);
   Punto(const Punto& p);
   double _x;
   double _y;
};

Punto::Punto(double x, double y) {
   _x = x;
   _y = y;
}

Punto::Punto(const Punto& p) {
   _x = p._x;
   _y = p._y;
}
```



questo codice da un errore di compilazione..

```
int main(){
   Punto* p = new Punto();

}
```

E282

283

Parametri con valori di default

Un costruttore può avere **parametri con valori di default**, permettendo di usarlo sia come costruttore con parametri sia come costruttore di default.

```
class Rettangolo {
  double _base;
   double _altezza;
public:
   Rettangolo(double base = 1.0, double altezza = 1.0){
      _base = base;
      _altezza=altezza;
};
```

```
Rettangolo r1; // base=1.0, altezza=1.0
Rettangolo r2(5.0); // base=5.0, altezza=1.0
Rettangolo r3(5.0, 3.0); // base=5.0, altezza=3.0
```

Vantaggio: ridurre il numero di costruttori da definire

284

Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
class Punto
{
public:
   Punto();
   Punto(double x, double y);
   Punto(const Punto& p);
   double _x;
   double _y;
};

Punto::Punto()
{
   _x = 1.0;
   _y = 1.0;
}

Punto::Punto(double x, double y) {
   _x = x;
   _y = y;
}

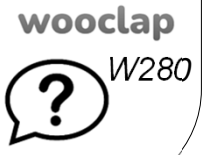
Punto::Punto(const Punto& p) {
   _x = p._x;
   _y = p._y;
}
```

```
int main()
{
   Punto* arr[5];

   for(int i=0; i<5; i++)
      arr[i] = new Punto();

   int sum_X=0;
   for(int i=0; i<5; i++)
      sum_X += arr[i]->_x;

   cout<<sum_X;
}
```



285



Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
class Punto
{
public:
    Punto();
    Punto(double x, double y);
    Punto(const Punto& p);
    double _x;
    double _y;
};

Punto::Punto()
{
    _x = 1.0;
    _y = 1.0;
}

Punto::Punto(double x, double y) {
    _x = x;
    _y = y;
}

Punto::Punto(const Punto& p) {
    _x = p._x;
    _y = p._y;
}
```

```
int main()
{
    Punto* arr[5];

    for(int i=0; i<5; i++)
        arr[i] = new Punto(i,i);

    int sum_X=0;
    for(int i=0; i<5; i++)
        sum_X += arr[i]->_x;

    cout<<sum_X;
}
```

wooclap

W281



286



Perché è utile avere un costruttore di copia?
Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
class Punto
{
    double _x;
    double _y;
    char * c;

public:
    Punto();
    Punto(double x, double y);
    void cambiaNome(char *x);
    void stampaNome ();
};

Punto::Punto() {
    _x = 0.0;
    _y = 0.0;
    c = new char [20];
    strcpy(c,"Uno");
}
void Punto::cambiaNome(char *x){
    delete[] c;
    c = new char[strlen(x) + 1];
    strcpy(c, x);
}
void Punto::stampaNome() {
    cout<<c;
}
```

```
int main() {
    Punto punto1;
    punto1.cambiaNome("Due");
    Punto punto2;
    punto2.cambiaNome("Tre");

    punto2=punto1;
    punto1.cambiaNome("Quattro");
    punto2.stampaNome();
    return 0;
}
```

La risposta è «Quattro».
Perché?





E082

287

Una soluzione migliore è di usare un costruttore che fa la copia dell'oggetto.

In questo caso, il costruttore di copia potrebbe essere:

```
Punto::Punto(const Punto &p)  {
    _x=p._x;
    _y=p._y;
    c = new char[strlen(p.c) + 1];
    strcpy(c, p.c);
}
….
```

```
int main() {
    Punto punto1;
    punto1.cambiaNome("Due");

    Punto punto2 (punto1);

    punto1.cambiaNome("Quattro");

    punto2.stampaNome();
    return 0;
}
```

E083



Questa volta la risposta è «Due»

288

Lista di inizializzazione

- Una parte del costruttore in cui gli attributi della classe vengono inizializzati prima che il corpo del costruttore venga eseguito.

NomeClasse::NomeClasse(parametri) : membro1(valore1), membro2(valore2),…. { // Corpo del costruttore }

- permette di inizializzare direttamente gli attributi della classe, invece di farlo nel corpo del costruttore con l'operatore di assegnazione.
- È necessaria quando si devono inizializzare gli attributi const o riferimenti (&), che devono essere inizializzati al momento della creazione dell'oggetto e non possono essere assegnati successivamente.

289

Esempio: lista di inizializzazione

```
class Punto {
public:
   Punto();
   Punto(double x, double y);
   Punto(const Punto& p);
   double _x;
   double _y;
};

Punto::Punto() : _x(1.0), _y(1.0) {}

Punto::Punto(double x, double y) : _x(x), _y(y) {}

Punto::Punto(const Punto& p) : _x(p._x), _y(p._y) {}

int main(){
   Punto* arr[5];
   for(int i = 0; i < 5; i++)
      arr[i] = new Punto(i, i);

  Punto punto2;
  cout<<punto2._x;

  Punto punto3 (punto2);
  cout<<punto3._x;
}
```



E283

290

Con la **lista di inizializzazione** posso:

- **utilizzare valori diretti**, ad esempio:

  Punto::Punto() : _x(1.0), _y(1.0) {}

- **utilizzare i parametri formali**, ad esempio:

  Punto::Punto(double x, double y) : _x(x), _y(y) {}

- **utilizzare gli attributi di un altro oggetto**, ad esempio:

  Punto::Punto(const Punto& p) : _x(p._x), _y(p._y) {}

Quest'ultimo caso è possibile anche quando gli attributi sono **privati**.

291

Lista di inizial. e attributi *const*

```
class Punto {
public:
    Punto();
    Punto(double x, double y);
    const double _x;            // attributo const
    const double _y;            // attributo const
};

Punto::Punto() : _x(1.0), _y(1.0) {}  // Inizializzazione tramite lista di
inizializzazione

Punto::Punto(double x, double y) : _x(x), _y(y) {}  // Inizializzazione tramite
lista di inizializzazione

int main() {
    Punto punto1;           // Oggetto con il costruttore di default
    Punto punto2(3, 4);     // Oggetto con il costruttore con parametri

    cout << "punto1: (" << punto1._x << ", " << punto1._y << ")\n";
    cout << "punto2: (" << punto2._x << ", " << punto2._y << ")\n";

    return 0;
}
```

quando si utilizzano attributi const all'interno di una classe, è obbligatorio inizializzarli tramite la **lista di inizializzazione** del costruttore

E084



292

Lista di inizial. e riferimenti (raro)

```
class PuntoRef {
public:
    PuntoRef(double& x, double& y);  // costruttore con parametri

    double& _x;  // riferimento
    double& _y;  // riferimento
};
PuntoRef::PuntoRef(double& x, double& y) : _x(x), _y(y) {}

int main() {
    double a = 1.0;
    double b = 2.0;

    PuntoRef punto(a, b);
    cout << "Prima: (" << punto._x << ", " << punto._y << ")\n";

    a = 5.0; // Modifico le variabili originali
    b = 6.0; // Modifico le variabili originali
    cout << "Dopo modifica esterna: (" << punto._x << ", " << punto._y << ")\n";

    punto._x = 10.0; // Modifico tramite l'oggetto
    punto._y = 20.0; // Modifico tramite l'oggetto
    cout << "Dopo modifica tramite oggetto: a=" << a << ", b=" << b << "\n";
```

quando si utilizzano attributi riferimenti all'interno di una classe, è obbligatorio inizializzarli tramite la **lista di inizializzazione** del costruttore

E080



293



Costruttori: classi composte

- Una classe si dice composta se contiene attributi che sono a loro volta istanze (oggetti) di una classe.
- Tali attributi vengono inizializzati utilizzando la lista di inizializzazione.
- Questa si colloca immediatamente dopo la lista dei parametri del costruttore.

294



Costruttori: classi composte

- Esempio (lista di inizializzazione):

```
class TestA {
public:
   TestA();
   TestA(int a);
   TestA(const TestA& a);
private:
   int _a;
};

class TestB {
public:
   TestB();
   TestB(TestA p, int n);
private:
   TestA _t;
   const int _c;
};
```

```
TestB::TestB() : _t(), _c(0)
{
}

TestB::TestB(TestA p, int n)
      : _t(p), _c(n)
{
}
```

```
int main()
{
      TestA ta(25);
      TestB tb1;
      TestB tb2(ta, 8);
      ...
      return 0;
}
```



E081

295



Perché nell'esempio precedente il costruttore di copia viene utilizzato due volte quando viene creato l'oggetto tb2*?*

296



Costruttori: classi composte

- Esempio (lista di inizializzazione):

```
class TestA {
public:
   TestA();
   TestA(int a);
   TestA(const TestA& a);
private:
   int _a;
};

class TestB {
public:
   TestB();
   TestB(TestA &p, int
n);
private:
   TestA _t;
   const int _c;
};
```

```
TestB::TestB() : _t(), _c(0)
{
}

TestB::TestB(TestA &p, int n)
      : _t(p), _c(n)
{
}
```

```
int main()
{
      TestA ta(25);
      TestB tb1;
      TestB tb2(ta, 8);
      ...
      return 0;
}
```



E085

297



Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
class Punto {
public:
    Punto();
    Punto(double x, double y = 0.0);
    void stampa();
private:
    double _x;
    double _y;
};

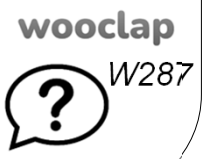
Punto::Punto() : _x(-1), _y(-1) {}

Punto::Punto(double x, double y) : _x(x), _y(y) {}

void Punto::stampa() {
    cout << "Punto: (" << _x << ", " << _y << ")" << endl;
}
int main() {
    Punto p1;
    Punto p3(5.0);

    p1.stampa();
    p3.stampa();

    return 0;
}
```



298



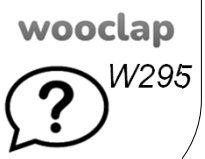
Cosa viene stampato sullo schermo dopo l'esecuzione di questo codice?

```
class Rettangolo {
public:
    Rettangolo(double base = 1.0, double altezza)
        : _base(base), _altezza(altezza) {}

    void stampa() const {
        cout << "Base: " << _base << ", Altezza: " << _altezza << endl;
    }
private:
    double _base;
    double _altezza;
};


int main() {
    Rettangolo r1;
    Rettangolo r2(5.0);

    r1.stampa();
    r2.stampa();
    return 0;
}
```



299



Costruttori

- Note:
  - Se una classe non ha altri costruttori, il compilatore crea automaticamente un costruttore di default. Tale costruttore tuttavia non inizializza gli attributi. Pertanto, è conveniente dichiarare sempre il costruttore di default.
  - Il costruttore di copia viene invocato quando un oggetto è passato per valore ad una funzione. Se la classe non possiede un costruttore di copia viene effettuata la copia bit a bit dell'oggetto. Questo può creare problemi se gli attributi sono, ad esempio, puntatori a strutture dati complesse.

300



Distruttori

- Il distruttore è un metodo speciale di una classe che viene chiamato automaticamente quando si distrugge un oggetto.
- Questo può avvenire, ad esempio, se la variabile corrispondente all'oggetto esce dal proprio campo di visibilità o se l'area di memoria assegnata all'oggetto viene deallocata per mezzo dell'operatore `delete`.
- Il distruttore di solito viene utilizzato per deallocare la memoria eventualmente allocata dinamicamente nel costruttore.

301



Distruttori

- Il distruttore ha lo stesso nome della classe, preceduto dal carattere ~ (tilde).
- In C++, ciascuna classe può avere un solo distruttore. Tale distruttore non ha parametri e non ha né tipo né valore di ritorno.
- Esempio:

```
class Punto {
public:
      ...
      // Distruttore
      ~Punto();
      ...
};
```

302



Distruttori

- Esempio (implementazione e uso):

```
Punto::~Punto() {
      cout << "Questo e' il distruttore" << endl;
}

int main() {
      Punto p1;
      Punto* p2 = new Punto(10, 20);
      ...
      delete p2; // viene chiamato il distruttore
                 // e distrutto p2
      ...
      return 0; // viene chiamato il distruttore
                // e distrutto p1
}
```

303

Distruttori

- Esempio:

Tipicamente
il distruttore si utilizza per deallocare la memoria allocata dinamicamente dal costruttore.

Attenzione:
se dimentichiamo
di deallocare
la memoria nel distruttore, questa resta allocata e non più utilizzabile!

```
class Test {
public:
      Test();
      ~Test();
private:
      int* pa;
};

Test::Test() {
      pa = new int;
      *pa = 25;
}

Test::~Test() {
      if (pa != 0) delete pa;
}
```

304

Esempio di distruttore

```
class Punto {
   double _x;
   double _y;
   char * c;
public:
   Punto();
   Punto(double x, double y);
   ~Punto();
   void cambiaNome(char *x);
   void stampaNome ();
};

Punto::Punto() {
   _x = 0.0;
   _y = 0.0;
   c = new char [20];
   strcpy(c,"Uno");
}

void Punto::cambiaNome(char *x){
   delete[] c;
   c = new char[strlen(x) + 1];
   strcpy(c, x);
}
void Punto::stampaNome() {
   cout<<c;
}

Punto::~Punto() {
   delete[] c;
   cout<<"\n Uso di distruttore"<<endl;
}

int main() {
 Punto punto1;
   punto1.cambiaNome("Due");

   Punto* p2 = new Punto();
   p2->cambiaNome("Tre");

   cout<< " Distruggo l'oggetto: puntato da p2"<<endl;
   delete p2;

    cout<< " Distruggo l'oggetto: punto1"<<endl;
   return 0;
}
```



E302

305



Distruttori: classi composte

- Nel caso di classi composte, l'ordine di chiamata dei distruttori è il reciproco rispetto all'ordine di chiamata dei costruttori.
- In particolare:
  - Vengono chiamati prima i costruttori delle classi degli oggetti assegnati agli attributi e poi il costruttore della classe composta.
  - Viene chiamato prima il distruttore della classe composta e poi i distruttori delle classi degli oggetti assegnati agli attributi.

306



Distruttori: classi composte

Esempio: E288

```
TERMINAL                                                    cppdbg: main.exe
n\WindowsDebugLauncher.exe --stdin=Microsoft-MIEngine-In-dqszwnuo.1yx --stdout=Microsoft-MIEngine-Out-icvdekkj
.lrw --stderr=Microsoft-MIEngine-Error-zpn0jdlb.oqy --pid=Microsoft-MIEngine-Pid-wt1qpmyr.iw4 --dbgExe=C:\msys
64\ucrt64\bin\gdb.exe --interpreter=mi "
TestA
Costruttore con parametro TestA: 25, Nome: OggettoA
TestB2
Costruttore di copia TestA: OggettoA
TestB creato con c = 8, Nome: OggettoB2
Distruttore TestB chiamato: OggettoB2
Distruttore TestA chiamato del: OggettoA
Distruttore TestA chiamato del: OggettoA
```

Perché distruttore TestA viene utilizzato due volte*?*

Ci sono 2 oggetti: ta e *tb2._t*



307



Selettori

- I selettori sono metodi che accedono agli attributi di un oggetto e ne restituiscono il valore. Il loro nome di solito inizia con get.
- Esempio:

```
class Punto {
public:
      ...
      // selettori
      double getX() const;
      double getY() const;

private:
      double _x;
      double _y;
};
```

308



Selettori

- Esempio (implementazione e uso):

```
double Punto::getX() const {
      return _x;
}
double Punto::getY() const {
      return _y;
}
int main() {
      Punto p1;
      Punto p2(25, 30);
      // stampa le coordinate di p1
      cout << p1.getX() << p1.getY() << endl;
      // stampa le coordinate di p2
      cout << p2.getX() << p2.getY() << endl;
      ...
      return 0;}
```

309

Selettori: metodi costanti

- I selettori sono <u>di norma</u> implementati come metodi costanti.
- I metodi costanti non modificano l'oggetto per il quale vengono chiamati, ovvero:
  - non possono modificare il valore degli attributi.
  - non possono chiamare metodi non costanti.
- I metodi costanti sono identificati dalla parola chiave `const` immediatamente dopo la lista dei parametri.
- Esempio:

```
double getX() const;
```

310

Attenzione: un selettore può anche non restituire direttamente il valore di un singolo attributo della classe, ma un'informazione ricavata da esso.

```
class Matrice {
  int _dati[2][3];
public:
 Matrice() {
       for (int i = 0; i < 2; i++) {
          for (int j = 0; j < 3; j++) _dati[i][j] = 0;
         }
    }
 int getElemento(int riga, int colonna) const {
       return _dati[riga][colonna];
    }
};

int main() {
    Matrice m;
    cout << "Elemento (0,0): " << m.getElemento(0, 0) << endl;
    cout << "Elemento (1,2): " << m.getElemento(1, 2) << endl;
}
```



E289

311



Modificatori

- I modificatori sono metodi che accedono agli attributi di un oggetto e ne modificano il valore. Il loro nome di solito inizia con set.
- Esempio:

```
class Punto {
public:
      ...
      // modificatori
      void setX(double x);
      void setY(double y);
      void setXY(double x, double y);
private:
      double _x;
      double _y;
};
```

312



Modificatori

- Esempio (implementazione e uso):

```
void Punto::setX(double x) {
      _x = x;
}
void Punto::setY(double y) {
      _y = y;
}
void Punto::setXY(double x, double y) {
      _x = x; _y = y;
}
int main() {
      Punto p1;
      // modifica le coordinate di p1
      p1.setX(15.0); p1.setY(32.0);
      ...
      return 0;}
```

313

Esempio 1: Punto

```
class Punto {
   double _x;
   double _y;

public:
   Punto();
   Punto(double x, double y);
   ~Punto();

   void setX(double x);
   void setY(double y);

   double getX() const;
   double getY() const;
};

Punto::Punto() {
   _x = -1;
   _y = -1;
 }

Punto::Punto(double x, double y) {
   _x = x;
   _y = y;
}
```

```
void Punto::setX(double x) {
   _x = x;}

void Punto::setY(double y) {
   _y = y;}

double Punto::getX() const {
   return _x;}

double Punto::getY() const{
   return _y;}

Punto::~Punto() {
   cout << "Questo e' il distruttore" << endl;
}

int main()
{
   Punto* p2 = new Punto(10, 20);
   cout << "(" << p2->getX() << ", " << p2->getX() << ")" << endl;

   p2->setX(30);
   cout << " (" << p2->getX() << ", " << p2->getX() << ")" << endl;
   delete p2;
   return 0;
}
```

E089

314

Esempio 2: film a) dichiarazione

```
class Film {
   public:
   char* _titolo;
   int _durata;
   int _anno;

   void stampaFilm();
};

void Film::stampaFilm()
{
   cout << "Titolo: " << _titolo << endl;
   cout << "Durata: " << _durata << endl;
   cout<< "Anno: " << _anno << endl;
}
```

E071

```
class Film {
   char* _titolo;
   int _durata;
   int _anno;

public:
   // Costruttori
   Film(); //default
   Film(char* titolo, int durata, int anno);
   Film(const film& otherFilm); // di copia

   // Distruttore
    ~Film();

   // Selettori e modificatori
   char* getTitolo() const;
   void setTitolo(const char* titolo);
   int getDurata() const;
   void setDurata(int durata);
   int getAnno() const;
   void setAnno(int anno);

   //altri metodi
   void stampaFilm();
};
```

E087

315

b) Implementazione dei metodi

```
film::film() {
    _titolo = new char[20];
    strcpy(_titolo, "N/D");
    _durata = 0;
    _anno = 0;}

film::film(char* titolo, int durata, int anno) {
    _titolo = new char[strlen(titolo) + 1];
    strcpy(_titolo, titolo);
    _durata = durata;
    _anno = anno;}

film::film(const film& otherFilm) {
    _titolo = new char[strlen(otherFilm._titolo) + 1];
    strcpy(_titolo, otherFilm._titolo);
    _durata = otherFilm._durata;
    _anno = otherFilm._anno;}

film::~film() {
    delete[] _titolo;}

char* film::getTitolo() const {
    return _titolo;}

void film::setTitolo(const char* titolo) {
    delete[] _titolo;
    _titolo = new char[strlen(titolo) + 1];
    strcpy(_titolo, titolo);}

int film::getDurata() const {
    return _durata;}

void film::setDurata(int durata) {
    _durata = durata;}

int film::getAnno() const {
    return _anno;}

void film::setAnno(int anno) {
    _anno = anno;}

void film::stampaFilm() {
    cout << "Titolo: " << _titolo << endl;
    cout << "Durata: " << _durata << endl;
    cout << "Anno: " << _anno << endl;
}
```



E087

316

c) ..finalmente il main()

```
int main() {
    int lunghezzaTitolo;
    cout << "Inserisci la lunghezza del titolo del film: ";
    cin >> lunghezzaTitolo;

    // Allocazione dinamica della memoria per il titolo del film
    char* titolo = new char[lunghezzaTitolo + 1];

    cout << "Inserisci il titolo del film (massimo " << lunghezzaTitolo << " caratteri): ";
    cin >> titolo;

    int durata, anno;
    cout << "Inserisci durata: ";
    cin >> durata;
    cout << "Inserisci anno: ";
    cin >> anno;

    film p(titolo, durata, anno);
    p.stampaFilm();

    // Liberazione della memoria allocata dinamicamente
    delete[] titolo;
    return 0;
}
```

Questo programma:

- chiede l'utente di inserire i dati di un film
- crea un oggetto di tipo Film
- stampa attributi di questo oggetto



E087

317

Un oggetto di una classe può:

- essere passato come argomento a una funzione
- può essere restituito da una funzione

Esempio: una funzione che, dato un oggetto di tipo film, crea il suo sequel:

- Il titolo del nuovo film corrisponde a quello del vecchio, ma viene aggiunto il numero '2'.
- Inoltre, il nuovo film ha un anno di produzione diverso.

```
film creaSequel(const film& f, int anno) {
    char* nuovoTitolo = new char[strlen(f.getTitolo()) + 3];
    strcpy(nuovoTitolo, f.getTitolo());
    strcat(nuovoTitolo, " 2");
    film nuovoFilm(nuovoTitolo, f.getDurata(), anno);
    delete[] nuovoTitolo;

    return nuovoFilm;
}
```

E088



318

Utilizzo di questa funzione nel main()

```
int main() {

    …

    film p(titolo, durata, anno);
    p.stampaFilm();

    // Chiamata alla funzione per creare un nuovo film
    film nuovo = creaSequel(p, p.getAnno()+3);

  cout << "Nuovo film creato: ";
    nuovo.stampaFilm();

    // Liberazione della memoria allocata
    delete[] titolo;

    return 0;
}
```



E088

Questo codice crea un nuovo oggetto di tipo film.

Il titolo del nuovo film è diverso rispetto a quello del film originale, e l'anno di produzione è più recente di tre anni.

319

Puntatori ad oggetti e funzioni

Un puntatore ad oggetto può:

- essere passato come argomento a una funzione
- può essere restituito da una funzione

```
film* creaSequel(const film* f, int anno) {
    char* nuovoTitolo = new char[strlen(f->getTitolo()) + 3];
    strcpy(nuovoTitolo, f->getTitolo());
    strcat(nuovoTitolo, " 2");

    film *nuovo = new film(nuovoTitolo, f->getDurata(), anno);
    delete[] nuovoTitolo;
    return nuovo;
}
```

```
int main(){
…
 film p(titoloInput, durataInput, annoInput);
    p.stampaFilm();

    film *nuovo = creaSequel(&p, p.getAnno()+3);
    cout << «Un nuovo film è stato creato: " <<endl;
    nuovo->stampaFilm();
…
}
```



E090

320

Supponendo di avere una classe Punto come nell'esercizio E089, quali coordinate vengono stampate sulla console quando viene eseguita la riga evidenziata in blue?

```
void sposta(Punto *p, double radius){
        double angle = 0;
        double newX = p->getX() + radius * cos(angle);
        double newY = p->getY() + radius * sin(angle);
        p->setX(newX);
        p->setY(newY);

}

int main() {
    Punto* p2 = new Punto(0, 0);

    sposta(p2,100);
    p2->stampaPunto();

    delete p2;
    return 0;
}
```

wooclap



W092

321

Supponendo di avere una classe Punto come nell'esercizio E089, quali coordinate vengono stampate sulla console quando viene eseguita la riga evidenziata in blue?

```
void sposta(Punto p, double radius){
      double angle = 0;
      double newX = p.getX() + radius * cos(angle);
      double newY = p.getY() + radius * sin(angle);
      p.setX(newX);
      p.setY(newY);

}

int main() {
   Punto* p2 = new Punto(0, 0);

   sposta(*p2,100);
   p2->stampaPunto();

   delete p2;
   return 0;
}
```



W091

322

Spiegazione di *W091*

- Quando viene chiamato il metodo *sposta*, viene fatta una copia di Punto (passaggio per valore) senza un costruttore di copia esplicito.
- Osservate cosa succede se aggiungiamo un costruttore di copia, come questo:

```
Punto::Punto(const Punto &p)
{
   _x=0;
   _y=-1;
}
```



E290

```
int main()
{
   Punto* p2 = new Punto(0, 0);

   sposta(*p2,100);
   p2->stampaPunto();

   delete p2;
   return 0;
}
```

323

Supponendo di avere una classe Punto come nell'esercizio E089 ed un nuovo metodo *sposta* definita sotto, quali coordinate vengono stampate sulla console quando viene eseguita la riga evidenziata in blue?

```
void Punto::sposta(double radius){
        double angle = M_PI;
        _x = _x + radius * cos(angle);
        _y = _y + radius * sin(angle);
    }

int main() {
    Punto* p2 = new Punto(0, 0);

    p2->sposta(100);
    p2->stampaPunto();
    delete p2;
    return 0;
}
```

wooclap



W093

Array di oggetti

- Una volta che abbiamo definito la classe Punto possiamo creare un array di oggetti.
- La sintassi è la stessa di quella per tipi semplici come int e double
- In questo esempio creiamo un array di 10 punti con coordinate da (1, 1) a (10, 10).

```
int main() {
    // Creazione dell'array di 10 punti
    Punto punti[10];

for (int i = 0; i < 10; ++i) {
        punti[i].setX(i + 1);
        punti[i].setY(i + 1);
    }

    // Stampa dei punti
    for (int i = 0; i < 10; ++i) {
        punti[i].stampaPunto();
    }

    return 0;
}
```



E094

Array di oggetti

- L'array degli oggetti può essere un parametro di una funzione.
- Questa funzione calcola la media dei punti nell'array.

```
void calcolaMediaCoordinate(Punto punti[], int size)
{
   double sommaX = 0, sommaY = 0;
   for (int i = 0; i < size; ++i)
   {
      sommaX += punti[i].getX();
      sommaY += punti[i].getY();
   }
   double mediaX = sommaX / size;
   double mediaY = sommaY / size;
   cout << "La media delle coordinate dei punti è: (" <<
mediaX << ", " << mediaY << ")" << endl;
}
```



E095

326

Array di oggetti

- L'array degli oggetti può essere un parametro di una funzione.
- In questo caso, la funzione calcola la media dei punti nell'array e restituisce il nuovo punto.

```
Punto calcolaMediaPunti(Punto punti[], int size) {
   double sommaX = 0, sommaY = 0;

   for (int i = 0; i < size; ++i) {
      sommaX += punti[i].getX();
      sommaY += punti[i].getY();
   }

   double mediaX = sommaX / size;
   double mediaY = sommaY / size;

   return Punto(mediaX, mediaY);
}
```



E095

327

Supponendo di avere una classe Punto come nell'esercizio E089, quali coordinate vengono stampate sulla console quando viene eseguita la riga evidenziata in blue?

```
int main() {
    Punto punti[10];
    double sommaX =0;
    double sommaY =0;

    for (int i = 0; i < 10; ++i) {
        sommaX += punti[i].getX();
        sommaY += punti[i].getY();
    }
    double mediaX = sommaX / 10;
    double mediaY = sommaY / 10;

    cout<<mediaX<<"; "<<mediaY;
    return 0;
}
```

wooclap



W097

328

Array dei puntatori ai oggetti

- È possibile creare un array di puntatori a oggetti
- In questo caso creiamo un array di 10 puntatori ad oggetti di tipo Punto
- liberare la memoria

```
Punto calcolaMediaPunti(Punto* punti[], int size) {
    double sommaX = 0, sommaY = 0;
    for (int i = 0; i < size; ++i) {
        sommaX += punti[i]->getX();
        sommaY += punti[i]->getY();
    }
    double mediaX = sommaX / size;
    double mediaY = sommaY / size;
    return Punto(mediaX, mediaY);
}

int main() {
    Punto* punti[10];
    for (int i = 0; i < 10; ++i)  punti[i] = new Punto(i + 1, i + 1);

    Punto media = calcolaMediaPunti(punti, 10);
    cout << «Il punto medio è: (" << media.getX() << ", " << media.getY
    for (int i = 0; i < 10; ++i)  delete punti[i];
    return 0; }
```



E096

329

Array dei puntatori ai oggetti

- Esempio 2: un array di tre puntatori ai oggetti di tipo film
- liberare la memoria



E098

```
int main() {
    const int size=3;
    film* listaFilm[size];

    for (int i = 0; i < size; ++i) {
        int lunghezzaTitolo;
        cout << "Inserisci la lunghezza del titolo del " << i + 1 << " film: ";
        cin >> lunghezzaTitolo;
        char* titolo = new char[lunghezzaTitolo + 1];
        cout << "Inserisci il titolo del " << i + 1 << " film:  (massimo " << lu
        cin >> titolo;

        int durata, anno;
        cout << "Inserisci la durata in minuti del " << i + 1 << " film: ";
        cin >> durata;
        cout << "Inserisci l'anno di produzione del " << i + 1 << " film: ";
        cin >> anno;
        listaFilm[i] = new film(titolo, durata, anno);
    }

for (int i = 0; i < size; ++i)  {
        cout << "Film " << i + 1 << ":" << endl; listaFilm[i]->stampaFilm(); }
for (int i = 0; i < size; ++i) delete listaFilm[i];
 return 0;}
```

330

Array dinamica di oggetti tipo Film

- Creiamo un archivio di film senza limitazioni sul numero di oggetti inseriti, usando un array dinamica dei puntatori ai oggetti tipo Film
- L'utente decide quanti film inserire nell'archivio (variabile *numFilm*)
- L'operatore *new* restituisce un puntatore a un film, che può essere inserito direttamente nell'array.

E099

```
int main() {
  int numFilm;
  cout << "Quanti film vuoi inserire? ";
  cin >> numFilm;

  film** listaFilm = new film*[numFilm];
  for (int i = 0; i < numFilm; ++i) {
     int lunghezzaTitolo;
     cout << "Inserisci la lunghezza del titolo del " << i + 1 << " film: ";
     cin >> lunghezzaTitolo;

     char* titolo = new char[lunghezzaTitolo + 1];
     cout << "Inserisci il titolo del " << i + 1 << " film: (massimo " << lungh
     cin >> titolo;
     int durata, anno;
     cout << "Inserisci la durata in minuti del " << i + 1 << " film: ";
     cin >> durata;
     cout << "Inserisci l'anno di produzione del " << i + 1 << " film: ";
     cin >> anno;

     listaFilm[i] = new film(titolo, durata, anno);
     delete[] titolo; // Liberiamo la memoria allocata per il titolo
  }
```

331

Array dinamica di oggetti tipo Film

```
...
for (int i = 0; i < numFilm; ++i) {
      cout << "Film " << i + 1 << ":" << endl;
      listaFilm[i]->stampaFilm();
      cout << endl;
   }

   // Deallocazione della memoria
   for (int i = 0; i < numFilm; ++i) {
      delete listaFilm[i];
   }
   delete[] listaFilm;

   return 0;
}
```



E099

- L'operatore «freccia» (->) viene utilizzato per invocare i metodi di un oggetto puntato da un puntatore
- Il modo corretto per liberare la memoria

332

Un archivio di film

Dopo aver creato il nostro archivio, possiamo eseguire alcune operazioni su di esso, ad esempio:

- funzione per trovare il film più vecchio

Per evitare la creazione di *shallow copy*, consigliabile usare il riferimento

```
Film& trovaFilmPiuVecchio( Film* listaFilm[], int size) {
    Film filmPiuVecchio = *listaFilm[0];

    for (int i = 1; i < size; i++) {
        if (listaFilm[i]->getAnno() < filmPiuVecchio.getAnno()) {
            filmPiuVecchio = *listaFilm[i];
        }
    }

    return filmPiuVecchio;
}
```

333



Un archivio di film

Dopo aver creato il nostro archivio, possiamo eseguire alcune operazioni su di esso, ad esempio:

- funzione per trovare il film più vecchio

Soluzione alternativa

```
Film trovaFilmPiuVecchio(Film* listaFilm[], int size) {
int indicePiuVecchio = 0;

for (int i = 1; i < size; i++) {
        if (listaFilm[i]->getAnno() < listaFilm[indicePiuVecchio]->getAnno()) {
        indicePiuVecchio = i;
        }
    }
    return *listaFilm[indicePiuVecchio];
}
```

334



Un archivio di film

Dopo aver creato il nostro archivio, possiamo eseguire alcune operazioni su di esso, ad esempio:

- funzione per trovare il film più lungo che restituisce il puntatore a quel film

```
Film* trovaFilmPiuLungo(Film* listaFilm[], int size) {
    Film* filmPiuLungo = listaFilm[0];

    for (int i = 1; i < size; i++) {
        if (listaFilm[i]->getDurata() > filmPiuLungo->getDurata()) {
            filmPiuLungo = listaFilm[i];
        }
    }

    return filmPiuLungo;
}
```

335

Un archivio di film

Il main() per trovare un film più vecchio e più lungo potrebbe essere così:

```
int main (){
   …
   // Trova e stampa il film più vecchio
   film filmPiuVecchio = trovaFilmPiuVecchio(listaFilm, numFilm);
   cout << "Il film piu vecchio e: " << endl;
   filmPiuVecchio.stampaFilm();

   // Trova e stampa il film più lungo
   film* filmPiuLungo = trovaFilmPiuLungo(listaFilm, numFilm);
   cout << "Il film piu lungo e: " << endl;
   filmPiuLungo->stampaFilm();
   cout << endl;
  …
}
```



E100

336

Un esempio: Funzione restituisce un array di oggetti (nuovi)

Avendo una classe Punto (come nell'esempio E089), creiamo una funzione che genera 10 nuovi punti distanti 100 dal punto specificato nei parametri di ingresso.

I nuovi punti vengono inseriti in un nuovo array che viene poi restituito a main().

```
Punto* creaPuntiDistanti(Punto* punto) {
   Punto* punti = new Punto[10];
   double angleIncrement = 2 * M_PI / 10;
   double radius = 100;

   for (int i = 0; i < 10; ++i) {
      double angle = i * angleIncrement;
      double newX = punto->getX() + radius * cos(angle);
      double newY = punto->getY() + radius * sin(angle);
      punti[i].setX(newX);
      punti[i].setY(newY);
   }

   return punti;
}
```

E104



337

Costruttore/Distruttore e file

I costruttori e i distruttori possono essere utili per gestire l'accesso ai file.

In questo esempio, abbiamo un file di testo con i dati di diversi studenti.

Il costruttore legge i dati dal file, mentre il distruttore chiude il file.

studente.txt
Mario
Rossi
28

```
Studente::Studente(ifstream& file) : file(&file) {
   if (file.is_open()) {
      // Leggiamo i dati dal file e li assegniamo agli attributi della classe
      getline(file, nome);
      getline(file, cognome);
      file >> voto;
      file.ignore(); // Ignoriamo il carattere di nuova linea residuo

      // Stampa messaggio di errore se la lettura del voto fallisce
      if (file.fail())  cerr << "Errore nella lettura del voto." << endl;
   } else cerr << "Impossibile aprire il file." << endl;
}

.....

Studente::~Studente() {
   // Chiudiamo l'accesso al file nel distruttore
   file->close();
   cout << "Distruttore del " << nome << " " << cognome << endl;
}
```



E103

338

Operatori

- Gli operatori sono metodi che elaborano gli attributi di un oggetto. Essi possono essere:
  - Metodi che applicano specifici algoritmi per l'elaborazione degli attributi (ad esempio, per la classe Punto, il metodo per il calcolo della distanza con un altro punto)
  - Metodi che implementano per la classe gli operatori standard del linguaggio C++ (ad esempio gli operatori +, -, =, ++, --, ecc.). In questo caso si parla di sovraccaricamento (overloading) degli operatori standard.

339



Operatori

- Esempio (calcolo della distanza):

```
class Punto {
      ...
      // operatori
      void riscala(double f);
      double distanza(Punto p) const;
private:
      double _x;
      double _y;
};

double Punto::distanza(Punto p) const {
      double dx = (_x - p._x);
      double dy = (_y - p._y);
      return sqrt(dx * dx + dy * dy);
}
```

340



Operatori

- Esempio (calcolo della distanza):

```
int main {
      Punto p;   // p ha coordinate (0,0)
      Punto q(3.0, 4.0);

      // stampa la distanza tra p e q
      cout << p.distanza(q) << endl;

      // modifica le coordinate di p
      p.setXY(2.0, 3.0);

      // stampa la nuova distanza tra p e q
      cout << p.distanza(q) << endl;

      return 0;
}
```

E291



341



Operatori

- A volte, per usare gli oggetti secondo un approccio procedurale, si definiscono delle cosiddette funzioni wrapper. Esempio:

```
double distanza(Punto p, Punto q) {
      return p.distanza(q);
}

int main {
      Punto p(1.0, 2.0);
      Punto q(4.0, 6.0);
      // stampa la distanza tra p e q
      cout << distanza(p, q) << endl;
      return 0;
}
```

342



Puntatore `this`

- Tramite il puntatore `this`, ciascun oggetto ha accesso al proprio indirizzo di memoria.
- Esempio: stampa delle le coordinate di un punto

```
class Punto {
      ...
      // operatori
      void stampa() const;
      ...
};
```

```
void Punto::stampa() const {
   // queste istruzioni sono equivalenti:
   cout << _x << ", " << _y << endl;
   cout << this->_x << ", " << this->_y << endl;
   cout << (*this)._x << ", " << (*this)._y <<
endl;
}
```

343

Puntatore `this`

- Un uso importante del puntatore `this` consiste nel consentire la chiamata a cascata di funzioni membro.
- Esempio: modificatori chiamati in cascata

```
class Punto {
  ...
  // modificatori
  Punto& setX(double x);
  Punto& setY(double y);
  ...
};
```

```
Punto& Punto::setX(double x){
      _x = x;
      return *this;
}
Punto& Punto::setY(double y){
      _y = y;
      return *this;
}
```



E292

```
int main(){
  Punto p;
  p.setX(4.0).setY(3.5);
  return 0; }
```

344

Overloading degli operatori

- Spesso è utile dotare una classe degli operatori predefiniti del linguaggio C++.
- Ciò permette ad esempio di scrivere codice come il seguente:



E293



```
int main {
      Punto a(2.0, 3.0);
      Punto b(8.0, 5.0);
      Punto c = a + b;
      ...
      return 0; }
```

- Il meccanismo che si utilizza è quello del sovraccaricamento (overloading) degli operatori standard.

345



Overloading degli operatori

- Operatori che si possono sovraccaricare:

```
+    -    *    /    %    ^    &    |    ~
!    =    <    >    +=   -=   *=   /=   %=
^=   &=   |=   <<   >>   >>=  <<=  ==   !=
<=   >=   &&   ||   ++   --   ,    ->*  ->
()   []   new  delete  new[]  delete[]
```

- Operatori che NON si possono sovraccaricare:
  . .* :: ?: (i.e.operatore ternario if-else)



Overloading degli operatori

- Sintassi:

  tipo_ritorno `operator` *id (lista parametri);*

- Note:
  - La parola chiave `operator` indica che si sta effettuando il sovraccaricamento di un operatore.
  - Non si possono cambiare precedenza, raggruppamento e numero degli operandi.
  - Le funzioni operatore sono di solito metodi di una classe. Possono anche essere funzioni non membro purché il tipo di almeno un parametro sia una classe o un riferimento a una classe.

Overloading di operatori unari

- Sintassi (solo operatori unari!!) :

  tipo `operator` *op_id ();*

- Esempio:

```
Punto operator-();
```

- Le funzioni operatore, implementate come funzioni membro, che sovraccaricano operatori unari non hanno parametri.
- L'operatore infatti si applica sull'oggetto stesso per il quale viene invocato.

348

Esempio

- Senza sovraccarico il seguente codice produce l'errore di compilazione..

```
int main() {
   Punto p1  (10, 10);
   p1.stampaPunto();

   Punto p2=-p1; //ERRORE
   p2.stampaPunto();

   return 0;
}
```



E105

error: no match for 'operator-' (operand type is 'Punto')

Cosa vuol dire: un punto opposto ad un altro punto?

E' difficile dare una risposta generica che vale per qualsiasi classe

349

Overloading di operatori unari

- Esempio (operatore - per calcolare l'opposto di un punto bidimensionale):

```
class Punto {
      ...
      // operatori
      Punto operator-();
private:
      double _x;
      double _y;
};

Punto Punto::operator-() {
      return Punto(-_x, -_y);
}
```

```
int main() {
  Punto p(3.0, 2.0);
  Punto q = -p;
  // q ha coordinate
  // (-3, -2)

  ...

  return 0;
}
```

E106



350

Overloading di operatori unari

- Esempio 2 (operatore - per calcolare l'inverso delle coordinate):

```
class Punto {
      ...
      // operatori
      Punto operator-();
private:
      double _x;
      double _y;
};

Punto Punto::operator-() {
      return Punto(1/_x, 1/_y);
}
```

```
int main() {
  Punto p(3.0, 2.0);
  Punto q = -p;
  // q ha coordinate
  // (-3, -2)

  ...

  return 0;
}
```

E106



351

Overloading di operatori unari

- Esempio 3 operatore – applicato ad un film? p.e. inverte il titolo

```
Film Film::operator-()  const {
    char* invertedTitolo = new char[strlen(_titolo) + 1];
    int length = strlen(_titolo);
    for (int i = 0; i < length; ++i) {
        invertedTitolo[i] = _titolo[length - 1 - i];
    }
    invertedTitolo[length] = '\0';

    Film invertedFilm(invertedTitolo, _durata, _anno);
    delete[] invertedTitolo;

    return invertedFilm;
}
```

```
int main() {
    film mioFilm("Inception", 148, 2010);
    mioFilm.stampaFilm();

    Film nuvovofilm = -mioFilm;

    cout << «Nuovo titolo: " << nuvovofilm.getTitolo();
    return 0;
}
```

352

Overloading di operatori unari

- Esempio 3 forse meglio utilizzare operatore ! per invertire il titolo

```
Film Film::operator!() const {
    char* invertedTitolo = new char[strlen(_titolo) + 1];
    int length = strlen(_titolo);
    for (int i = 0; i < length; ++i) {
        invertedTitolo[i] = _titolo[length - 1 - i];
    }
    invertedTitolo[length] = '\0';

    Film invertedFilm(invertedTitolo, _durata, _anno);
    delete[] invertedTitolo;

    return invertedFilm;
}
```

```
int main() {
    film mioFilm("Inception", 148, 2010);
    mioFilm.stampaFilm();

    Film nuvovofilm = !mioFilm;
    cout << «Nuovo titolo: " << nuvovofilm.getTitolo();
    return 0;
}
```

E107



353



Overloading di ++ e di --

- Gli operatori unari ++ e -- hanno due forme: la forma prefissa e quella postfissa. Entrambe possono essere sovraccaricate.
- Forma prefissa:

  tipo& `operator` ++();

  tipo& `operator` --();
- Forma postfissa:

  tipo `operator` ++(int);

  tipo `operator` --(int);

354



Overloading di ++ e --

- Esempio (operatore di incremento e di decremento unitario prefisso e postfisso):

```
class Punto {
      ...
      // operatori
      Punto& operator++();
      Punto operator++(int);
      Punto& operator--();
      Punto operator--(int);
private:
      double _x;
      double _y;
};
```

355

Overloading di ++ e --

- Esempio (operatore di incremento unitario prefisso e postfisso):

```
// incremento prefisso
Punto& Punto::operator++() {
      _x += 1.0; _y += 1.0;
      return *this;
}

// incremento postfisso
Punto Punto::operator++(int) {
      Punto temp(*this);
      _x += 1.0; _y += 1.0;
      return temp;
}
```

```
int main() {
  Punto p(3.0, 2.0);
  Punto q = p++;
  // q ha coordinate
  // (3, 2)
  Punto r = ++p;
  // r ha coordinate
  // (5, 4)
  …
  return 0;
}
```

E108



356

Overloading di ++ e – per film

- Questi operatori non hanno interpretazione intuitiva per la classe *film*. In questo esempio:
  - Il metodo *operator++()* è definita per incrementare l'anno del film. Restituisce un riferimento al film stesso dopo l'incremento.
  - Il metodo *operator++(int)* è definita per incrementare la durata del film. Restituisce una copia del film **prima** dell'incremento.

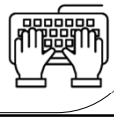
```
// Operatore di incremento prefix ++
Film& Film::operator++() {
   ++_anno;
   return *this;
}

// Operatore di incremento postfix ++
Film Film::operator++(int) {
   film temp(*this);
   ++_durata;
   return temp;
}

int main()
{
   film mioFilm("Matrix", 120, 2020);
   mioFilm.stampaFilm();
   mioFilm++;  // Utilizzo dell'operatore postfix ++
   mioFilm.stampaFilm();
   ++mioFilm;  // Utilizzo dell'operatore prefix ++
   mioFilm.stampaFilm();

   return 0;
}
```

E109



357

Cosa viene stampato dopo esecuzione dell'ultima riga?

```
Punto& Punto::operator++()
{
   _x += 1.0;
   _y += 1.0;
   return *this;
}

Punto Punto::operator++(int)
{
   Punto temp(*this);
   _x += 1.0;
   _y += 1.0;
   return temp;
}

Punto Punto::operator-()
{
   return Punto(-_x, -_y);
}
```

```
int main() {

   Punto p(3.0, 2.0);
   cout <<" p = ";
   p.stampaPunto();

   Punto q =-p++;
   cout <<" q = ";
   q.stampaPunto();

   cout <<" p = ";
   p.stampaPunto();

   Punto z =-++p;
   cout <<" z = ";
   z.stampaPunto();

}
```



wooclap

W294



358

Overloading di operatori binari

- Sintassi:

  tipo `operator` *op_id (tipo id);*

- Esempio:

  ```
  Punto operator+(Punto p);
  ```

- Le funzioni operatore, implementate come funzioni membro, che sovraccaricano operatori binari hanno un parametro.
- L'operatore infatti si applica sull'oggetto stesso per il quale viene invocato e sull'oggetto passato come parametro.

359

Overloading di operatori binari

- Esempio (operatore + per calcolare la somma di due punti):

E110

```
class Punto {
   ...
   // operatori
   Punto operator+(Punto p);
private:
   double _x;
   double _y;
};

Punto Punto::operator+(Punto p)
  Punto q(_x + p._x, _y + p._y);
  return q;
}
```

```
int main() {
  Punto p(5.0, 2.0);
  Punto q(4.0, 1.0);
  Punto r = p + q;
  // r ha coordinate
  // (9, 3)
  ...
  return 0;
}
```

Attenzione! Operatore = non è (ancora) sovraccaricato

360

Overloading di operatori binari

Esempio2 (operatore + per calcolare la somma di due film):

- concateniamo i titoli
- sommiamo gli anni e le durate

Attenzione: il codice NON funziona con l'operatore +=, che è diverso da quello considerato

```
Film film::operator+(const Film& other) {
   char* nuovo = new char[strlen(_titolo) + strlen(other._titolo) + 2];
   strcpy(nuovo, _titolo);
   strcat(nuovo, "&");
   strcat(nuovo, other._titolo);

   int nuovaDurata = _durata + other._durata;
   int nuovoAnno = _anno + other._anno;

   film nuovoFilm(nuovo, nuovaDurata, nuovoAnno);

   delete[] nuovo;
   return nuovoFilm;
}
int main() {
   film film1("Matrix", 120, 2000);
   film film2("Solaris", 90, 1995);
   film filmSomma = film1 + film2;
   filmSomma.stampaFilm();
   return 0;}
```

Attenzione! Operatore = nc è (ancora) sovraccarica

E111

361

Overloading di operatori binari

Esempio3 (operatore += per calcolare la somma di due film):

- concateniamo i titoli
- sommiamo gli anni e le durate

```
Film film::operator+=(const Film& other) {
    // Concatena i titoli dei film
    char* nuovo = new char[strlen(_titolo) + strlen(other._titolo) + 2];
    strcpy(nuovo, _titolo);
    strcat(nuovo, "&");
    strcat(nuovo, other._titolo);

    _durata += other._durata;
    _anno += other._anno;
    // Aggiorna il titolo del film corrente con il titolo concatenato
    delete[] _titolo;
    _titolo = nuovo;
    return *this;
}

int main() {
        film film3("Inception", 148, 2010);
        film film4("Interstellar", 169, 2014);
        film4 += film3;
        film4.stampaFilm();
```

E111



362

Overloading dell'operatore di assegnamento

- Sintassi:
  tipo& `operator=` *(const tipo& id);*
- Esempio:
  `Punto& operator=(const Punto& p);`
- Se l'operatore di assegnamento non è sovraccaricato, viene eseguito l'assegnamento membro a membro.
- Occorre che l'operatore di assegnamento sovraccaricato verifichi che un oggetto non venga assegnato a se stesso.

363



Overloading dell'operatore di assegnamento

- Esempio:

```
class Punto {
  ...
  // operatori
  Punto& operator=(const Punto& p);
  ...
};
  Punto& Punto::operator=(const Punto& p){
      if (this != &p) { // evita l'autoassegnamento
          _x = p._x; _y = p._y;
      }
      return *this;
}
```

364



Overloading dell'operatore di assegnamento

- In questo caso, anche senza sovraccaricare l'operatore di assegnamento (=), il risultato sarebbe lo stesso (vedi E110)

```
int main() {

    Punto punto1(3.5, 4.2);
    Punto punto2(1.8, 2.5);

    cout << "Coordinate del primo punto: ";
    punto1.stampaPunto();
    cout << "Coordinate del secondo punto: ";
    punto2.stampaPunto();

    punto1 = punto2;
    cout << "Dopo l'assegnamento, coord. del 1 punto: ";
    punto1.stampaPunto();

    return 0;
}
```

E112



365

Supponendo di avere una classe **Film** definita come in E87, cosa viene stampato quando viene eseguita la riga verde?

Assegnamento senza overloading

- Supponendo di avere una classe *Film* definita come in E087, cosa viene stampato quando viene eseguita la riga blue?




```
int main()
{
    // Creazione di due oggetti Punto
    film film1("Matrix", 120, 2000);
    film film2("Solaris", 90, 1995);

    film1 = film2;
    film1.stampaFilm();
    film2.stampaFilm();

    film2.setTitolo("Matrice");
    film1.stampaFilm();

    return 0;
}
```

W114



366

Overloading dell'operatore di assegnamento

- Sovraccaricando l'operatore =, possiamo garantire la corretta copia dei dati, inclusa **una copia profonda** del titolo del film
- In questo caso, ci sono due spazi nella memoria che contengono la stessa sequenza di simboli (titolo di film)

```
Film& Film::operator=(const Film& otherFilm) {
    if (this != &otherFilm) {
        delete[] _titolo;
        _titolo = new char[strlen(otherFilm._titolo) + 1];
        strcpy(_titolo, otherFilm._titolo);
        _durata = otherFilm._durata;
        _anno = otherFilm._anno;
    }
    return *this;
}

int main() {
    Film film1("Matrix", 120, 2000);
    Film film2("Solaris", 90, 1995);

    film1 = film2;
    film1.stampaFilm();
    film2.stampaFilm();

    film2.setTitolo("Matrice");
    film1.stampaFilm();
    film2.stampaFilm();

    return 0;
}
```

E115



367

Operatore di confronto o uguaglianza (==)

- Sintassi:

  bool operator==(const NomeClasse& other) const;

- E' accetabile usare *int* come tipo di ritorno:

  int operator==(const NomeClasse& other) const;

369

Esempio

E126



```
class Punto {
    double _x;
    double _y;
public:
    Punto();
    Punto(double x, double y);
    ~Punto();
    Punto operator+(const Punto& p) const;
    bool operator==(const Punto& p) const;

    void setX(double x);
    void setY(double y);

    double getX() const;
    double getY() const;
    void stampaPunto() const;
};
```

```
..
Punto Punto::operator+(const Punto& p) const {
    // crea un nuovo oggetto risultato
    Punto q(_x + p._x, _y + p._y);
    return q; // ritorno per valore
}

bool Punto::operator==(const Punto& p) const {
    return (_x == p._x) && (_y == p._y);
}

int main() {
    Punto p(5.0, 2.0);
    Punto q(4.0, 1.0);

    if(p+q==q+p) cout <<"Si"; else cout<<"No";

    return 0;
}
```

370



Overloading degli operatori di inserimento ed estrazione

- Sintassi:
  ostream& `operator`*<< (ostream& os, const tipo& id);*
  istream& `operator`*>> (istream& is, tipo& id);*
- Sovraccaricare gli operatori di inserimento e di estrazione consente di poter leggere da tastiera e stampare a video gli attributi di un oggetto.
- Solitamente gli operatori di inserimento e di estrazione non sono implementati come metodi della classe, ma come funzioni amiche (friend functions) della classe.

371



Funzioni amiche

- Le funzioni esterne ad una classe (ad esempio la funzione main), di norma non possono accedere ai membri privati della classe.
- Una funzione amica (friend function) è una funzione esterna alla quale viene concesso di accedere ai membri privati di una classe.
- Le funzioni amiche di una classe sono dichiarate con la parola chiave `friend` all'interno della classe di cui sono amiche.

372



Lezione 8

373



Funzioni amiche

- Esempio:

```
class TestClass {
  ...
  friend int TestF(int x);
  private:
     int _z;
};
int TestF(int x) {
  TestClass tc;
  tc._z = x;
  // può accedere a _z
  ...
  return tc._z;
}
```

- Nel caso degli operatori di inserimento ed estrazione, questi sono implementati come funzioni esterne, ma occorre che possano accedere ai membri privati della classe: sono dichiarati quindi come funzioni amiche.

374

Esempio 1: friend

```
class Punto {
..
// Dichiarazione di funzione amica
   friend double calcolaDistanza(const Punto& p1, const Punto& p2);
};
…
double calcolaDistanza(const Punto& p1, const Punto& p2) {
   double distanza = sqrt(pow(p2._x - p1._x, 2) + pow(p2._y - p1._y, 2));
   return distanza;
}

int main() {
   Punto punto1(1, 2);
   Punto punto2(4, 6);

   double distanza = calcolaDistanza(punto1, punto2);
   cout << "Distanza tra i due punti: " << distanza << endl;
   return 0;
}
```

E123

375

Esempio 2: friend

```
class Film {
...
   // Dichiarazione di funzione amica
   friend bool stessoAnno(const Film& f1, const Film& f2);
} ;
....
// Definizione della funzione amica
bool stessoAnno(const Film& f1, const Film& f2) {
   return f1._anno == f2._anno;
}

int main() {
   Film film1("Matrix", 120, 2000);
   Film film2("Solaris", 90, 1995);

   cout << "Film 1 e Film 2 sono stati realizzati nello stesso anno? ";
   if (stessoAnno(film1, film2)) cout << "Sì" << endl;
   else cout << "No" << endl;
   return 0;
}
```

E124

376

Il codice senza utilizzare le funzioni amiche

```
// Funzione per verificare se due film sono stati realizzati nello stesso anno

bool stessoAnno(const Film& f1, const Film& f2) {
   return f1.getAnno() == f2.getAnno();
}
```

E124 

- Invece di utilizzare le funzioni amiche (keyword: *friend)*, è possibile ottenere lo stesso risultato utilizzando i modificatori e i selettori appropriati.

377

Operatori di inserimento (<< *cout*) ed estrazione (>> *cin*)

```
class Punto {
  ...
  // operatori
  friend ostream& operator<<(ostream& os, const Punto& p);
  friend istream& operator>>(istream& is, Punto& p);
  ...
};
ostream& operator<<(ostream& os, const Punto& p){
  os << '(' << p._x << ',' << p._y << ')';
  return os;
}
istream& operator>>(istream& is, Punto& p){
  is >> p._x >> p._y;
  return is;
}
```

378

Operatori di inserimento (<< cout) ed estrazione (>> cin)

- Esempio: uso degli operatori sovraccaricati di inserimento, di estrazione e di somma binaria:

```
int main() {
      Punto p, q;
      cout << "Inserire le coordinate di due punti: ";
      cin >> p >> q;
      cout << "La somma di " << p << " e " << q
           << " vale " << p + q << endl;
      cout << "La distanza di " << p << " da " << q
           << " vale " << p.distanza(q) << endl;
      return 0;
}
```

E116



379

Overloading degli operatori di inserimento ed estrazione

```
class Film {
…
   friend ostream& operator<<(ostream& os, const Film& film);
   friend istream& operator>>(istream& is, Film& film);
}
…
ostream& operator<<(ostream& os, const Film& film) {
   os << "Titolo: " << film.titolo << ", Durata: " << film.durata << " minuti, Anno: " << film.anno;
   return os; }

istream& operator>>(istream& is, Film& film) {
   char temp[100];
   is >> temp;
   film.titolo = new char[strlen(temp) + 1];
   strcpy(film.titolo, temp);
   is >> film.durata >> film.anno;
   return is; }

int main() {
   Film film1("Matrix", 120, 1999);
   cout << "Film 1: " << film1 << endl;

   Film film2;
   cout << "Inserisci il titolo, la durata e l'anno del film: ";
   cin >> film2;
   cout << "Film 2: " << film2 << endl;
   return 0;}
```

Adesso è possibile stampare e inserire attributi a video utilizzando *cin* e *cout*

E127



380

Overloading dell'operatore *new*

```
class Film {
...

   void* operator new(size_t size);
}

void* Film::operator new(size_t size) {
   cout << "Allocare memoria per un oggetto Film..." << endl;
   return ::operator new(size);//invocare l'operatore new definito a livello globale
}


int main() {
   // Creazione di un oggetto Film utilizzando l'overloading di operatore new
   Film* filmPtr = new Film("Matrix", 120, 1999);
   filmPtr->stampaFilm();

   // Deallocazione della memoria
   delete filmPtr;
   return 0;
}
```

E128



381

size_t

- ..è un tipo di dato utilizzato per rappresentare la dimensione di oggetti in memoria, in particolare la dimensione di un'area di memoria allocata. Può rappresentare solo valori positivi o zero.
- Il motivo principale per cui size_t è utilizzato per rappresentare la dimensione è che può essere garantito di essere in grado di rappresentare la dimensione di qualsiasi oggetto allocabile in memoria.
- La dimensione effettiva di size_t può variare a seconda dell'implementazione del compilatore e del sistema operativo, ma è generalmente abbastanza grande da gestire anche gli oggetti più grandi che è possibile allocare.

382

Esempio completo: libreria

- Supponiamo di aver bisogno di fare un registro dei libri in biblioteca:
  - Abbiamo bisogno di una classe *Libro* ed una classe **composta** *Libreria* che contiene dei libri
  - La classe *Libro* rappresenta un singolo libro con attributi come *autore, titolo* e *anno* di stampa
  - La classe *Libreria* rappresenta una libreria che contiene un array dinamico di libri
  - Creiamo il metodo *inserisciLibro* per aggiungere un libro alla libreria e il metodo *getNumLibri* per ottenere il numero di libri nella libreria
  - Nel *main*, chiediamo all'utente quanti libri contiene la libreria e quindi chiediamo i dettagli di ciascun libro da inserire nella libreria

383

Esempio biblioteca: classe libro

```
class Libro {
    char* _autore;
    char* _titolo;
    int _anno;
public:
    // Costruttori
    Libro();
    Libro(const char* autore, const char* titolo, int anno);
    Libro(const Libro& other);  // costruttore di copia
    Libro& operator=(const Libro& other);   // operatore di assegnazione

    // Distruttore
    ~Libro();

    // Metodi di accesso
    const char* getAutore() const;
    void setAutore(const char* autore);
    const char* getTitolo() const;
    void setTitolo(const char* titolo);
    int getAnno() const;
    void setAnno(int anno);
    void stampa() const;
};
```



E101

384

Costruttore di Libro



```
Libro::Libro(const char* autore, const char*
titolo, int anno) : _anno(anno) {
    _autore = new char[strlen(autore) + 1];
    strcpy(_autore, autore);


    _titolo = new char[strlen(titolo) + 1];
    strcpy(_titolo, titolo);
}
```

E101

385

Esempio biblioteca: classe Libreria

```
class Libreria {
    int _id;
    Libro* _libri;
    int _numLibri;
public:
    Libreria(int id);
    ~Libreria();
    void inserisciLibro(const Libro& libro);
    int getNumLibri() const;
    void stampaLibri() const;
};
```



E101

```
void Libreria::inserisciLibro(const Libro& libro) {
    Libro* temp = new Libro[_numLibri + 1];

    for (int i = 0; i < _numLibri; ++i) {
        temp[i] = _libri[i];   // usa operator=
    }


    temp[_numLibri] = libro;   // usa operator=

    delete[] _libri;
    _libri = temp;
    _numLibri++;
}
```

Riallocazione dinamica

386

```
Libro& Libro::operator=(const Libro& other) {
    if (this != &other) {
        delete[] _autore;
        delete[] _titolo;

        _anno = other._anno;

        if (other._autore != nullptr) {
            _autore = new char[strlen(other._autore) + 1];
            strcpy(_autore, other._autore);
        } else {
            _autore = nullptr;
        }


        if (other._titolo != nullptr) {
            _titolo = new char[strlen(other._titolo) + 1];
            strcpy(_titolo, other._titolo);
        } else {
            _titolo = nullptr;
        }
    }

    return *this;
}
```



E101

Serve un operatore di assegnamento

387

Esempio biblioteca: il main()

```
int main() {
    int numLibri;

    cout << "Quanti libri contiene la libreria? ";
    cin >> numLibri;
    Libreria libreria(1); // Creiamo una libreria con l'identificatore  1
    for (int i = 0; i < numLibri; ++i) {

        //CHIEDERE L'UTENTE DI INSERIRE I DATI…

        Libro libro(autore, titolo, anno); //creare un oggetto di tipo libro
        libreria.inserisciLibro(libro); //inserire un libro in libreria
    }

    cout << "La libreria contiene " << libreria.getNumLibri() << " libri." << endl;
    return 0;
}
```



E101

388



Esempio biblioteca: estenzioni

- Un nuovo metodo della classe Libreria per trovare il libro più vecchio.
- Il metodo restituisce il puntatore al libro più vecchio della libreria

```
Libro* Libreria::trovaLibroPiuVecchio() const {
   if (_numLibri == 0) return nullptr;

   Libro* libroPiuVecchio =  libri[0];
   for (int i = 1; i < _numLibri; ++i) {
      if (libri[i]->getAnno() < libroPiuVecchio->getAnno()) {
         libroPiuVecchio = _libri[i];
      }
   }
   return libroPiuVecchio;
}
```



E102

389



Classi e template di funzione

- Quando una classe viene utilizzata per istanziare un template di funzione, essa deve sovraccaricare gli operatori che il template di funzione applica agli oggetti della classe stessa.

390

Classi e template di funzione

- Esempio. Template di funzione BubbleSort:

```
template<typename T>
void BubbleSort (T v[], int n) {
   int i, j;
   T tmp;
    for (i = n - 1; i > 0; i--)
         for (j = 0; j < i; j++)
            if (v[j] > v[j+1]) {
                        tmp = v[j];
                        v[j] = v[j+1];
                        v[j+1] = tmp;
            }
}
```

E050

Volendo applicare il template di funzione E050 a una classe, p.e., la classe *Impiegato*, bisogna sovraccaricare l'operatore di maggiore >

391

Classi e template di funzione

```
class Impiegato {
   char _nome[32];
   char _cognome[32];

public:
   Impiegato(const char* nome, const char* cognome);

   bool operator>(const Impiegato& imp) const;
   void stampa() const;
};
```

È necessario definire quando un impiegato è maggiore dell'altro.

Un esempio potrebbe essere considerare l'ordine alfabetico

```
bool Impiegato::operator>(const Impiegato& imp) const
{
   if (strcmp(_cognome, imp._cognome) > 0) return true;
    return false;
}
```



E119

392

Classi e template di funzione

- Esempio 2: BubbleSort per la classe *Film* sovraccarico dell'operatore >

Per due oggetti di tipo *Film* si potrebbe considerare, p.e.:

```
bool operator>(const Film& film1, const Film& film2)
{
    return film1.getDurata() > film2.getDurata();
}
```

- la durata,
- la data di produzione dei film (dove maggiore significa più recente).

```
bool operator>(const Film& film1, const Film&
film2) {
   return film1.getAnno() < film2.getAnno();
}
```

E120



393

Template di classe

- I template di classe forniscono un meccanismo per definire classi generiche, ovvero classi in cui uno stesso attributo può assumere differenti tipi.
- Esempio: abbiamo definito la classe Punto in modo tale che le coordinate *x* e *y* siano rappresentate come due attributi di tipo double.

```
class Punto {
      ...
private:
     double _x;
     double _y;
};
```

394



Template di classe

- Le coordinate di un punto 2D potrebbero essere rappresentate sia con attributi di tipo float che con numeri interi in spazi discreti.

```
class PuntoDouble {
      ...
private:
      double _x;
      double _y;
};
class PuntoFloat {
      ...
private:
      float _x;
      float _y;
};
```

Posso evitare di scrivere tante classi Punto diverse, che hanno tutte gli stessi attributi e gli stessi metodi e differiscono solo per i tipi che utilizzano?

395



Template di classe

- I template di classe permettono di definire un modello di classe applicabile a diversi tipi di dato. Nel caso della classe Punto:

```
template <typename T>
class Punto {
public:
      Punto();
      Punto(T x, T y);
      ...
      T getX() const;
      T getY() const;
      ...
private:
      T _x;
      T _y;
};
```

- Quando il compilatore analizzerà la dichiarazione di un oggetto della classe, produrrà un codice in cui T viene sostituito dal tipo effettivamente utilizzato.

396



Template di classe

- Esempio:

```
int main() {
  Punto<double> p(3.5, 2.8);
  Punto<int> q(6, 8);

  cout << "Coordinate di p: "
       << p.getX() << ", " << p.getY() << endl;
  cout << "Coordinate di q: "
       << q.getX() << ", " << q.getY() << endl;

  return 0; }
```

- Nella dichiarazione della variabile p il tipo generico T viene sostituito con double; nella dichiarazione di q, T viene sostituito con int.

397



Template di classe

- Per riassumere la sintassi:

  `template <typename` T_1, ..., `typename` T_M`>`
  `class` *nome_classe* **`{ ... };`**

- Note:
  - Al posto di **`typename`** si può talvolta trovare la parola chiave **`class`**.
  - Nella dichiarazione di un oggetto di una classe template, il tipo da utilizzare viene esplicitato:
    `Punto<double> p(3.5, 2.8);`
  - Le classi template possono avere più di un tipo generico. Ad esempio:

```
template <typename S, typename T>
class Test { ... };
```

398



Template di classe

- L'implementazione di un template di classe richiede la definizione delle funzioni membro.
- A tal fine si usa la seguente sintassi:

Costruttore (e distruttore):

template <typename T_1, …, typename T_M>
nome_classe<T_1, …, T_M>::*nome_classe()*
{ *corpo* }

Funzione membro:

template <typename T_1, …, typename T_M>
tipo_ritorno nome_classe<T_1, …, T_M>::*nome_funzione(parametri)*
{ *corpo* }

399



Template di classe

- Esempio (funzioni membro della classe Punto):

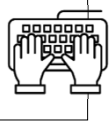
```
template<typename T>
Punto<T>::Punto(T x, T y) {       // costruttore
      _x = x;                     // con parametri
      _y = y;
}

template<typename T>
Punto<T>::~Punto() {}             // distruttore

template<typename T>
T Punto<T>::getX() const {        // selettore
      return _x;                  // per x
}

template<typename T>
T Punto<T>::getY() const {        // selettore
      return _y;                  // per y
}
```

E117



400

Template di classe: più di uno tipo generico

Esempio: un template di classe *Coppia* che rappresenta una coppia ordinata di valori di tipi generici

```
template <typename T, typename U>
class Coppia {
private:
   T _primo;
   U _secondo;
public:
   Coppia(T p, U s); // Costruttore
   void stampaCoppia() const;     // Metodo stampa
};
```

401

Template di classe: più di uno tipo generico

Esempio: un template di classe *Coppia* che rappresenta una coppia ordinata di valori di tipi generici

```
template <typename T, typename U>
Coppia<T, U>::Coppia(T p, U s) : _primo(p), _secondo(s) {}

template <typename T, typename U>
void Coppia<T, U>::stampaCoppia() const {
   cout << "Primo elemento: " << _primo
        << ", Secondo elemento: " << _secondo << endl;
}
```

402

Template di classe: più di uno tipo generico

Nel main è possibile creare diverse coppie di valori appartenenti a due tipi qualsiasi, come ad esempio *int* e *float*, oppure *int* e *char**

```
int main() {

   //la prima coppia è composta da un intero e un double
   Coppia<int, double> coppia1(10, 3.14);
   coppia1.stampaCoppia();

   //la seconda coppia è composta da un intero e una stringa
   Coppia<int, const char*> coppia2(25, "Smith");
   coppia2.stampaCoppia();

   return 0;
}
```

E118



403

Cosa viene stampato sul schermo?

```
template <typename T, typename U>
class Punto
{
   T _x;
   U _y;

public:
   Punto();
   Punto(T x, U y);

   void scala(T param);
   void stampaPunto();
};

template <typename T, typename U>
Punto<T,U>::Punto()
{
   _x = -1;
   _y = -1;
}

template <typename T, typename U>
Punto<T,U>::Punto(T x, U y)
{
   _x = x;
   _y = y;
}

template <typename T, typename U>
void Punto<T,U>::scala(T param)
{
 _x*=param;
 _y*=param;

}

template <typename T, typename U>
void Punto<T,U>::stampaPunto()
{
   cout << "Coordinate:(" << _x << ", " << _y << ")" << end
}

int main() {
   Punto<double,int> p(3, 2.8);
   p.scala(1.5);
   p.stampaPunto();

   return 0;
}
```

wooclap



W311



404

Cosa viene stampato sul schermo?

```
template <typename T, typename U>
class Punto
{
   T _x;
   U _y;

public:
   Punto();
   Punto(T x, U y);

   void scala(T param);
   void stampaPunto();
};

template <typename T, typename U>
Punto<T,U>::Punto()
{
   _x = -1;
   _y = -1;
}

template <typename T, typename U>
Punto<T,U>::Punto(T x, U y)
{
   _x = x;
   _y = y;
}

template <typename T, typename U>
void Punto<T,U>::scala(T param)
{
 _x*=param;
 _y*=param;
}

template <typename T, typename U>
void Punto<T,U>::stampaPunto()
{
   cout << "Coordinate del punto: (" << _x << ", "
<< _y << ")" << endl;
}

int main() {
   Punto<int,double> q(6, 8);
   q.scala(0.5);
   q.stampaPunto();
   return 0;
}
```

wooclap



W312



405

Utilizzo dei template di classe con classi come tipi generici

Estensione del E118, supponiamo di voler utilizzare il template di classe *Coppia* con le classi *Person* e *Film*

```
…
template <typename T, typename U>
 void Coppia<T, U>::stampaCoppia() {
   cout << "Primo : " << _primo << ", Secondo : " << _secondo << endl;
 }

int main() {
   Coppia<int, double> coppia1(10, 3.14);
   coppia1.stampaCoppia();

   Coppia<Person, Film> coppia2(Person("John","Smith"),Film("Shrek",120,2000));
   coppia2.stampaCoppia();
   return 0;
}
```

In questo caso è necessario far
l'overloading dell'operatore <<



E130

406

Utilizzo dei template di classe con classi come tipi generici

- Per ogni classe (e.g., Film, Person) è necessario definire un overloading specifico
- Esempi di overloading dell'operatore << tramite funzioni amiche:

```
ostream& operator<<(ostream& os, const Film& film) {
    os << "Titolo: " << film._titolo << ", Durata: " << film._durata << " minuti, Anno: "
<< film._anno;
    return os;
}

ostream& operator<<(ostream& os, const Person& p){
    os << "Name: " << p._nome << ", Cognome: " << p._cognome;
    return os;
}
```

E130



407

Due (o più) tipi generici possono essere istanziati con lo stesso tipo specifico

```
template <typename T, typename U>
class Coppia {
    T _primo;
    U _secondo;
public:
    Coppia(T p, U s) : _primo(p), _secondo(s) {}
    void stampaCoppia();
}
template <typename T, typename U>
 void Coppia<T, U>::stampaCoppia() {
    cout << "Primo elemento: " << _primo << ", Secondo elemento: " << _secondo << endl;
 }

int main() {
    Coppia<double, double> coppia1(10, 3.14);
    coppia1.stampaCoppia();
    Coppia<Person, Person> coppia2(Person("John", "Smith"),Person("Mary", "Smith"));
    coppia2.stampaCoppia();
    return 0;
}
```

E130



408

possibile, utilizzando i template con le classi come tipi, chiamare un metodo della classe indicata come parametro generico..

```
template <typename T, typename U>
class Coppia {
   T _primo;
   U _secondo;

public:
   Coppia(T p, U s) : _primo(p), _secondo(s) {}
   void stampaCoppia();
};

 void Coppia<T, U>::stampaCoppia() {
   cout << "Primo elemento: ";
   _primo.stampa();
   cout << ", Secondo elemento: ";
   _secondo.stampaFilm();
   cout << endl;
   }
```

…a condizione che il metodo specifico sia definito per la classe data

In questo esempio, invece di utilizzare operatore << vengono usati i metodi *stampa* e *stampaFilm* per stampare su console

E132



409

possibile, utilizzando i template con le classi come tipi, chiamare un metodo della classe indicata come parametro generico?

Attenzione: il template di classe come definito in E132 non funziona con i tipi semplici come *int* e *double*!

```
int main() {
   //Coppia<int, double> coppia1(10, 3.14);
   //coppia1.stampaCoppia();

   Coppia<Person, Film>    coppia2(Person("John","Smith"),Film("Shrek",120,2000));
   coppia2.stampaCoppia();

   return 0;
}
```

E132



410



Template di classe

Gli argomenti dei template non sono limitati ai tipi, ma possono anche essere valori costanti o nomi di funzioni. Ad esempio:

```
template<typename T, int n>
class Vettore {
      T dati[n];
      ...
};
Vettore<double, 100> v; // vettore di 100 double
```

Il valore (100) è noto al momento di compilazione

411



Template di classe

Gli argomenti dei template non sono limitati ai tipi, ma possono anche essere valori costanti o nomi di funzioni. Ad esempio:

```
template<typename T, int n>
class Vettore {
      T dati[n];
      ...
};
const int a=100;
Vettore<double, a> v; // vettore di 100 double
```

Il valore (100) è noto al momento di compilazione

Il codice crea un vettore *v* per memorizzare 100 double

412



Esempio del *template di classe* con dei valori costanti come argomenti

n è un valore costante, cioè la dimensione del vettore

E135



```
#include <iostream>
#include <cstdlib>
#include <ctime
using namespace std;

template<typename T, int n>
class Vettore
{
   T dati[n];
public:
   Vettore();
   void inserisciCasuale();
   void stampa();
};
```

```
template<typename T, int n>
Vettore<T,n>::Vettore(){
}

template<typename T, int n>
void Vettore<T,n>::inserisciCasuale() {

   for (int i = 0; i < n; ++i)  dati[i] = rand() ;
}

template<typename T, int n>
void Vettore<T,n>::stampa()
{
   for (int i = 0; i < n; i++)  cout << dati[i] << " ";
   cout << endl;
}
```

413



Esempio del *template di classe* con dei valori costanti come argomenti

```
int main()
{
   srand(time(0));
   const int a=5;

   Vettore<double, a> v1;
   v1.inserisciCasuale();
   v1.stampa();

   Vettore<int, 10> v2;
   v2.inserisciCasuale();
   v2.stampa();
   return 0;
}
```

E135



Questo codice definisce una classe chiamata *Vettore,* la quale è progettata per memorizzare un insieme di dati di un tipo specifico (simile ad un array 1D ma con dei metodi e.g., per stampare i dati)

Il numero di elementi del vettore è noto al momento della compilazione del programma

La classe *Vettore* è progettata in modo da poter essere utilizzata con diversi tipi di dati

414



Il template *Vettore* funziona per le classi, p.e., classe *Person*

```
template<typename T, int n>
class Vettore
{
   T dati[n];
public:
   Vettore();
   void stampa();
};
template<typename T, int n>
Vettore<T,n>::Vettore() { }

template<typename T, int n>
void Vettore<T,n>::stampa() {
   for (int i = 0; i < n; i++)
         cout  << dati[i] << " ";
}
```

```
ostream& operator<<(ostream& os, const Person& p)
{
   os << "Name: " << p._nome << ", Cognome: " << p._cognome;
   return os;
}

int main()
{
   srand(time(0));
   Vettore<Person, 2> v1;
   v1.stampa();
   Vettore<int, 8> v2;
   v2.stampa();

   return 0;
}
```

E136



Per usare la classe *Person* è necessario l'overloading del operatore <<
Due oggetti di tipo *Person* vengono inizializzati con il costruttore di default

415



Esempio del template di classe con delle funzioni come argomenti

Un'altra versione di calcolatrice
- 3 funzioni = 3 operazioni principali
- template di classe *Calcolatrice*, per creare uno oggetto di tipo *Calcolatrice*
- ogni Calcolatrice che può fare solo una operazione

E137



```
#include <iostream>
using namespace std;

int somma(int a, int b) {
   return a + b; }

int sottrazione(int a, int b) {
   return a - b;}

int moltiplicazione(int a, int b) {
   return a * b;}
```

```
template<typename T, int Funzione(T, T)>
class Calcolatrice {
   T x;
   T y;

public:
   Calcolatrice(T val1, T val2) : x(val1), y(val2) {}

   T eseguiOperazione() {
      return Funzione(x, y);
   }
};
```

416



Esempio del template di classe con delle funzioni come argomenti

Un'altra versione di calcolatrice

- creiamo 3 oggetti e seguiamo tre volte il metodo *eseguiOperazione*

```
int main() {
    Calcolatrice<int, somma> calcSomma(5, 3);
    Calcolatrice<int, sottrazione> calcSottrazione(5, 3);
    Calcolatrice<int, moltiplicazione> calcMoltiplicazione(5, 3);

    int risultatoSomma = calcSomma.eseguiOperazione();
    cout << "Il risultato della somma è: " << risultatoSomma << endl;

    int risultatoSottrazione = calcSottrazione.eseguiOperazione();
    cout << "Il risultato della sottrazione è: " << risultatoSottrazione << endl;

    int risultatoMoltiplicazione = calcMoltiplicazione.eseguiOperazione();
    cout << "Il risultato della moltiplicazione è: " << risultatoMoltiplicazione << endl;
    return 0;
}
```

E137



417



- E137 da risultati corretti solo per i numeri interi
- Per creare una calcolatrice generica dobbiamo usare sia *template di classi* che *template di funzioni*

```
#include <iostream>
using namespace std;

template<typename W>
W somma(W a, W b) {
    return a + b;}

template<typename Z>
Z sottrazione(Z a, Z b) {
    return a - b;}

template<typename T>
T moltiplicazione(T a, T b) {
    return a * b;}
```

```
template<typename T, T Funzione(T, T)>
class Calcolatrice {
    T x;
    T y;
public:
    Calcolatrice(T val1, T val2);
    T eseguiOperazione();
};

template<typename T, T Funzione(T, T)>
Calcolatrice<T, Funzione>::Calcolatrice(T val1, T val2) :
x(val1), y(val2) {}

template<typename T, T Funzione(T, T)>
T Calcolatrice<T, Funzione>::eseguiOperazione() {
    return Funzione(x, y);
}
```



E138

418

Template con funzione come parametro

- I template può parametrizzare non solo tipi, ma anche funzioni
  - permette di scegliere il comportamento della classe in fase di compilazione
- Il template riceva **una funzione**
- Diversi oggetti di stesso template usano implementazioni diversi per fare «la stessa» operazione

419

Template con funzione come parametro

- Esempio 2: il sistema che gestisce esami. Il voto finale può essere calcolato in modi diversi a seconda del corso o del docente. **Voglio calcolare il voto finale di un esame usando criteri diversi senza duplicare codice.**
  - unica classe (Esame) che rappresenti un esame con voto scritto e orale,
  - posso **cambiare la logica di calcolo del voto finale** (media semplice, media pesata, ecc.) in modo flessibile.
- Il problema viene risolto il template che accetta un **puntatore a funzione ovvero**
  - un **template di classe con parametro funzione**

420

Template con funzione come parametro

```
double mediaSemplice(double a, double b) {
    return (a + b) / 2.0;
}

double mediaPesata(double a, double b) {
    return a * 0.7 + b * 0.3;
}

// Template con funzione come parametro
template <double (*F)(double, double)>

class Esame {
double scritto;
    double orale;

public:
    Esame(double s, double o) : scritto(s), orale(o) {}

    double votoFinale() const {
        return F(scritto, orale);
    }
};
```

```
int main() {

    Esame<mediaSemplice> e1(24, 28);
    Esame<mediaPesata> e2(24, 28);

    cout << «Semplice: " << e1.votoFinale() << endl;
    cout << «Pesata: " << e2.votoFinale() << endl;

    return 0;
}
```

`F` è un **parametro «non di tipo»** (non-type template parameter)

E131



421

Stesso risultato con il template classi

```
template < typename T>
class Esame {
    double scritto;
    double orale;
public:
     Esame(double s, double o);
    double votoFinale() const;
};

template < typename T>
Esame<T>::Esame(double s, double o) :
scritto(s), orale(o) {}

template < typename T>
double Esame<T>::votoFinale() const {
    return T::calcola(scritto, orale);
}
```



E133

```
class MediaSemplice {
public:
    static double calcola(double a, double b) {
        return (a + b) / 2.0;
    }
};

class MediaPesata {
public:
    static double calcola(double a, double b) {
        return a * 0.7 + b * 0.3;
    }
};

int main() {
    Esame<MediaSemplice> e1(24, 28);
    Esame<MediaPesata> e2(24, 28);

    cout << e1.votoFinale() << endl;
    cout << e2.votoFinale() << endl;
}
```

422

template di classe e template di funzione insieme

```
template <typename T, typename U>
class Coppia {
   T primo;
   U secondo;

public:
   Coppia(T p, U s) : primo(p), secondo(s) {}
   void stampaCoppia();

   template <typename Z>
   void stampaOggetto(const Z& oggetto);
};

template <typename T, typename U>
 void Coppia<T, U>::stampaCoppia() {
   cout << "Primo elemento: ";
   stampaOggetto(primo);
   cout << ", Secondo elemento: ";
   stampaOggetto(secondo);
}

template <typename T, typename U>
template <typename Z>
 void Coppia<T, U>::stampaOggetto(const Z& oggetto) {
          oggetto.stampa();
}
```

Il nuovo template di funzione stampaOggetto che appartiene a template di classe Coppia usa un tipo generico Z

- *Z* può corrispondere sia a tipo *U* che al tipo *T*
- *Da nottare l'utilizzo «doppio» di parola chiave template*

E143

423

template di classe e template di funzione insieme

```
int main() {

   Coppia<Film, Person> coppia1(Film("Shark", 125, 1975),Person("Steven", "Spielberg") );
   coppia1.stampaCoppia();

   Coppia<Person, Film> coppia2(Person("Andrew", "Adamson"), Film("Shrek", 120, 2001));
   coppia2.stampaCoppia();

   return 0;
}
```

E143



- Adesso si può creare le coppie in qualsiasi ordine

424

Definizione dei metodi di una classe

- L'implementazione (definizione) dei metodi di una classe si può trovare:
  - All'interno della dichiarazione stessa della classe. I metodi definiti all'interno della dichiarazione della classe si dicono metodi in linea (in-line);
  - Nello stesso file contenente la definizione, ma dopo la dichiarazione stessa;
  - In un file separato rispetto a quello che contiene la dichiarazione.
- Nel secondo e nel terzo caso si parla di metodi fuori linea (out-of-line).

425

Definizione dei metodi di una classe

- Quando l'implementazione fuori linea si trova in un file separato, occorre creare:
  - Un file di intestazione (header) contenente la dichiarazione della classe. Tale file in genere termina con l'estensione `h`
  - Un file contenente le definizioni dei metodi. Tale file in genere termina con l'usuale estensione `cpp`
  - Il file contenente le definizioni dei metodi deve includere il file di intestazione con la direttiva **`#include <`**file_intestazione.h**`>`**
  - Generalmente i due file hanno lo stesso nome, ma estensioni diverse (`.h` e `.cpp`).

426

Definizione dei metodi di una classe

- Esempio (dichiarazione in linea):

```
class Punto  {
public:
      Punto() {_x = 0.0; _y = 0.0;};
      Punto(double x, double y)
               {_x = x; _y = y;};
      Punto(const Punto& p)
               {_x = p._x; _y = p._y;};
      ~Punto() {};
      double getX() const {return _x;};
      double getY() const {return _y;};
      ...
private:
      double _x;
      double _y;
};
```

427

Definizione dei metodi di una classe

- Esempio (dichiarazione fuori linea in due file):

```
class Punto  {
public:
      Punto();
      Punto(double x, double y);
      Punto(const Punto& p);
      ~Punto();
      double getX() const;
      double getY() const;
      ...
private:
      double _x;
      double _y;
};
```

File Punto.h

428

Definizione dei metodi di una classe

- Esempio (dichiarazione fuori linea in due file):

```
#include "Punto.h"

Punto::Punto() {
      _x = 0.0; _y = 0.0;
}
...

Punto::~Punto() {}

double Punto::getX() const {
      return _x;
}
...
```

File Punto.cpp

429

Definizione dei metodi di una classe

- Esempio (dichiarazione fuori linea in due file):

```
#include <iostream>
#include "Punto.h"

int main() {
   Punto p(30, 40);
   Punto q;
   cout << p.getX() << ", "
        << p.getY() << endl;
   cout << q.getX() << ", "
        << q.getY() << endl;
   ...
}
```

File main.cpp

430

Definizione dei metodi di una classe

- Una funzione in linea è normalmente più veloce di una fuori linea.
- Si può fare in modo che anche una funzione fuori linea benefici dei vantaggi delle funzioni in linea, facendola precedere dalla parola chiave `inline`.
- Non è garantito, tuttavia, che una funzione definita in linea o fuori linea, ma usando `inline`, sia effettivamente trattata come una funzione in linea: ciò dipende dal compilatore!

431

Definizione dei metodi di una classe: esempio *inline*

- Esempio (funzioni dichiarate `inline`):

```
class Punto {
...
};


inline double Punto::getX() const
{
      return _x;
}
inline double Punto::getY() const
{
      return _y;
}
...
```

File Punto.h

433

Definizione dei metodi di una classe

- Separare la dichiarazione di una classe dalla definizione dei suoi metodi significa separare l'interfaccia della classe (la dichiarazione) dalla sua implementazione (la definizione).
- La definizione dei metodi può anche essere fornita sotto forma di codice oggetto. In tal modo altri programmatori:
  - Possono utilizzare la classe attraverso l'inclusione del file di intestazione nei loro programmi;
  - Ma non possono accedere all'implementazione.

434

Nascondere l'informazione

- Realizzando una classe con attributi privati e con metodi fuori linea:
  - Abbiamo nascosto la rappresentazione dei dati nella parte privata della classe.
  - Abbiamo nascosto l'implementazione dei metodi in un file separato rispetto a quello contenente la dichiarazione della classe.
- Questo modo di procedere è indicato come occultamento dell'informazione (oppure, in inglese, information hiding).

435



Nascondere l'informazione

- Secondo il principio dell'information hiding, i dettagli implementativi di una classe vengono nascosti al resto del programma.
- In tale modo, a determinate condizioni, una modifica all'implementazione della classe non ha ripercussioni sul resto del programma, finché l'interfaccia pubblica resta invariata.
- Questo approccio è fondamentale per ottenere programmi più robusti, flessibili e riusabili.

436



Astrazione sui dati

- Definire un nuovo tipo significa definire una collezione di valori e un insieme di operazioni.
- Un linguaggio può fornire meccanismi che consentano al programmatore di definire propri tipi, nascondendone gli aspetti implementativi, secondo il principio dell'information hiding.
- Per essere efficaci, questi meccanismi devono impedire l'accesso alla rappresentazione dei dati come accade per i tipi predefiniti.
- Tali meccanismi consentono di realizzare quella che si chiama astrazione sui dati.

437

Esempio 1: la «vecchia» classe *Punto (E089)*

```
class Punto {
   double _x;
   double _y;

public:
   Punto();
   Punto(double x, double y);
   ~Punto();

   void setX(double x);
   void setY(double y);

   double getX();
   double getY();

   void stampaPunto();
};

Punto::Punto() {
   _x = -1;
   _y = -1; }

Punto::Punto(double x, double y) {
   _x = x;
   _y = y; }

void Punto::setX(double x) {
   _x = x;}

void Punto::setY(double y) {
   _y = y;}

double Punto::getX() {
   return _x;}

double Punto::getY() {
   return _y;}

void Punto::stampaPunto() {
   cout << "Coordinate del punto: (" << _x << ", " << _y << ")" << en
}

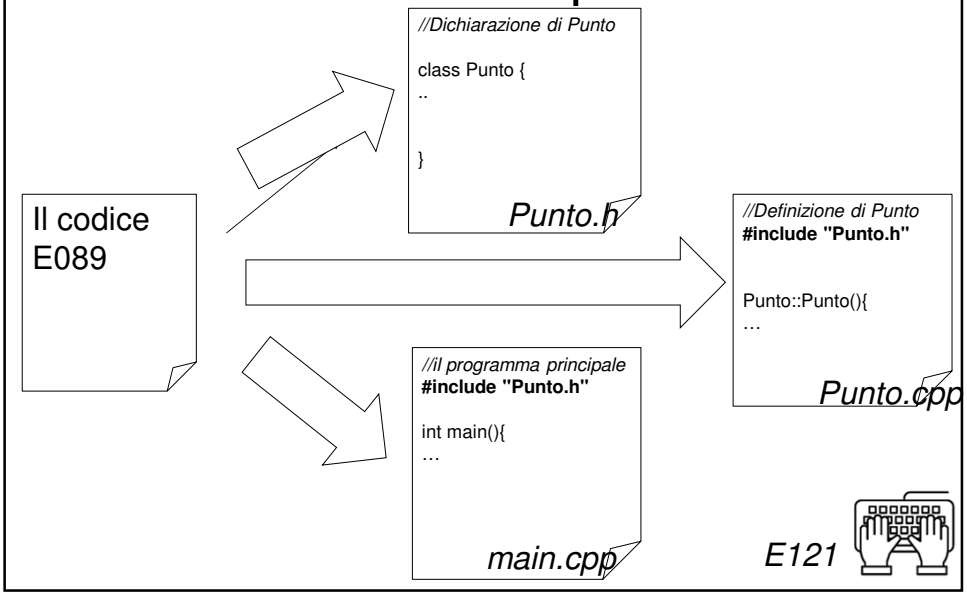
Punto::~Punto() {
   cout << "Questo e' il distruttore" << endl;
}


int main() {
   Punto* p2 = new Punto(10, 20);
   p2->stampaPunto();
   delete p2;
   return 0;
}
```

E089

438

Classe *Punto* – separazione dell'interfaccia dall'implementazione



E121

439

Esempio 2: la «vecchia» classe *Film (E087)*

```
class Film {
   char* _titolo;
   int _durata;
   int _anno;

public:
   Film();
   Film(char* titolo, int durata, int anno);
   Film(const Film& otherFilm);
    ~Film();

   char* getTitolo() const;
   void setTitolo(const char* titolo);
   int getDurata() const;
   void setDurata(int durata);
   int getAnno() const;
   void setAnno(int anno);
   void stampaFilm();
};
```

```
int main() {
   int lunghezzaTitolo;
   cout << "Inserisci la lunghezza del titolo del film: ";
   cin >> lunghezzaTitolo;

   char* titolo = new char[lunghezzaTitolo + 1];

   cout << "Inserisci il titolo del film (mass. " << lunghezzaTitolo <<
   cin >> titolo;

   int durata, anno;
   cout << "Inserisci durata: ";
   cin >> durata;
   cout << "Inserisci anno: ";
   cin >> anno;

   film p(titolo, durata, anno);
   p.stampaFilm();

   delete[] titolo;
   return 0;
}
```

440

Film: implementazione dei metodi

```
Film::Film() {
   _titolo = new char[20];
   strcpy(_titolo, "N/D");
   _durata = 0;
   _anno = 0;}

Film::Film(char* titolo, int durata, int anno) {
   _titolo = new char[strlen(titolo) + 1];
   strcpy(_titolo, titolo);
   _durata = durata;
   _anno = anno;}

Film::Film(const Film& otherFilm) {
   _titolo = new char[strlen(otherFilm._titolo) + 1];
   strcpy(_titolo, otherFilm._titolo);
   _durata = otherFilm._durata;
   _anno = otherFilm._anno;}

Film::~Film() {
   delete[] _titolo;}

char* Film::getTitolo() const {
   return _titolo;}
```

```
void Film::setTitolo(const char* titolo) {
   delete[] _titolo;
   _titolo = new char[strlen(titolo) + 1];
   strcpy(_titolo, titolo);}

int Film::getDurata() const {
   return _durata;}

void Film::setDurata(int durata) {
   _durata = durata;}

int Film::getAnno() const {
   return _anno;}

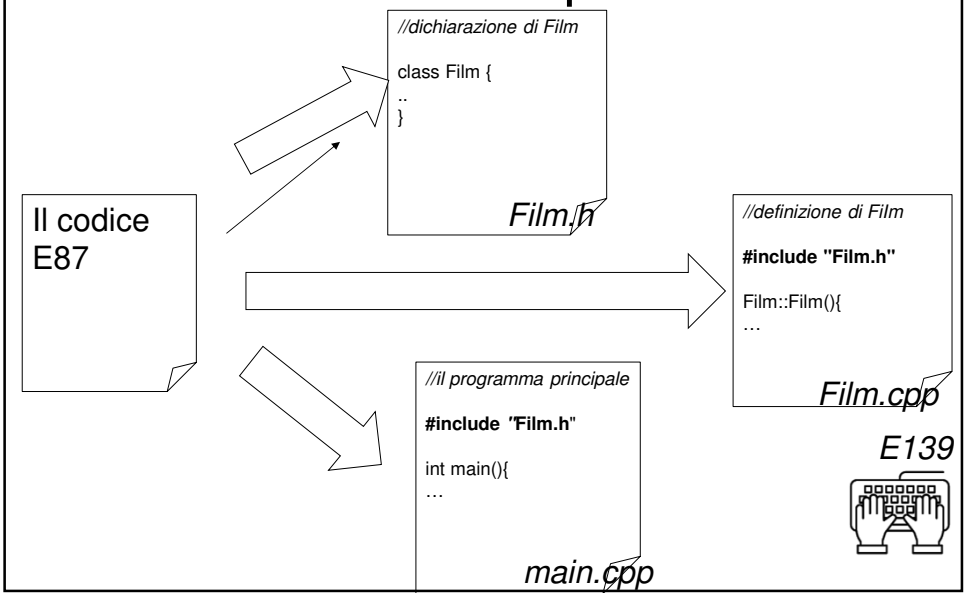
void Film::setAnno(int anno) {
   _anno = anno;}

void Film::stampaFilm() {
   cout << "Titolo: " << _titolo << endl;
   cout << "Durata: " << _durata << endl;
   cout << "Anno: " << _anno << endl;
}
```

E087

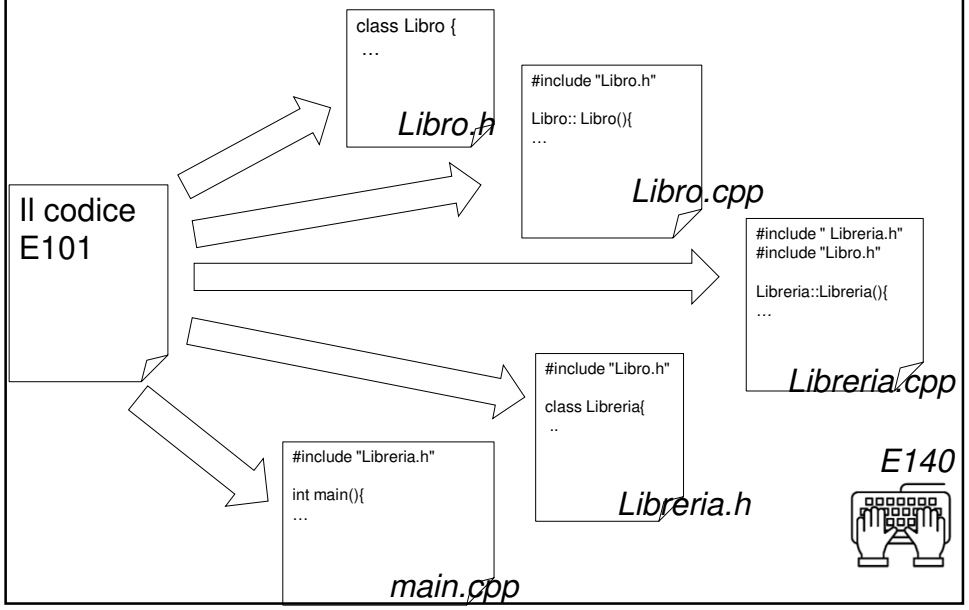
441

Classe Film – separazione dell'interfaccia dall'implementazione



442

Esempio 3: Libro e Libreria – separazione di interfaccia e implementazione



443



Utilità di separazione

- Possiamo scrivere un programma principale (main) conoscendo l'interfaccia della classe *X* anche senza conoscere la sua implementazione.
- Qualcun altro potrà occuparsi di scrivere il codice della classe *X*, seguendo la stessa interfaccia.
- Se l'interfaccia è corretta e rispettata, il nostro programma funzionerà con qualsiasi implementazione compatibile della classe *X*

444



Nuova implementazione di classe Film

- Creo una seconda implementazione di classe *Film* che stampa i commenti in inglese:
  - Non devo modificare né il *main* né *Film.h*.
  - La nuova versione di *Film* implementa la stessa interfaccia di *Film.h*.
  - Tutti i metodi e attributi hanno gli stessi nomi.

```
#include "Film.h"
…
void film::stampaFilm() {
    cout << "Title: " << _titolo << endl;
    cout << "Duration: " << _durata << endl;
    cout << "Year: " << _anno << endl;
}
```



E141

445



E139 Vs. E141

- L'interfaccia della classe Film è la stessa dell'esercizio E139.
- La definizione (implementazione) della classe Film è diversa in E141
- Il main usato è lo stesso dell'esercizio E139.

446



Tipi di dato astratti

- Un tipo di dato astratto (in inglese Abstract Data Type, ADT) è caratterizzato da:
  - Un nome per il tipo
  - Un'implementazione (o rappresentazione)
  - Un insieme di operazioni per la manipolazione dei valori associati al tipo
  - Per ogni operazione, un'implementazione che usi la rappresentazione sopra menzionata
  - Una *"capsula di sicurezza"* che separi i nomi del tipo e delle operazioni dalle loro implementazioni.

447



Tipi di dato astratti

- La rappresentazione (implementazione) di un tipo di dato astratto è detta tipo concreto.
- La superficie esterna della “capsula” che racchiude l’implementazione, ovvero l’insieme dei nomi e dei tipi delle operazioni disponibili è detta interfaccia o segnatura (in inglese signature) del tipo di dato astratto.
- I tipi di dato astratto si comportano come i tipi predefiniti: è possibile dichiarare variabili di un tipo di dato astratto e usare i tipi di dato astratto per la definizione di altri tipi.

448



Specifica

- Chiamiamo specifica la descrizione della semantica (ovvero del significato) delle operazioni di un ADT.
- La specifica può essere espressa in linguaggio naturale o con metodi semi-formali o formali.
- Una specifica è una sorta di contratto tra l’ADT e i suoi clienti (utilizzatori): l’ADT garantisce che l’implementazione delle operazioni soddisfi la specifica.
- Quando ciò accade l’implementazione è corretta rispetto alla specifica.

449



Specifica

- Esempio. Frammento di specifica (informale) in linguaggio naturale per un ADT Poligono:
  - `numEdges`: restituisce il numero di lati del poligono;
  - `area`: calcola l'area del poligono;
  - `perimeter`: calcola la lunghezza del perimetro del poligono;
  - `scale(f)`: riscala il poligono di un fattore *f*;
  - `rotate(alpha)`: ruota il poligono attorno al proprio baricentro di un angolo *alpha*.
  - …

450



Principio di indipendenza dalla rappresentazione

- Due implementazioni corrette di una stessa specifica di un tipo di dato astratto sono osservabilmente indistinguibili da parte dei clienti del tipo di dato astratto.
- Se un tipo gode dell'indipendenza dalla rappresentazione è possibile modificare la sua implementazione con una equivalente senza che si verifichino effetti osservabili nei suoi clienti.

451



Principio di indipendenza dalla rappresentazione

- L'indipendenza dalla rappresentazione dipende dalla correttezza rispetto alle specifiche che non è facile da dimostrare.
- Vi è allora una versione "debole": in un linguaggio sicuro rispetto ai tipi e con tipi di dato astratti, la sostituzione di un ADT con un altro con la stessa interfaccia, ma diversa implementazione, non causa errori di tipo.

452



Tipi di dato astratti in C++

- In C++ gli ADT si realizzano con i meccanismi di incapsulamento offerti dalle classi.
- Il controllo dell'accesso alla rappresentazione di un ADT e la separazione dell'interfaccia dall'implementazione sono ottenuti attraverso:
  - La suddivisione di una classe in una parte pubblica e in una parte privata;
  - La definizione fuori linea dei metodi della classe, in un file sorgente separato dal file di intestazione.
- Attenzione: a rigore il C++ non è un linguaggio sicuro rispetto ai tipi!

453



Paradigma data abstraction

- I principi di information hiding e indipendenza dalla rappresentazione conducono a un nuovo paradigma di programmazione, il paradigma data abstraction.
- Il suo motto può essere sintetizzato in:

  “Decidi quali tipi vuoi; fornisci un insieme completo di operazioni per ogni tipo”.
- Questo paradigma è metodologicamente importante, ma paga una certa rigidità d’uso.

454



Paradigma data abstraction

- Consideriamo ad esempio questa classe C++:

```
enum kind { circle, triangle, square };

class shape {
     point center;
     color col;
     kind k;
     // rappresentazione di shape
public:
     point where();
     void move(point to)
     void draw();
     void rotate(int);
     // altre operazioni
};
```

455



Paradigma data abstraction

- Il “type field” k è necessario per consentire operazioni come draw() e rotate():

```
void shape::draw() {
     switch(k) {
     case circle:
          // draw a circle
          break;
     case triangle:
          // draw a triangle
          break;
     case square:
          // draw a square
          break;
     }
}
```

456



Paradigma data abstraction

- Funzioni come draw() devono “conoscere” tutti i tipi di forme esistenti.
- Il codice di ogni funzione cresce ogni volta che una nuova forma è aggiunta al sistema.
- Se poi viene definita una nuova forma, ogni operazione sul tipo *shape* deve essere esaminata ed eventualmente modificata.
- Ciò è difficile e causa di errori.

457



Paradigma data abstraction

- Il problema è che non c'è distinzione tra le *proprietà generali* di ogni forma (es. avere un colore) e le *proprietà specifiche* di una data forma (un cerchio ha un raggio, ...).
- Inoltre, deve essere possibile selezionare dinamicamente quale operazione invocare; es. un array di forme diverse: per ogni elemento deve essere invocata a run-time la funzione draw della forma corretta.
- Questi problemi sono risolti nel paradigma orientato agli oggetti (object oriented).

458



Moduli e programmazione modulare

- Si possono raggruppare in uno stesso modulo (o package) più ADT correlati dei quali si vuole dare ai clienti una visione limitata.
- Tale approccio, comunemente detto programmazione modulare, è importante per la cosiddetta programmazione in the large.
- Il motto della programmazione modulare può essere sintetizzato in: "Decidi quali moduli vuoi; partiziona il programma in modo da nascondere i dati nei moduli".

459

Le strutture statiche Vs dinamiche

```
const int n = 4;
int a[n];
```

```
int *p;
int n = 0;
cin>>n;
p = new int [n];
```

Strutture dinamiche, p.e., liste

- Non posso cambiare le dimensioni di *a* dopo la compilazione del programma
- Una volta deciso il valore di *n* durante l'esecuzione del programma, non è più possibile modificare le dimensioni dell'array a cui punta *p*
- Posso effettuare una riallocazione dinamica:
  - creo un nuovo array con *n+1* elementi,
  - copio tutti gli elementi,
  - cancello il vecchio array.
- è sempre possibile aggiungere nuovi elementi,
- l'unico limite è la memoria disponibile.

maggiore flessibilità

460

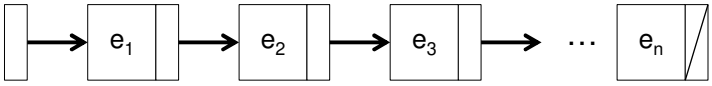
tipi di dato astratti e strutture dati

- Un impiego tipico dei tipi di dato astratti consiste nella realizzazione di strutture dati.
- Le strutture dati sono entità usate per organizzare i dati elaborati da un programma.
- Esempi: array, strutture, array di strutture.
- Tre esempi di strutture dati usate di frequente in informatica:
  - Lista
  - Pila
  - Coda

461

Liste

- Una lista concatenata (linked list) è una sequenza di elementi (detti anche nodi) collegati l'uno all'altro da un puntatore.

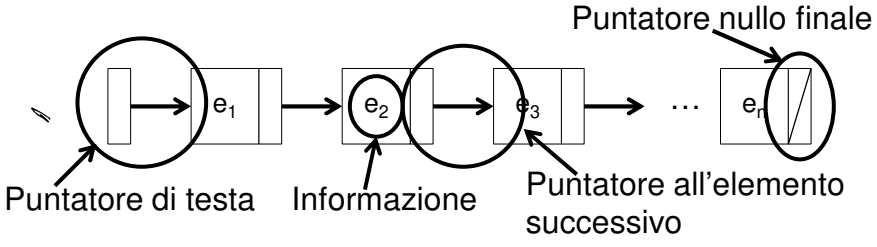


- Una lista concatenata può contenere un numero qualunque di elementi (a meno delle dimensioni della memoria del calcolatore) e può crescere o ridursi a seconda delle esigenze, grazie all'allocazione dinamica.

462

Liste

- Gli elementi si compongono di due parti:
  - L'informazione (di qualunque tipo)
  - Il puntatore all'elemento successivo nella lista.
- La lista inizia con un puntatore che punta al suo primo elemento (puntatore di testa, head pointer) e termina con un puntatore nullo.

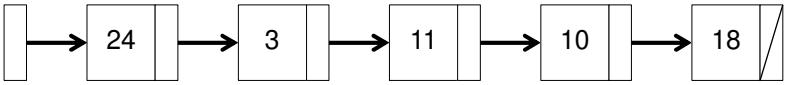


463

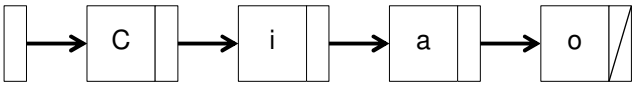


Liste

- Qualche esempio:
  - Una lista di cinque numeri interi



  - Una lista di quattro caratteri alfanumerici



  - Una lista di punti, film, persone…

464



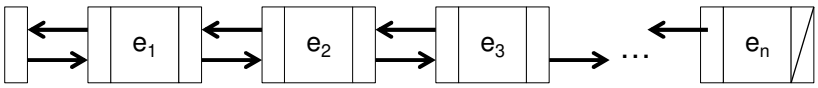
Liste

- Le liste concatenate si possono distinguere in quattro diverse categorie:
  - Liste semplici: ogni elemento contiene un puntatore che lo collega all'elemento successivo.
  - Liste doppiamente concatenate: ogni elemento contiene due puntatori che lo collegano all'elemento successivo e a quello precedente.
  - Liste circolari semplici: l'ultimo elemento contiene un puntatore al primo elemento.
  - Liste circolari doppiamente concatenate: l'ultimo elemento contiene un puntatore al primo elemento e il primo un puntatore all'ultimo.

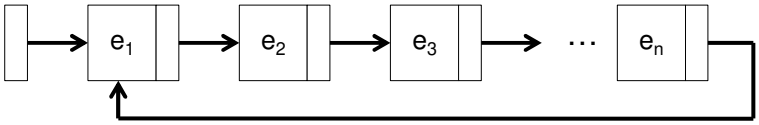
465

Liste

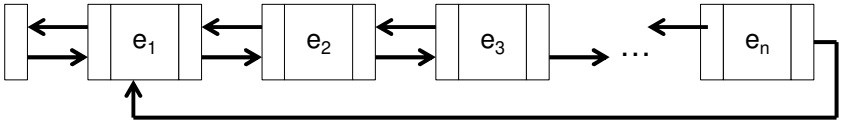
- Lista doppiamente concatenata



- Lista circolare semplice



- Lista circolare doppiamente concatenata



466

Operazioni sulle liste

- Un elenco (non esaustivo) di operazioni applicabili alle liste include:
  - Creazione, distruzione e copia
  - Verifica di proprietà di una lista: ad esempio se una lista è vuota o di quanti nodi è composta
  - Inserimento di nuovi nodi in testa, in coda, in mezzo ad una lista
  - Rimozione di nodi in testa, in coda, in mezzo ad una lista
  - Attraversamento di una lista applicando specifiche operazioni a ciascun nodo

467



Operazioni sulle liste

- Alcune operazioni vengono svolte in modo più efficiente se viene scelta una specifica tipologia di lista. Ad esempio:
  - Le liste semplici permettono di realizzare in modo efficiente l'attraversamento in avanti (ovvero dalla testa alla coda).
  - L'attraverso all'indietro (dalla coda alla testa) è efficiente con le liste doppiamente concatenate.



468



Operazioni sulle liste

- Alcune operazioni vengono svolte in modo più efficiente se viene scelta una specifica tipologia di lista. Ad esempio:
  - Nelle liste semplici l'inserimento in testa è più efficiente dell'inserimento in coda.



469



Liste semplici in C++

- Tipicamente occorre implementare:
  - Il singolo nodo
  - L'intera lista
- Il nodo è spesso implementato come un template di struttura o di classe.
- La lista è in genere un template di classe.
- Esistono librerie che offrono implementazioni già pronte delle liste, ad esempio la Standard Template Library (STL).

470



- Nel seguito due implementazioni di una lista concatenata semplice
  - 1) una lista per memorizzare una sequenza dei numeri interi (una lista concatenata per un tipo specifico: int)
  - 2) una lista generica per qualsiasi tipo (tipo astratto)

471

Esempio 1: interfaccia della classe *Node*

```
class Node {
   int _data;
   Node* _next;

public:
   Node(int data, Node* next);
   Node(const Node& node);
   Node();
   ~Node();

   int getData() const;
   Node* getNext() const;

   void setData(int data);
   void setNext(Node* next);
};
```

- Serve per memorizzare un intero
- oltre a un attributo di tipo *int*, contiene un puntatore a un elemento di tipo *Node*.
- Il puntatore servirà per indicare l'elemento successivo della lista.

472

Esempio 1: implementazione della classe *Node*

Il costruttore di default inizializza gli attributi a zero.

```
Node::Node() : _data(0), _next(0) {}
Node::Node(int data, Node* next): _data(data),
_next(next) {}

Node::Node(const Node& node): _data(node._data),
_next(node._next) { }

Node::~Node() {}

int Node::getData() const {
   return _data;
}

Node* Node::getNext() const {
   return _next;
}

void Node::setData(int data){
   _data = data;
}

void Node::setNext(Node* next){
   _next = next;
}
```



E148A

473

Esempio 2: interfaccia della classe List

```
class List {
   Node* _head;
   int _numItems;

public:
    List();
   ~List();

   Node* getHead() const;
    int getNumItems() const;

   void insertFront(int data);
   void insertBack(int data);
   void insertAfter(Node* p, int data);

   void clear();
   Node* find(int data) const;
   bool isEmpty() const;
   void print() const;
};
```

- Il puntatore al primo elemento (di tipo *Node*)
- Costruttore e distruttore
- Selettori
- Metodi per inserire un nuovo elemento, almeno una di queste operazioni deve essere implementata
- Altre operazioni

474

Classe List

- implementazione del costruttore e distruttore

```
List::List()
    : _head(0), _numItems(0) {}


List::~List()
{
    clear();
}

void List::clear()
{
    Node* p = _head;

    while (p != 0)
    {
        Node* q = p;
        p = p->getNext();
        delete q;
    }

    _head = 0;
    _numItems = 0;
}
```

Scorrere la lista:

- leggere il puntatore al prossimo elemento
- cancellare la memoria dell'elemento attuale



E148B

475

Classe List

- Selettori per gli attributi:

```
Node* List::getHead() const
{
    return _head;
}

bool List::isEmpty() const
{
    return (_head == 0);
}

int List::getNumItems() const
{
    return _numItems;
}
```

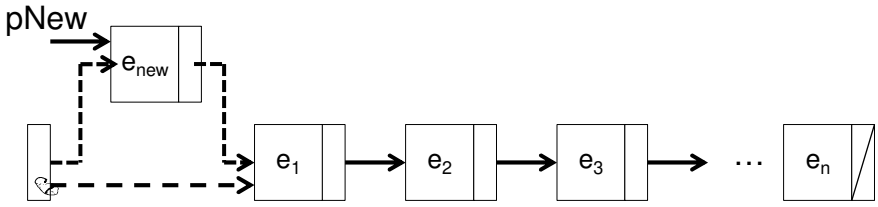


E148B

476

Classe List

- Inserimento in testa:




```
void List::insertFront(int data)
{
    Node* pNew = new Node(data, _head);

    _head = pNew;
    _numItems++;
}
```



E148B

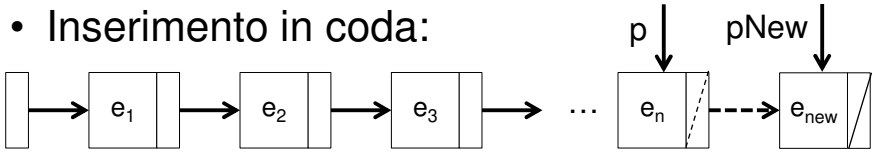
Creare un nuovo nodo:

- Il nuovo nodo punterà all'attuale inizio della lista
- Il puntatore al nuovo nodo diventa il nuovo inizio della lista.

477

Classe List

- Inserimento in coda:




```
void List::insertBack(int data)
{
    if (_head != 0)
    {
        Node* pNew = new Node(data, 0);
        Node* p = _head;

        while (p->getNext() != 0) p = p->getNext();

         p->setNext(pNew);
        _numItems++;

    }
    else
        insertFront(data);
}
```

E148b

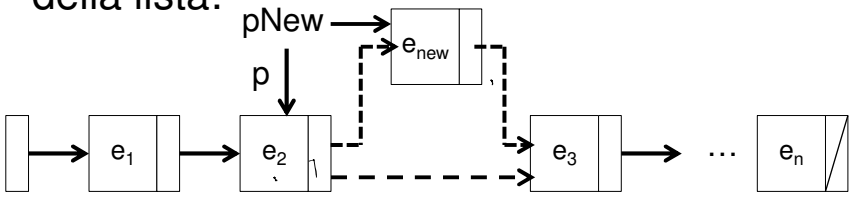
Se la lista non è vuota:
- Crea un nuovo nodo
- Scorri la lista fino all'ultimo elemento
- Inserisci il nuovo nodo come ultimo elemento

Altrimenti:
- Crea un nuovo nodo e impostalo come primo elemento della lista

479

Classe List

- Inserimento dopo un dato nodo al centro della lista:




```
void List::insertAfter(Node* p, int data)
{
    if (p != 0)
    {
        Node* pNew = new Node(data, p->getNext());

    p->setNext(pNew);

        _numItems++;
    }
}
```

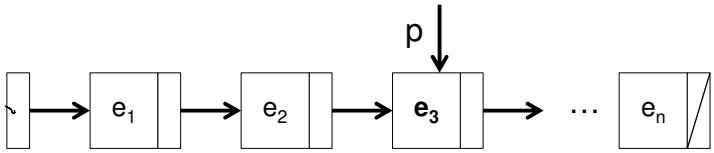
E148B

Inserire un nuovo elemento dopo il nodo puntato da p:
- Crea un nuovo nodo
- Il nuovo nodo punterà al nodo successivo a p
- Il nodo *p* punterà al nuovo nodo

480

Classe *List*

- Attraversamento della lista per cercare un elemento avente un dato valore:




```
Node* List::find(int data) const
{
    Node* p = _head;
    while (p != 0)
    {
        if (p->getData() == data)
            return p;

        p = p->getNext();
    }
    return 0;
}
```



E148B

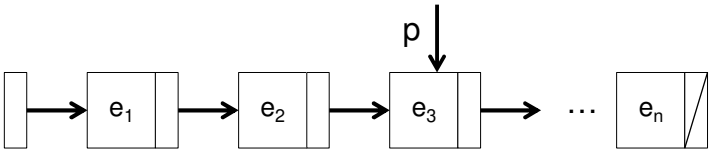
Scorrere la lista per:

- Trovare un elemento che corrisponde al parametro in ingresso
- Se l'elemento non viene trovato, restituire NULL

481

Classe *List*

- Attraversamento della lista per stampare i valori dei suoi elementi:




```
void List::print() const
{
    Node* p = _head;

    while (p != 0)
    {
        cout << p->getData() << ", ";
        p = p->getNext();
    }
}
```



E148B

Scorrere la lista:

- stampare i valori su console

482

Un semplice *main()*

```
int main() {
   List l;

   l.insertFront(10);

   l.insertFront(20);

   l.print();  cout << endl;

   l.insertBack(120);

   l.print(); cout << endl;

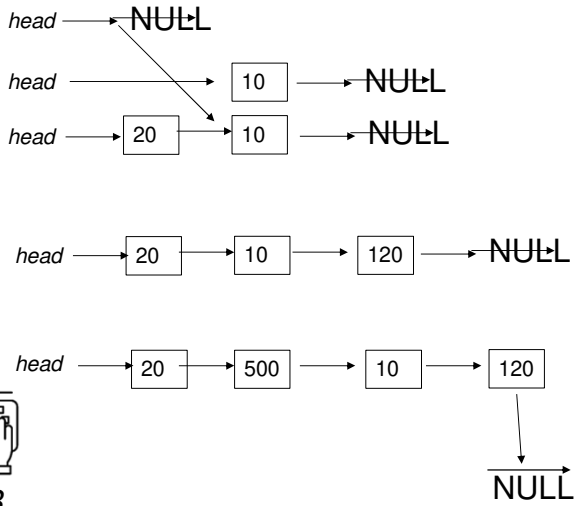
   l.insertAfter(l.find(20), 500);

   l.print(); cout << endl;

   return 0;
}
```



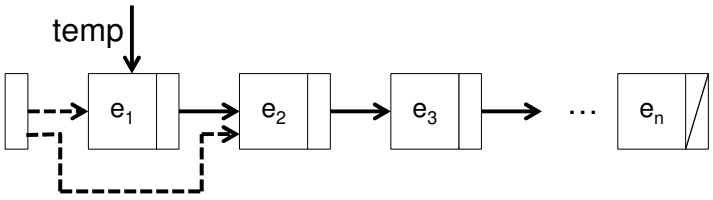
E148B



483

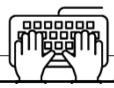
Classe List

- Rimozione in testa:




```
void List::removeFront()
{
    if (_head != 0)
    {
        Node* temp = _head;
        _head = _head->getNext();

        delete temp;
        _numItems--;
    }
}
```



E148C

Per eliminare la testa della lista:

- Il nuovo inizio della lista punterà all'attuale secondo elemento
- Viene cancellato il primo elemento

484

Classe List

- Rimozione in coda:

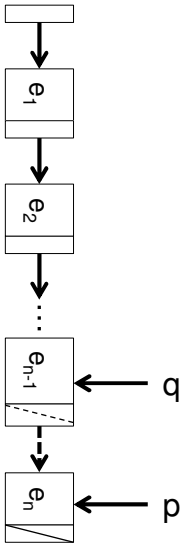
```
void List::removeBack()
{
    if (_head != 0)
    {
        if (_head->getNext() != 0)
        {
            Node* p = _head->getNext();
            Node* q = _head;
            while (p->getNext() != 0)
            {
                q = q->getNext();
                p = p->getNext();
            }
            q->setNext(0);
            delete p;
        }
        else
        {
            delete _head;
            _head = 0;
        }
        _numItems--;
    }
}
```

Se c'è più di 1 elemento..

Se c'è solo 1 elemento..



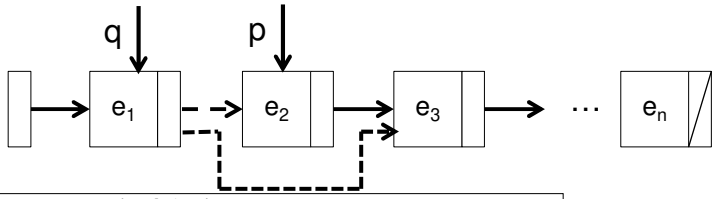
E148C



485

Classe List

- Rimozione di un nodo al centro della lista:




```
void List::removeAt(Node* p)
{
    if (_head != 0)
    {
        Node* q = _head;
        if (q == p) removeFront();
        else
        {
            while (q->getNext() != 0 && q->getNext() != p)
                q = q->getNext();

            if (q->getNext() != 0)
            {
                q->setNext(p->getNext());
                delete p;
                _numItems--;
            }
        }
    }
```



E148c

Verifica se la lista è vuota: Se la testa non è nulla, continua.
Rimuove il primo nodo se p è la testa, chiamando removeFront().

Cerca p nella lista, aggiorna il nodo precedente (q) per saltare p, poi cancella p

Decrementa il conteggio degli elementi.

486

Un semplice *main()*

```
int main() {
    List l;
    l.insertFront(10);
    l.insertFront(20);
    l.insertBack(120);
    l.insertAfter(l.find(20), 500);

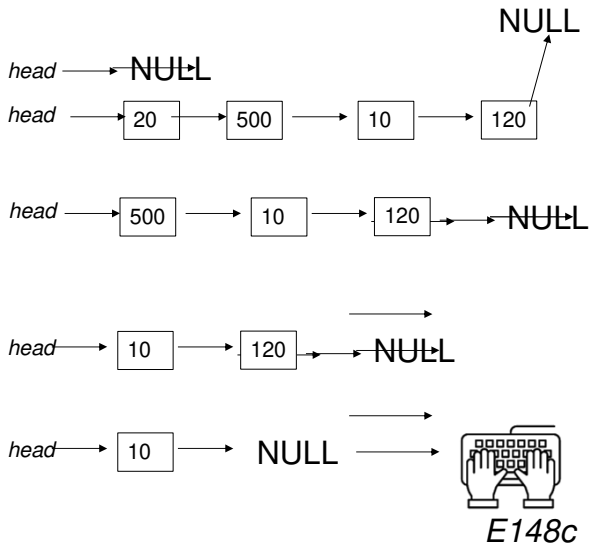
    l.removeAt(l.find(20));

    l.print();
    cout << endl;

    l.removeFront();
    l.print();
    cout << endl;

    l.removeBack();
    l.print();  cout << endl;

    return 0;
}
```



E148c

487

Esempio 2. Implementazione di una lista semplice per un tipo generico T

488

Classe Node

```
template <typename T>
class Node {
public:
      // costruttori e distruttore
      Node();
      Node(const T& data, Node<T>* next);
      Node(const Node<T>& node);
      ~Node();
      // selettori
      const T& getData() const;
      Node<T>* getNext() const;
      // modificatori
      void setData(const T& data);
      void setNext(Node<T>* next);
private:
      T _data;
      Node<T>* _next;
};
```



E150A

489

Classe Node

- Costruttori e distruttore:

```
template <typename T>
Node<T>::Node() : _data(), _next(0) {}

template <typename T>
Node<T>::Node(const T& data, Node<T>* next)
      : _data(data), _next(next) {}

template <typename T>
Node<T>::Node(const Node<T>& node)
      : _data(node._data), _next(node._next) {}

template <typename T>
Node<T>::~Node() {}
```



E150A

490

Classe Node

- Selettori e modificatori:

```
template <typename T>
const T& Node<T>::getData() const {
      return _data;
}
template <typename T>
Node<T>* Node<T>::getNext() const {
      return _next;
}
template <typename T>
void Node<T>::setData(const T& data){
      _data = data;
}
template <typename T>
void Node<T>::setNext(Node<T>* next){
      _next = next;
}
```



E150A

491

- Attenzione: quando si usa le template di classe, interfaccie e implementazione delle metodi devono stare nello stesso file .h (in questo caso Node.h)



E150A

492



Classe List

- Interfaccia pubblica:

```
template <typename T>
class List {
public:
      // costruttori e distruttore
      List();
      List(const List<T>& l);
      ~List();

      // selettori per gli attributi
      int isEmpty() const;
      int numItems() const;

      Node<T>* getHead() const;

      // inserimento di nuovi nodi
      void insertFront(const T& data);
      void insertBack(const T& data);
      void insertAfter(Node<T>* p, const T& data);
```



E150B

493



Classe List

- Interfaccia pubblica e parte privata:

```
      ...
      // rimozione di nodi
      void removeFront();
      void removeBack();
      void removeAt(Node<T>* p);

      // operazioni

      void clear();
      Node<T>* find(const T& data) const;
      void print() const;

private:
      Node<T>* _head;
      int _numItems;
};
```



E150B

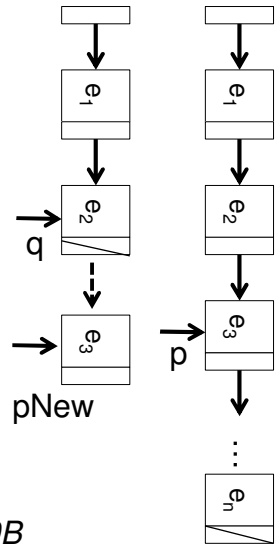
494

Classe List

- Implementazione dei costruttori incluso un costruttore di copia:

```
template <typename T>
List<T>::List()
: _head(0), _numItems(0) {}


template <typename T>
List<T>::List(const List<T>& l)
: _head(0), _numItems(l._numItems)
{
   if (l._head != 0) {
       Node<T>* p = l.getHead();
       _head = new Node<T>(*p);
       Node<T>* q = _head;
       p = p->getNext();
       while (p != 0) {
          Node<T>* pNew =
           new Node<T>(p->getData(),0);
          q->setNext(pNew);
          q = pNew;
          p = p->getNext();
       }
   }
}
```





E150B

495

Classe List

- Distruttore e selettori per gli attributi:

```
template <typename T>
List<T>::~List() {
      clear();
      _head = 0;
      _numItems = 0;
}

template <typename T>
int List<T>::isEmpty() const {
      return (_head == 0);
}

template <typename T>
int List<T>::numItems() const {
      return _numItems;
}
```

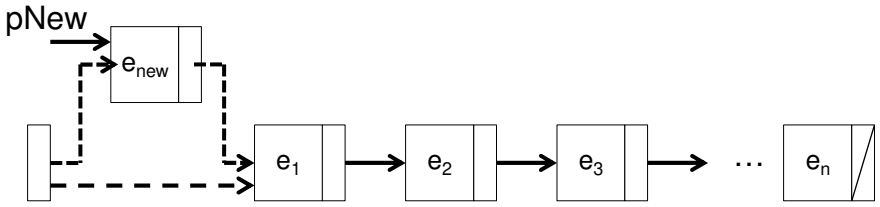


E150B

496

Classe List

- Inserimento in testa:




```
template <typename T>
void List<T>::insertFront(const T& data)
{
      Node<T>* pNew = new Node<T>(data, _head);
      _head = pNew;
      _numItems++;
}
```

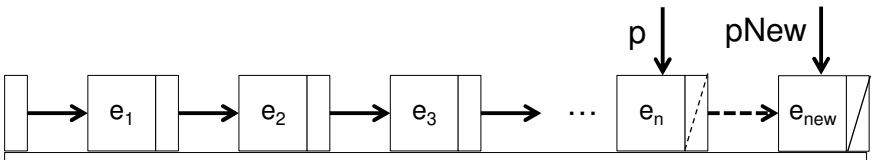


E150B

497

Classe List

- Inserimento in coda:




```
template <typename T>
void List<T>::insertBack(const T& data){
      if (_head != 0) {
            Node<T>* pNew = new Node<T>(data, 0);
            Node<T>* p = _head;
            while (p->getNext() != 0)
                  p = p->getNext();
            p->setNext(pNew);
            _numItems++;
      }
      else
            insertFront(data);
}
```

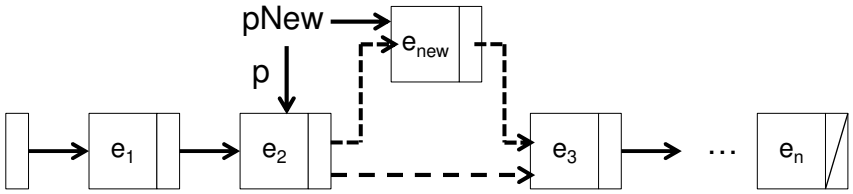


E150B

498

Classe List

- Inserimento dopo un dato nodo al centro della lista:




```
template <typename T>
void List<T>::insertAfter(Node<T>* p, const T& data)
{
   if (p != 0) {
      Node<T>* pNew = new Node<T>(data, p->getNext());
      p->setNext(pNew);
      _numItems++;
   }
}
```

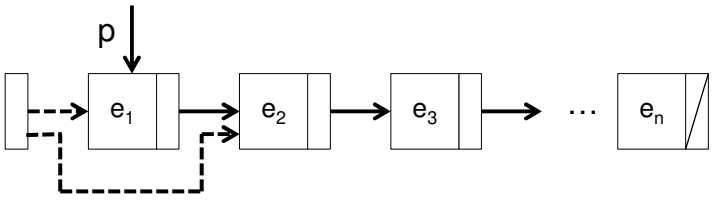


E150B

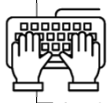
499

Classe List

- Rimozione in testa:




```
template <typename T>
void List<T>::removeFront() {
      if (_head != 0) {
            Node<T>* p = _head;
            _head = _head->getNext();
            delete p;
            _numItems--;
      }
}
```



E150B

500

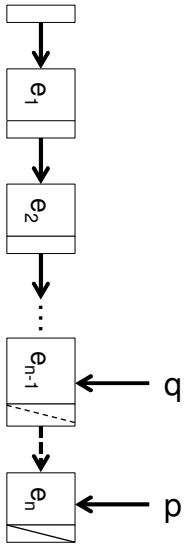
Classe List

- Rimozione in coda:

```
template <typename T>
void List<T>::removeBack() {
   if (_head != 0) {
       if (_head->getNext() != 0) {
           Node<T>* p = _head->getNext();
           Node<T>* q = _head;
           while (p->getNext() != 0) {
              q = q->getNext();
              p = p->getNext();
           }
           q->setNext(0);
           delete p;
       } else {
              delete _head;
              _head = 0;
       }
       _numItems--;
   }
}
```

Se c'è più di 1 elemento..

Se c'è solo 1 elemento..

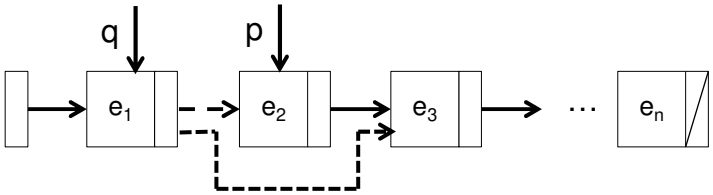


E150B

501

Classe List

- Rimozione di un nodo al centro della lista:



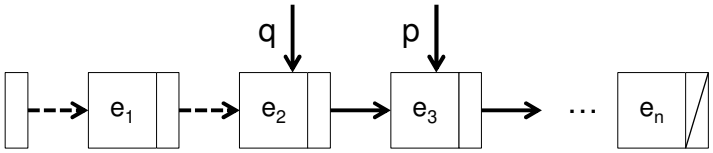

```
template <typename T>
void List<T>::removeAt(Node<T>* p) {
   if (_head != 0) {
       Node<T>* q = _head;
       if (q == p) removeFront();
       else {  while (q->getNext() != 0 && q->getNext() != p)
                     q = q->getNext();
               if (q->getNext() != 0) {
                     q->setNext(p->getNext());
                     delete p; _numItems--;
               }
       }
   }
}
```

E150B

502

Classe List

- Attraversamento della lista per rimuovere tutti gli elementi:




```
template <typename T>
void List<T>::clear(){
       Node<T>* p = getHead();
       while (p != 0) {
              Node<T>* q = p;
              p = p->getNext();
              delete q;
       }
       _head = 0;
       _numItems = 0;
}
```

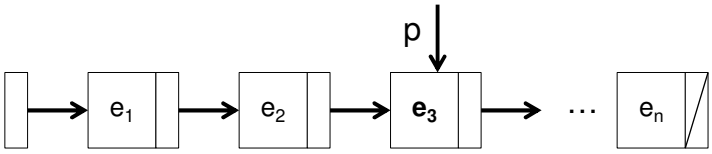


E150

503

Classe List

- Attraversamento della lista per cercare un elemento avente un dato valore:




```
template <typename T>
Node<T>* List<T>::find(const T& data) const {
       Node<T>* p = _head;
       while (p != 0) {
              if (p->getData() == data)
                     return p;
              p = p->getNext();
       }
       return 0;
}
```



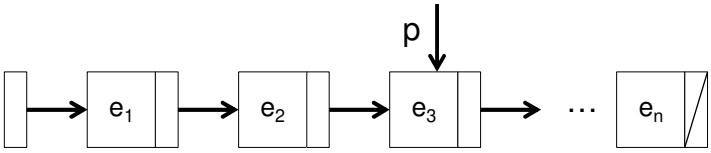
E150B

504



Classe List

- Attraversamento della lista per stampare i valori dei suoi elementi:




```
template <typename T>
void List<T>::print() const {
       Node<T>* p = _head;
       if (p != 0) {
              cout << p->getData();
              p = p->getNext();
       }
       while (p != 0) {
              cout << ", " << p->getData();
              p = p->getNext();
       }
}
```

La funzione stampa:

$e_1, e_2, e_3, \ldots, e_n$

505



Classe List

- Esempio di utilizzo dell'ADT lista:

```
#include "List.h"
int main() {
      List<int> l; // dichiarazione di oggetto lista
      // inserimento di nodi
      l.insertFront(10);
      l.insertFront(20);
      l.insertBack(120);
      l.insertAfter(l.find(20), 500);
      l.print(); // stampa: 20, 500, 10, 120
      // rimozione di nodi
      l.removeAt(l.find(20));
      l.removeFront();
      l.removeBack();
      l.print(); // stampa: 10
      return 0;  // distruzione della lista
}
```



E150B

506



Classe List

- Che cosa devo aggiungere al codice per poter creare una lista dei Film?

```
int main() {
   List<film> l;
   l.insertFront(film("Matrix",120,1999));
   l.insertFront(film("Pirates",125,2010));
   l.print();
   l.insertBack(film("Godfather",145,1975));
   l.print();
   l.insertAfter(l.find(film("Pirates",125,2010)), film("Jaws",125,2015));
   l.print();
   return 0;
}
```

507



La «vecchia» classe *Film (E087)*

```
class film {
   char* _titolo;
   int _durata;
   int _anno;

public:
   film();
   film(char* titolo, int durata, int anno);
   film(const film& otherFilm);
    ~film();

   char* getTitolo() const;
   void setTitolo(const char* titolo);
   int getDurata() const;
   void setDurata(int durata);
   int getAnno() const;
   void setAnno(int anno);
   void stampaFilm();
};
```

```
int main() {
   int lunghezzaTitolo;
   cout << "Inserisci la lunghezza del titolo del film: ";
   cin >> lunghezzaTitolo;

   char* titolo = new char[lunghezzaTitolo + 1];

   cout << "Inserisci il titolo del film (mass. " << lunghezzaT
   cin >> titolo;

   int durata, anno;
   cout << "Inserisci durata: ";
   cin >> durata;
   cout << "Inserisci anno: ";
   cin >> anno;

   film p(titolo, durata, anno);
   p.stampaFilm();

   delete[] titolo;
   return 0;
}
```

508



La «vecchia» classe *Film (E087)*

```
film::film() {
   _titolo = new char[20];
   strcpy(_titolo, "N/D");
   _durata = 0;
   _anno = 0;}

film::film(char* titolo, int durata, int anno) {
   _titolo = new char[strlen(titolo) + 1];
   strcpy(_titolo, titolo);
   _durata = durata;
   _anno = anno;}

film::film(const film& otherFilm) {
   _titolo = new char[strlen(otherFilm._titolo) + 1];
   strcpy(_titolo, otherFilm._titolo);
   _durata = otherFilm._durata;
   _anno = otherFilm._anno;}

film::~film() {
   delete[] _titolo;}

char* film::getTitolo() const {
   return _titolo;}

void film::setTitolo(const char* titolo) {
   delete[] _titolo;
   _titolo = new char[strlen(titolo) + 1];
   strcpy(_titolo, titolo);}

int film::getDurata() const {
   return _durata;}

void film::setDurata(int durata) {
   _durata = durata;}

int film::getAnno() const {
   return _anno;}

void film::setAnno(int anno) {
   _anno = anno;}

void film::stampaFilm() {
   cout << "Titolo: " << _titolo << endl;
   cout << "Durata: " << _durata << endl;
   cout << "Anno: " << _anno << endl;
}
```

E087

509



Bisogna aggiungere l'overloading di operatori ==, >>, <<

```
bool film::operator==(const film& film1) const {
   // Compare titolo, durata, e anno di due film
   return (strcmp(this->getTitolo(), film1.getTitolo()) == 0 &&
         this->getDurata() == film1.getDurata() &&
         this->getAnno() == film1.getAnno());
}

ostream& operator<<(ostream& os, const film& film) {
   os << "Titolo: " << film._titolo << ", Durata: " << film._durata << " minuti, Anno: " <<
film._anno;
   return os; }

istream& operator>>(istream& is, film& film) {
   char temp[100];
   is >> temp;
   film._titolo = new char[strlen(temp) + 1];
   strcpy(film._titolo, temp);
   is >> film._durata >> film._anno;
   return is; }
```



E151

510

Il main()

```
int main()
{
   List<film> l; l.insertFront(film("Matrix",120,1999));
   l.insertFront(film("Pirates",125,2010));

   l.print(); cout<<endl;

   l.insertBack(film("Godfather",145,1975));

   l.print(); cout<<endl;

   l.insertAfter(l.find(film("Pirates",125,2010)),
film("Pirates2",125,2015));
   l.print(); cout<<endl;

   l.removeFront();
   l.print(); cout<<endl;

   l.removeBack();
   l.print(); cout<<endl;

   return 0; // distruzione della lista
}
```



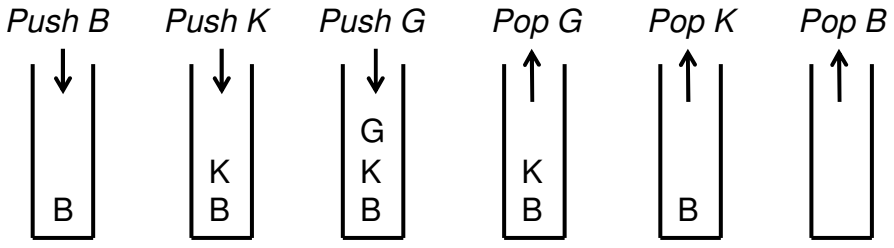
E151

- Per trovare un nodo bisogna creare un oggetto «eguale»
- Questa soluzione non è ottima se ci sono tanti attributi
- Meglio avere un identificatore unico per ogni oggetto, p.e., codice fiscale, ISBN, per fare la ricerca

511

Pile (stack)

- Una pila (in inglese stack) è una lista nella quale si inseriscono o si rimuovono elementi solo dalla testa.
- Tale politica di gestione è detta Last In First Out (LIFO), cioè l'ultimo elemento ad essere inserito è il primo ad essere rimosso.



512



Operazioni sulle pile

- Un elenco (non esaustivo) di operazioni applicabili alle pile include:
  - Creazione, distruzione e copia
  - Verifica di proprietà di una pila: ad esempio se una pila è vuota o di quanti nodi è composta
  - Inserimento di un nuovo nodo (Push)
  - Rimozione di un nodo (Pop)
  - Ottenimento dell'informazione contenuta nel primo elemento della pila
  - Svuotamento della pila (rimozione di tutti i nodi).

513



Interfaccia della classe Pila

```
template <typename T>
class Stack {
public:
      // costruttori e distruttore
      Stack();
      Stack(const Stack<T>& s);
      ~Stack();

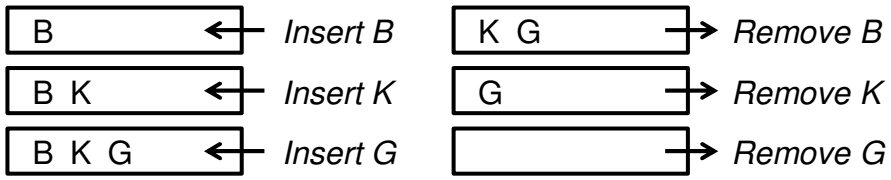
      // selettori per gli attributi
      int isEmpty() const;
      int numItems() const;

      // operazioni
      void push(const T& data);
      void pop();
      Node<T>* getHead() const;
      void clear();
private:
      Node<T>* _head;
      int _numItems;
};
```

514

Code (queue)

- Una coda (in inglese queue) è una lista nella quale gli elementi si inseriscono solo in coda e si rimuovono solo in testa.
- Tale politica di gestione è detta First In First Out (FIFO), cioè il primo elemento ad essere inserito è anche il primo ad essere rimosso.



515

Operazioni sulle code

- Un elenco (non esaustivo) di operazioni applicabili alle code include:
  - Creazione, distruzione e copia
  - Verifica di proprietà di una coda: ad esempio se una coda è vuota o di quanti nodi è composta
  - Inserimento di un nuovo nodo
  - Rimozione di un nodo
  - Ottenimento dell'informazione contenuta nell'elemento in testa alla coda
  - Svuotamento della coda (rimozione di tutti i nodi).

516



Interfaccia della classe Coda

```
template <typename T>
class Queue {
public:
      // costruttori e distruttore
      Queue();
      Queue(const Queue<T>& q);
      ~Queue();

      // selettori per gli attributi
      int isEmpty() const;
      int numItems() const;

      // operazioni
      void insert(const T& data);
      void remove();
      Node<T>* getHead() const;
      void clear();
private:
      Node<T>* _head; Node<T>* _tail;
      int _numItems;
};
```

517



FINE

518